

The Archer Documentation

Welcome

Take a look at The Archer - Roguelike Documentation

The Archer Roguelike is a complete 3D wave/room-based roguelike game template. It takes inspiration from the greatest games in the genre like Hades and Archero. Designed for developers at most skill levels, from beginners to experts. Super modular and customizable, packed with features like player moveset, abilities, enemies and bosses with unique behavior, stage creator, and many more features - including original art.

Introduction

Tools and Options

The template provides **three main tools: Stage Creator, Abilities Tester, and Pipeline Switcher.**

- [Stage Creator](#) - A powerful editor tool that gives you full control over stages, rooms, and waves.
You can create, shuffle, and remove them, add, position, or delete enemies, chests, and props, calculate gold and XP rewards for each wave, and even playtest specific rooms or waves without having to replay the entire stage.
- [Abilities Tester](#) - A tool that allows you to create ability presets to start with in both the Editor and builds. It also lets you toggle abilities on and off during gameplay directly in the Editor, making ability balancing and testing much faster.
- [Pipeline Switcher](#) - A simple tool that enables quick switching from the **Built-in Render Pipeline** to **URP** with a single click.

You can access all tools from the Unity menu at **Tools → October**, or use the **Shift + S** shortcut for **Stage Creator** and **Shift + A** shortcut for **Abilities Tester**.

- ✓ There, while in the game, you can also grant additional Gold, open all stages, and delete the save file.

Get 10K Gold	
Get 1K Gold	
Open All Stages	
Delete Save File	
Abilities Tester	Shift+A
Pipeline Switcher	
Stage Creator	Shift+S
Readme	

October Tools menu

Scenes

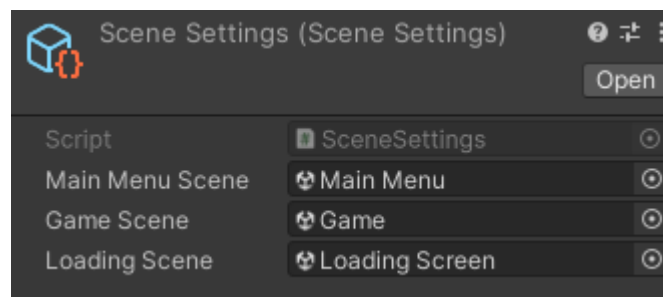
The game includes **three main scenes: Main Menu, Game, and Loading Screen.**

- **Main Menu** - A UI-based scene that contains the **Stage Selector, Armory, and Upgrades** pages.
- **Game** - The main gameplay scene where all core game logic and interactions take place.
- **Loading Screen** - A lightweight scene used for transitioning between the **Main Menu** and **Game** scenes.

When you click **Play** in the Unity Editor, the asset's custom tool automatically overrides the starting scene to **Main Menu**.

Default scenes

If you want to replace the default scenes with your own, you can do so using the **Scene Settings** asset located in: **Assets/The Archer/Scriptables**



Scene Setting asset

General Tips

Extending the project

Most classes in the project are **inheritance-friendly**, exposing **virtual** and **protected** methods for easy extension and modification.

It is **recommended** to create new classes that **inherit** from existing ones and **override** their methods, rather than editing the originals directly.

The same is true for Scriptable files, Scenes, and Prefabs. Ideally, it's best to create your own files so that an update won't overwrite them.

This approach ensures your custom functionality remains intact when updating the template to newer versions.

Updating the project

It is **strongly recommended** to use a **version control system** (such as git) throughout development.

Before updating to a new version of the template, **commit all your changes**, then **download the updated version**.

Carefully **review and merge changes** using your version control tool to identify and resolve any potential conflicts or compatibility issues.

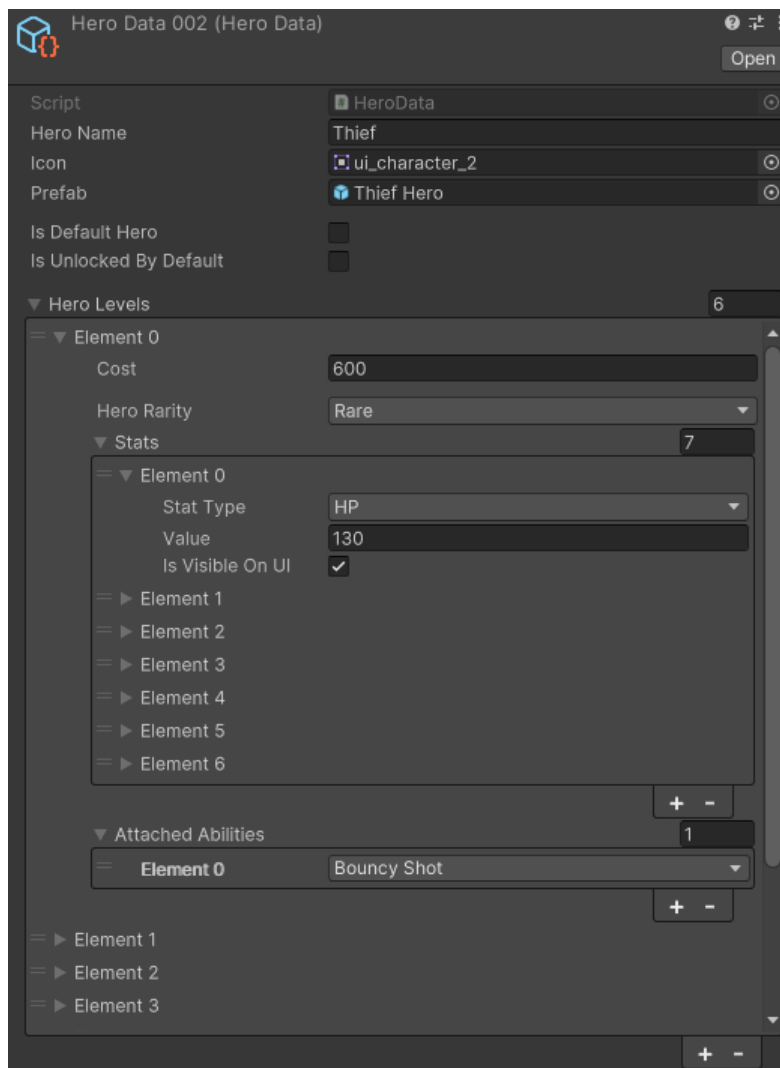
Heroes

Heroes Overview

Heroes are the playable characters in the game.

The template includes **two heroes** by default: **Archer** and **Thief**. Heroes can be **selected**, **purchased**, and **upgraded** in the **Heroes Tab** of the **Armory Page** within the **Main Menu**.

Hero Data



Hero Data asset example

- **Hero Name, Hero Icon** – Self-explanatory. These values are displayed on the **Armory Page** UI.
- **Prefab** – The hero prefab containing the model, animator, visual effects, and helper components.

- **Is Default Hero** - If enabled, this hero will be selected automatically when the player launches the game for the first time.
- **Is Unlocked by Default** - If enabled, the hero will be available from the start without requiring a purchase.
- **Hero Levels** - A list of upgrade levels that players can purchase to increase stats and unlock starting abilities. Each level contains the following fields:
 - **Cost** - The amount of gold required to upgrade the hero to this level.
 - **Hero Rarity** - **Common**, **Rare**, **Mystic**, or **Legendary**. Each rarity has unique UI visuals and gives players a stronger sense of progression.
 - **Stats** - The hero's base attributes (HP, Damage, etc.). These form the foundation of the player's total stats, combined with bonuses from equipped items. Abilities and upgrades typically apply multipliers to these values.
 - **Attached Abilities** - The abilities that the hero starts with at the beginning of a stage.



Note: Hero Level values do **not** stack.

For example, if Level 1 grants **100 HP** and Level 2 grants **120 HP**, the hero's HP will be **120**, not 220.

Likewise, if a stat or ability exists at Level 1 but not at Level 2, it will be **lost** upon upgrading.

Hero Prefab Structure

Each hero is represented by a **prefab** that defines its components, visuals, and gameplay behavior.

Here's an overview of the hero prefab structure:

- **Root GameObject** - Contains the [HeroBehavior](#) component, which controls the hero's core logic and interactions.
- **Character Model** - The 3D model of the hero, with an **Animator** and a **HeroEventsHandler** component.
- **Weapon Parents** - Two GameObjects, one for each hand, that act as anchors for weapons. When initialized, the active weapon is placed in one of them.
- **Flying Weapon Holder** - A component that manages the weapon when the **Flying Weapon** ability is active.

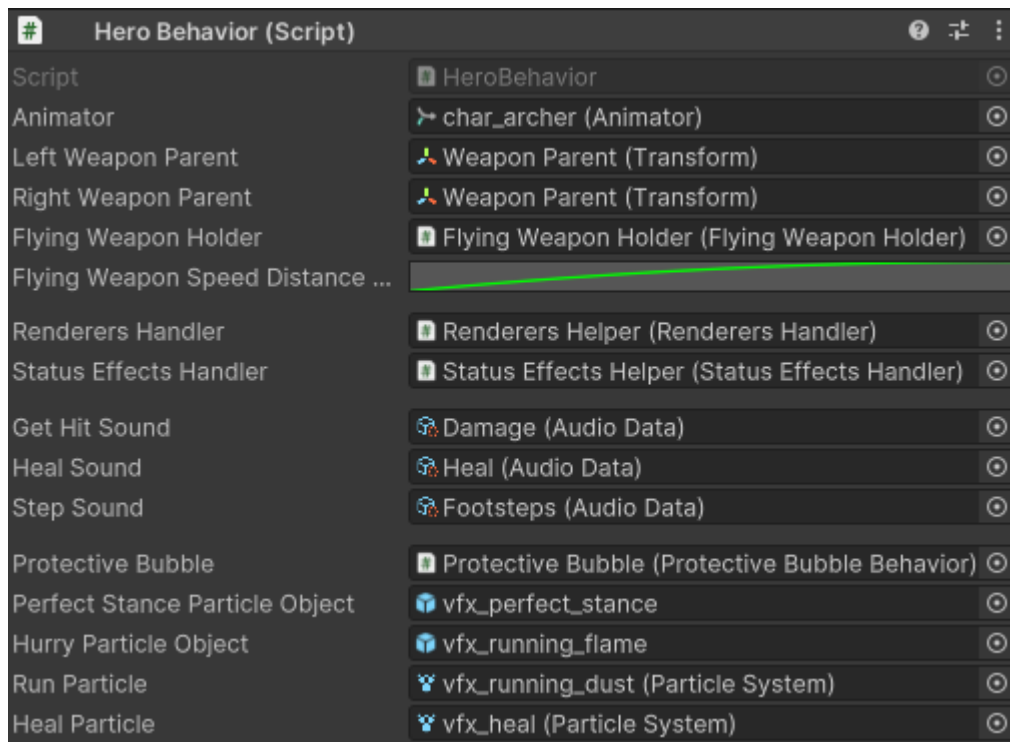
- [Status Effects Helper](#) - Defines how the hero responds to various status effects.
- **Renderers Helper** - Applies the rim visual effect to the hero's renderers.
- **Particles** - A collection of particles used for **healing**, **walking**, and **abilities** visuals.

 The Rim effect is only visible if the materials use **October.shader** or a shader that implements the rim effect in the same way.

Hero Behavior Structure

The **HeroBehavior** component implements the `IHeroBehavior` interface. All interactions between the hero and the rest of the project are routed through this interface, allowing you to create custom implementations of **HeroBehavior** without rewriting references across the project.

All methods in **HeroBehavior** are **virtual** and there are **no private methods**, making it highly extensible and easy to customize.



Hero Behavior Inspector

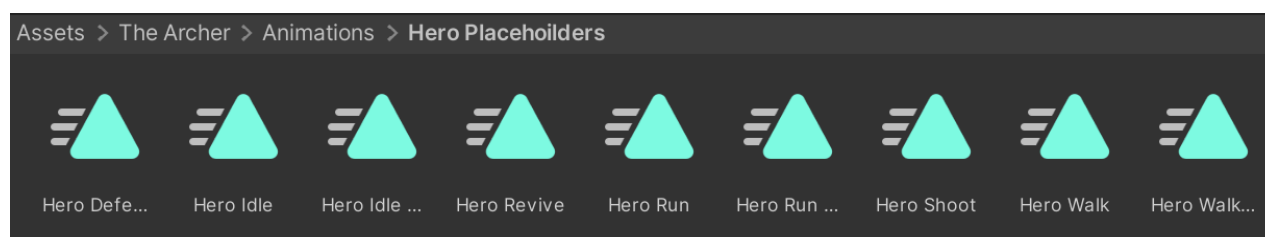
Most fields of **HeroBehavior** are references to components within the hero prefab, with a few notable exceptions:

- **Flying Weapon Speed Distance** - A speed curve that ensures smooth weapon movement while the **Flying Weapon** ability is active, keeping the weapon close to the player in all gameplay situations.
- **Get Hit Sound, Heal Sound, Step Sound** - References to **Audio Data assets** containing the respective sound effects.

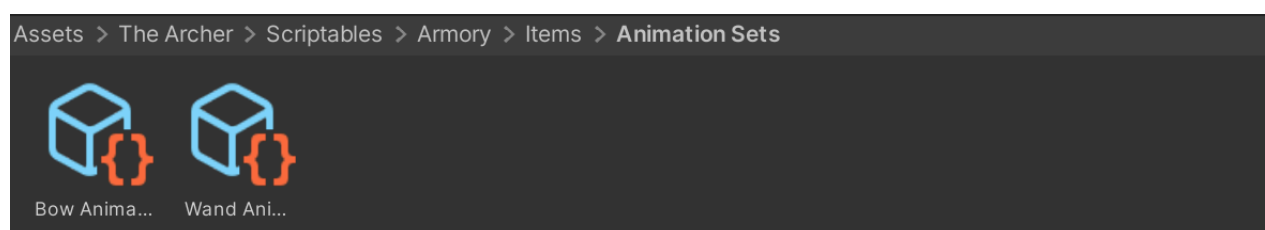
Hero Animations

To support a wide variety of weapon types, the template provides a **flexible animation set system**.

By default, all animations in the hero's Animator are **empty placeholders**:



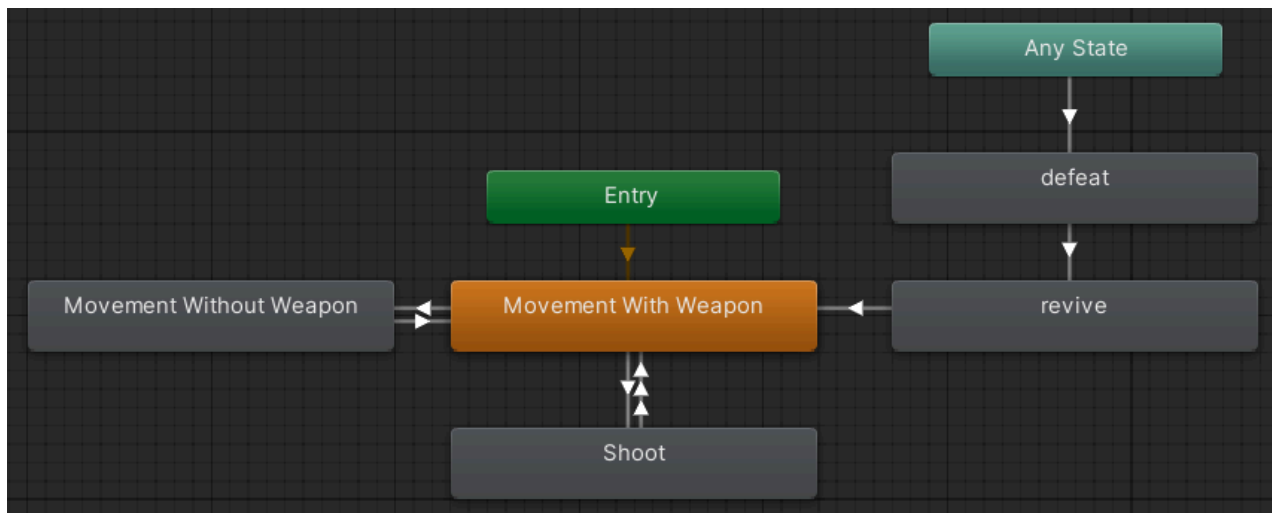
At runtime, the game replaces these placeholders with animations from an **animation set asset** specific to each **hero-weapon** combination.



You can assign these animation sets in the **Armory Database** asset.

	Signature Bow	Ceremonial Wand
Archer	Bow Anima	Wand Anim
Thief	Bow Anima	Wand Anim

Hero Animator



The animator has the following structure:

1. **Movement With Weapon** - A blend tree that handles *Idle*, *Walk*, and *Run* animations while the hero is holding a weapon.
2. **Movement Without Weapon** - A blend tree for *Idle*, *Walk*, and *Run* animations when the hero is unarmed.
3. **Shoot** - The animation of the hero firing a weapon. It transitions back to movement when the animation finishes or when the player starts moving. *Only plays if the Flying Weapon ability is **not** active.*
4. **Defeat** - The hero's death animation.
5. **Revive** - The animation played when the hero revives.

Hero Events Handler

The [HeroBehavior](#) component responds to several animation events:

- **Shoot** (from the *Shoot* animation) - Spawns a projectile.
- **OnDefeatAnimationEnded** (from the *Defeat* animation) - Notifies the hero that the defeat animation has finished.
- **OnReviveAnimationEnded** (from the *Revive* animation) - Notifies the hero that the revive animation has finished.
- **Step** (from the *Walk* and *Run* animations) - Triggers step-related visual and audio effects.

Player Stats

A large part of the player's behavior - damage, attack speed, movement speed, dodge chance, and more - is controlled through **Stats**.

A Stat is a value with an initial base amount that can be modified by **multipliers** or **adders** applied by abilities, upgrades, status effects, and other systems.

There are **two types of Stats**:

Multiplicative Stat

- Stores **float** values.
- The initial value is usually **1**.
- Contains a list of **Stat Multipliers**.
- Its final value is calculated by multiplying the initial value by each multiplier in the list.

Example:

Initial damage = 20

Multipliers = 1.1 and 1.2

Final value = $20 \times 1.1 \times 1.2 = 26.4$

Additive Stat

- Stores **int** values.
- The initial value is usually **0**.
- Contains a list of **Stat Adders**.
- Its final value is calculated by adding all adders to the initial value.

Example:

Initial dodge chance = 1%

Adders = +2% and +3%

Final value = $1 + 2 + 3 = 6\%$

Stat Events and Automatic Updates

- Each Stat has an **onStatChanged** event, fired whenever:
 - its initial value changes,
 - one of its multipliers or adders changes value,
 - a multiplier/adder is added or removed.
 - **StatMultipliers** and **StatAdders** also expose **onMultiplierChanged** and **onAdderChanged** events.
 - The Stat automatically listens to these events, so you **never need to manually recalculate** a Stat's value.
-

Example

If the player fires a front arrow using:

```
player.Stats.FrontArrowDamageStat
```

...and then an ability modifies a multiplier on that stat,

the next arrow automatically uses the newly recalculated damage value - no extra work required.

How to add a new Hero

Follow these steps to add a new hero to the project.

Importing assets

First, you'll need to import the following assets for your hero:

- **3D model** of the hero
- **Animations**
- **Texture**
- **Armory sprite (icon)**

Required animations:

- Idle
- Run
- Defeat
- Shoot (one for each weapon type)

Optional but recommended animations:

- Walking
- Walking Without Weapon
- Running Without Weapon
- Idle Without Weapon
- Revive

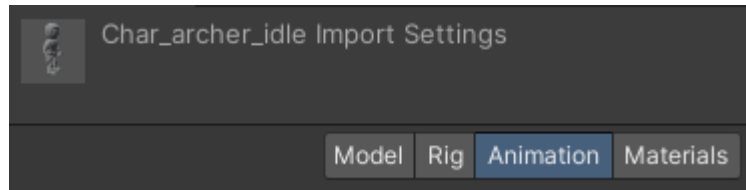
Folder setup

1. Inside **Assets/The Archer/Models/Heroes**, create a **new folder** named after your hero.
Import the 3D model and animation clips there.
2. Inside **Assets/The Archer/Textures/Heroes**, import your hero's texture.

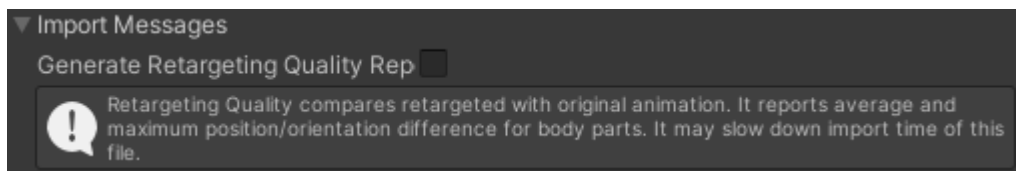
3. Inside **Assets/The Archer/Sprites/UI/Armory**, import your hero's sprite and set its **Texture Type** to **Sprite (2D and UI)** in the **Inspector**.

Configuring animations

1. Open each animation file and select its **Animation** tab.



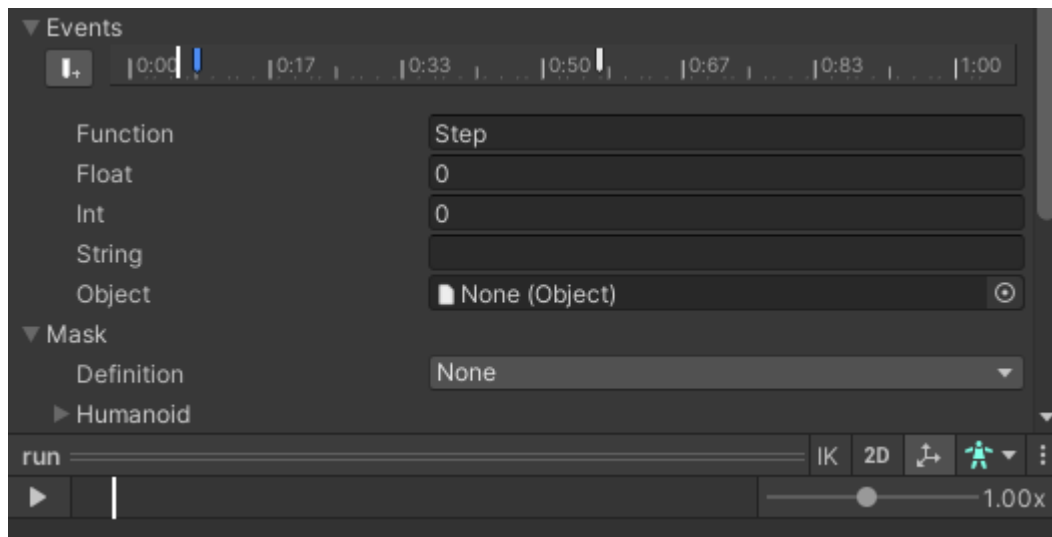
2. Ensure there are **no warnings or errors** under the **Import Messages** section.



3. For **Idle**, **Walking**, and **Running** animations, enable the **Loop Time** toggle.



4. For **Walking** and **Running** animations:
 - Open the **Events** section.
 - Add animation events whenever the hero's feet touch the ground.
 - Set the function name to **Step**.



- For the **Defeat** animation, add an event near the end of the animation with the function **OnDefeatAnimationEnded**.



- If you have a **Revive** animation, add **OnReviveAnimationEnded** near its end. If you don't have a revive animation, add that event near the **start** of the **Defeat** animation — it will be played in reverse to simulate revival.

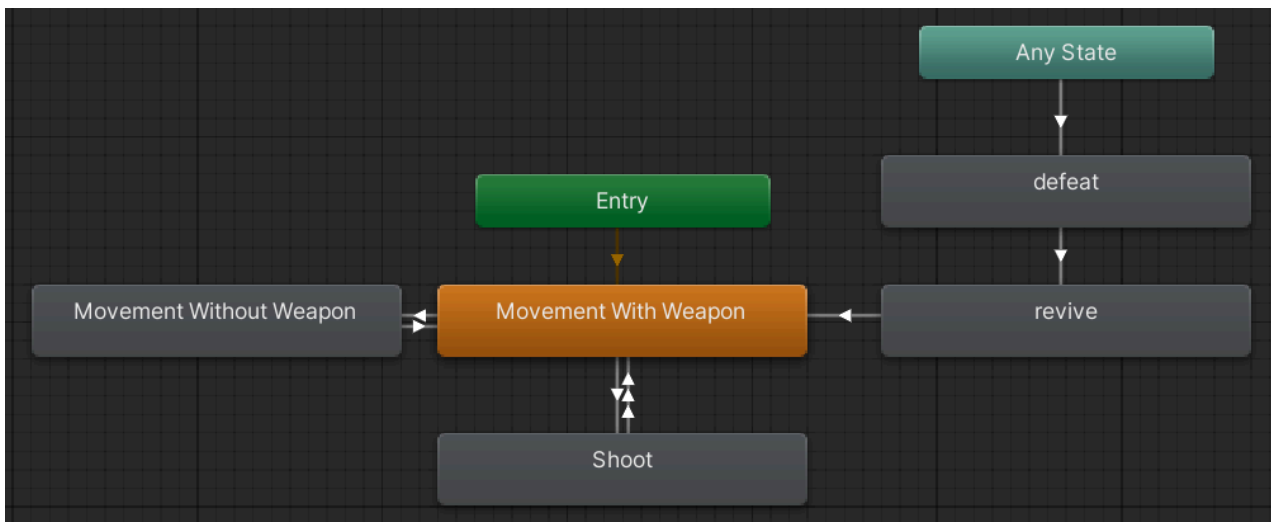


7. For every **Shoot** animation, add an event with the function **Shoot** at the frame where the projectile should be spawned.

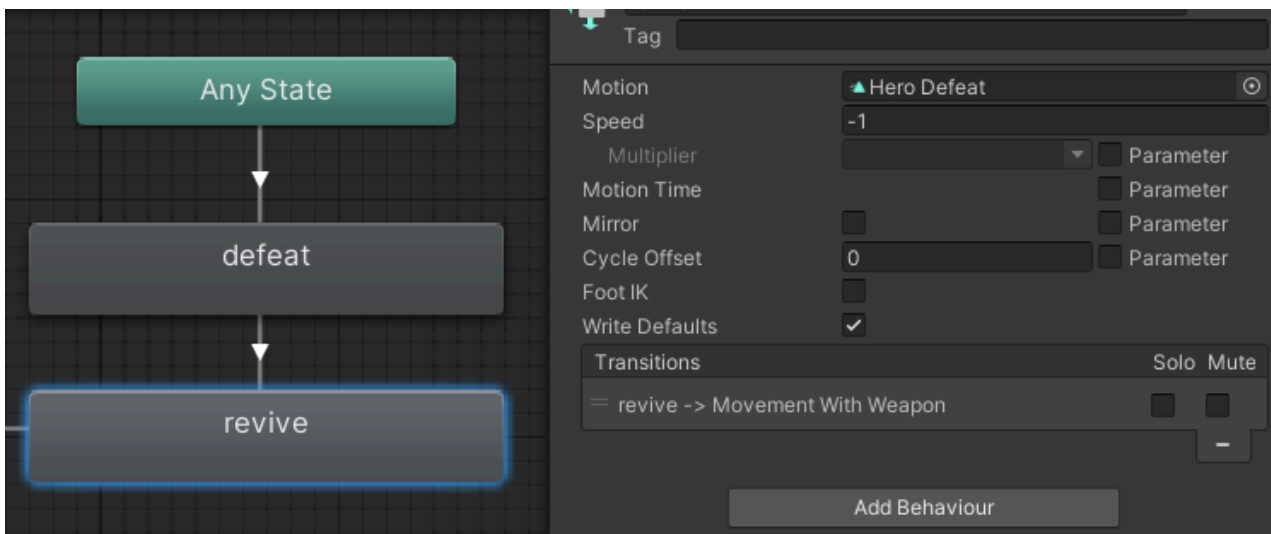
Animator Setup

As mentioned in the **Hero Overview**, actual animation clips are not stored inside the Hero Animator asset.

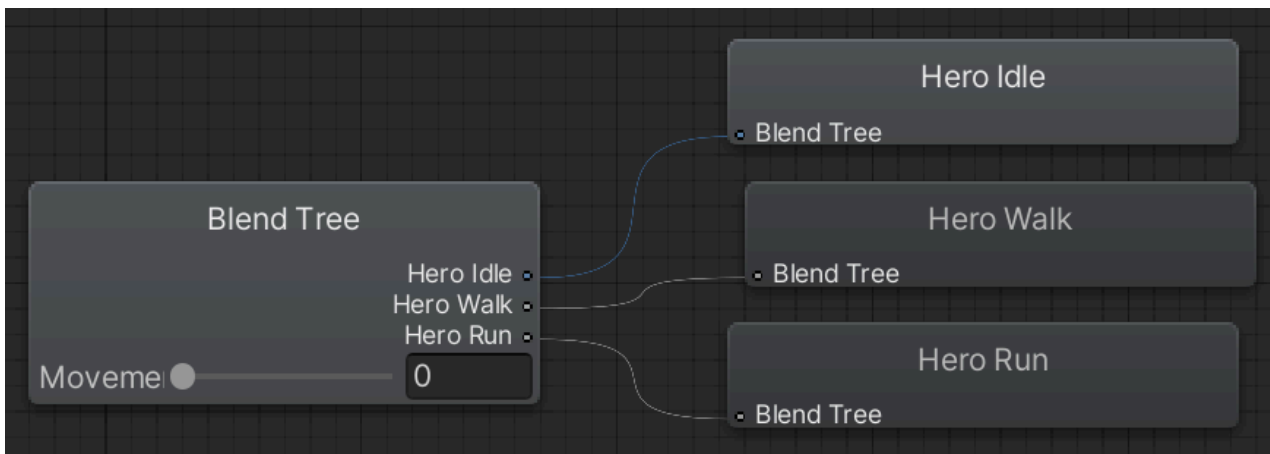
If you have only the standard animation set, you can skip this step.



1. **Clone** the base **Hero Animator**, rename it, and place it near your hero's model folder.
2. If you **don't have revive animation**:
 - Select the **Revive** state in the Animator.
 - In the **Motion** field, assign the **Hero Defeat** clip.
 - Set its **Speed** to **-1** (to play it in reverse).

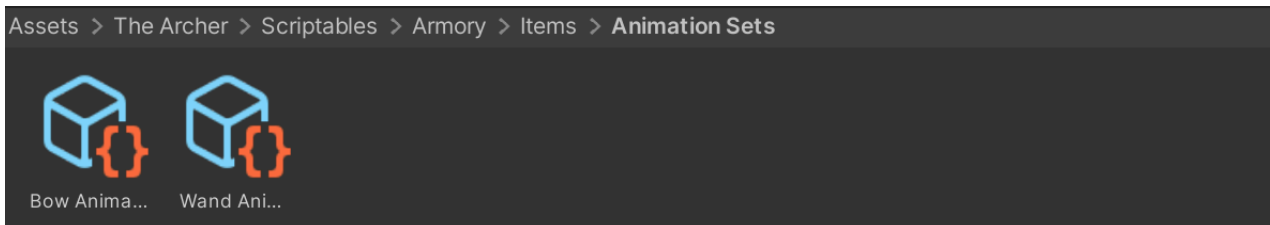


3. Movement with and without weapon are **blend trees** of three animations.
If you don't have a **Walking** animation, remove it from the blend tree.



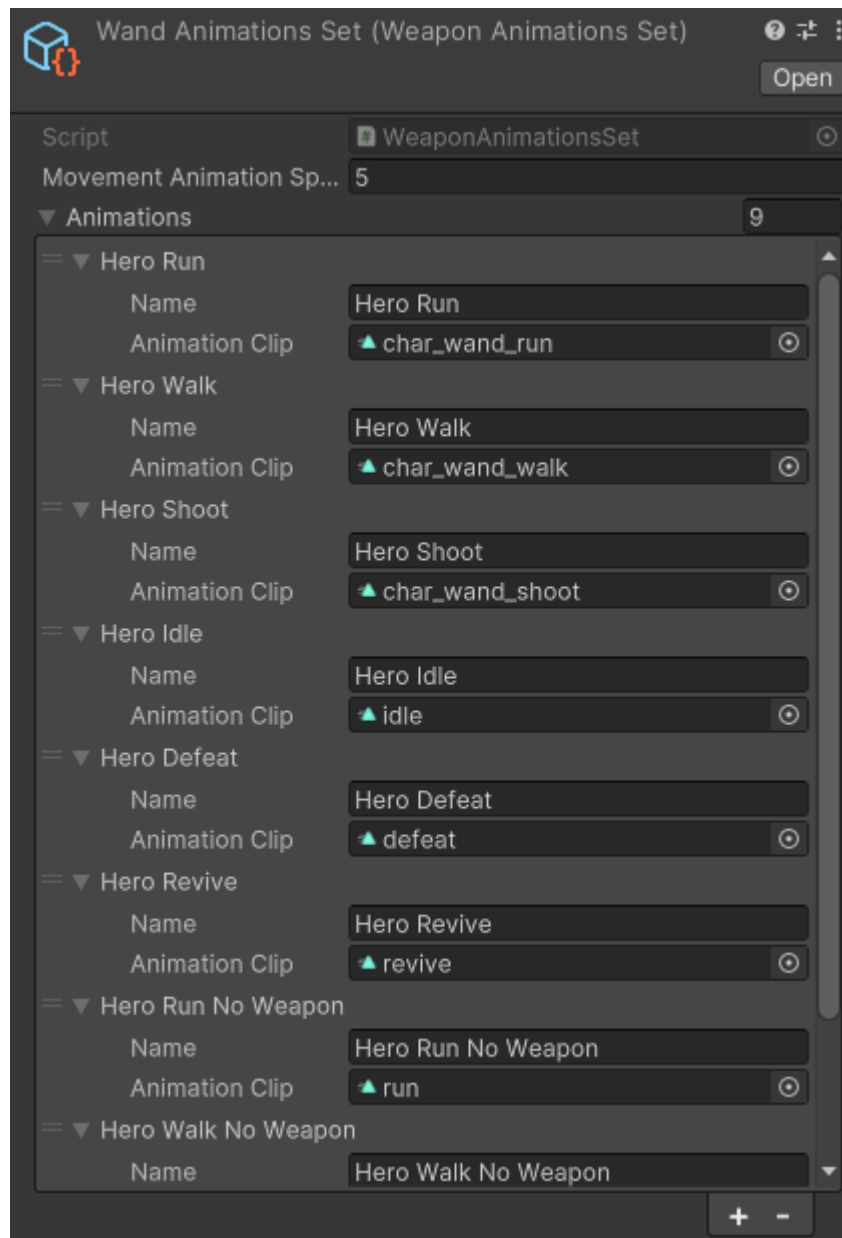
Configuring Animation Sets

1. Open **Assets/The Archer/Scriptables/Armory/Items/Animation Sets**.



2. **Clone** one of the existing animation set assets for each weapon type.
3. Assign your hero's animations in each asset and **remove any unused clips**.

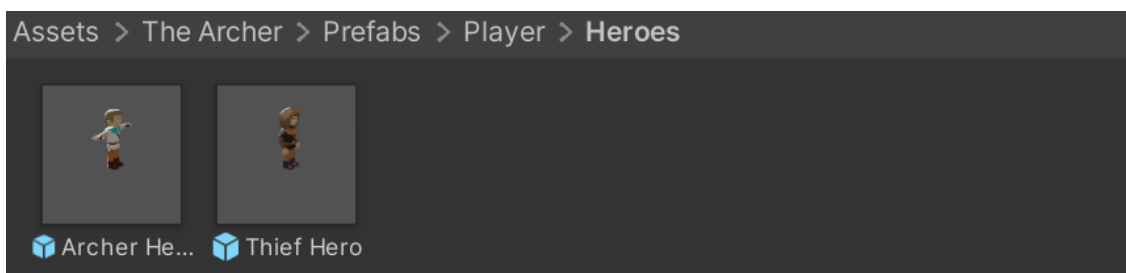
Make sure every animation used in your Animator has a corresponding entry here.



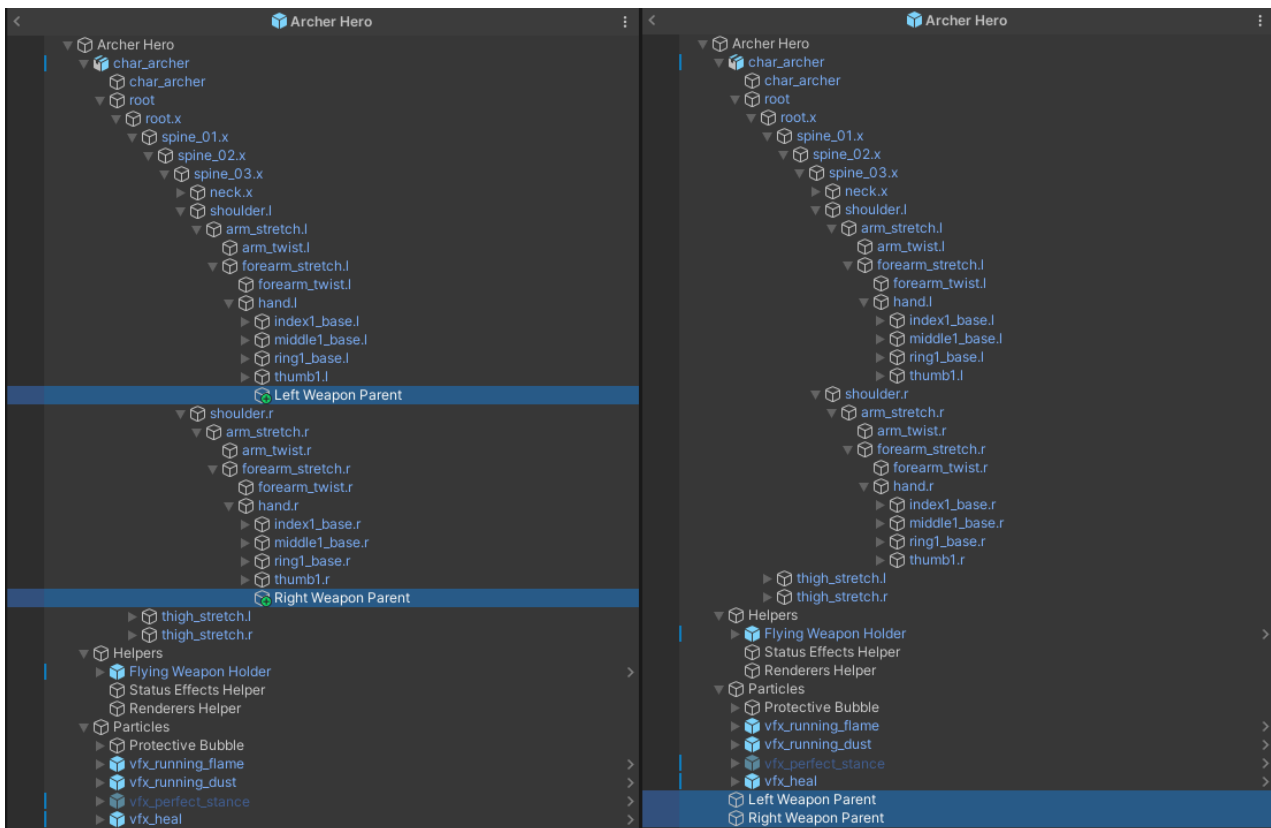
You'll finalize these sets later when connecting them to the hero's data.

Creating Hero Prefab

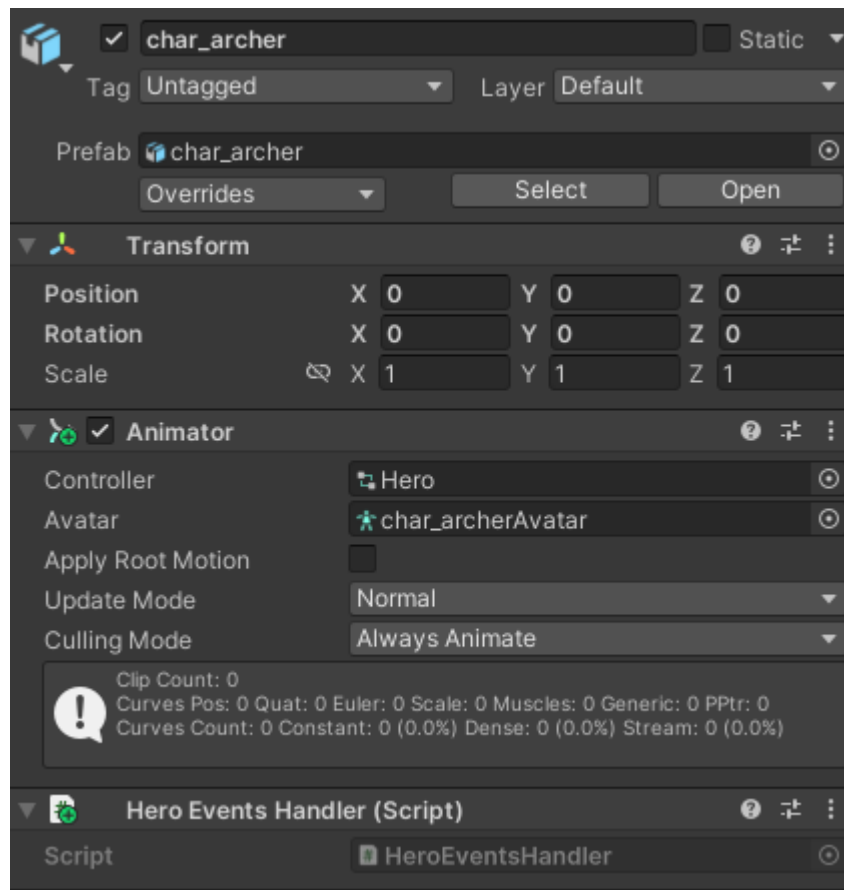
1. Go to **Assets/The Archer/Prefabs/Player/Heroes**.



2. Clone an existing hero prefab and open it.
3. Inside the model hierarchy, find **Left Weapon Parent** and **Right Weapon Parent**, and move them to the prefab's **root**.



4. Delete the old hero model.
5. Drag and drop your **new hero model** into the prefab.
6. Select it and assign your **Animator Controller** from the previous step.
7. Add the **HeroEventHandler** component to your model.



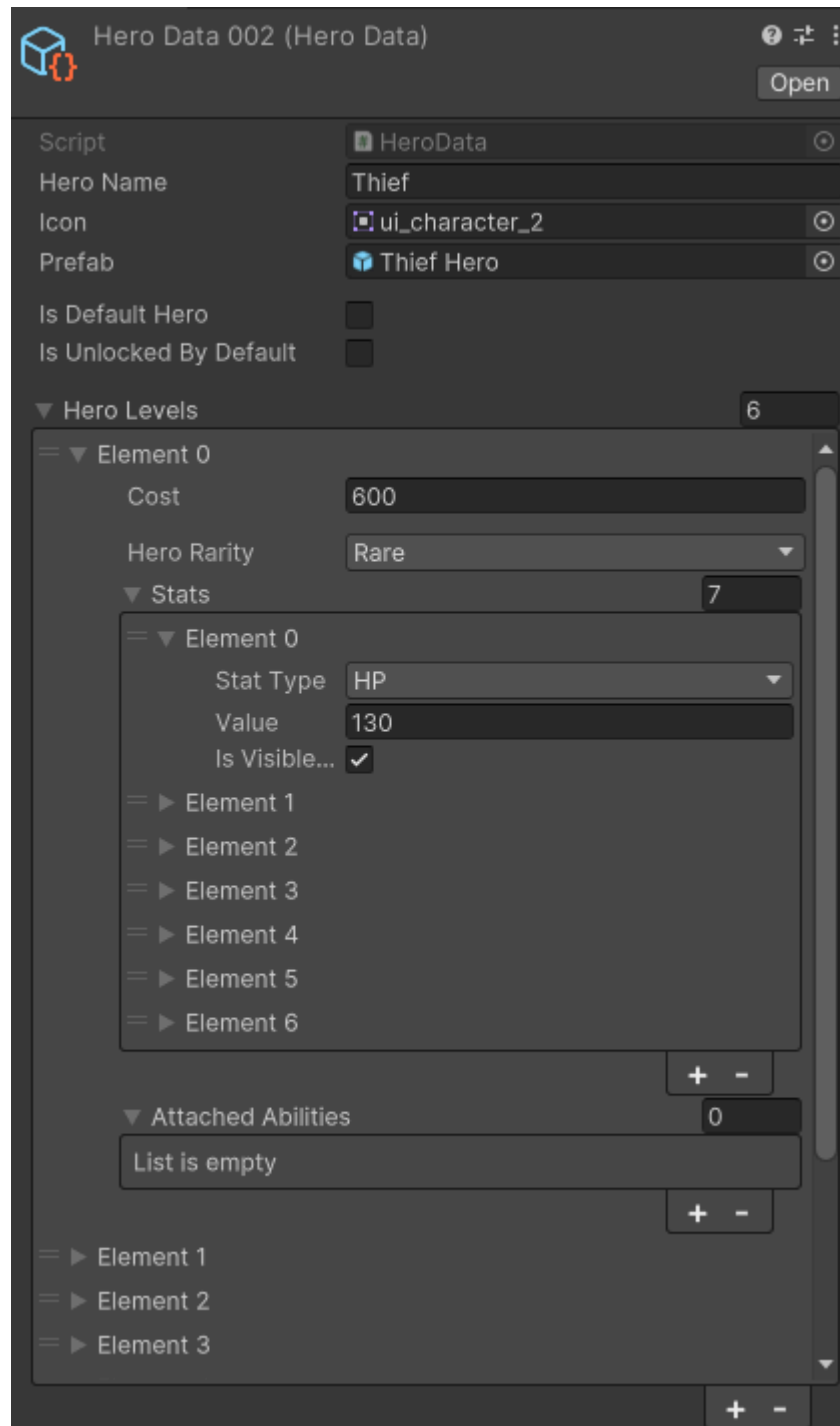
8. Reattach the **Weapon Parents** (from step 3) to the correct hand bones.
9. In **Assets/The Archer/Materials/Heroes**, create a new material for your hero.
 - Use the **October Shader** to enable custom rim effects.
 - Assign your hero's texture and configure parameters as needed.
10. Assign this material to your hero's model.
11. Select the **root GameObject** and assign its **Animator** field to the one used by your model.

Creating Hero Data

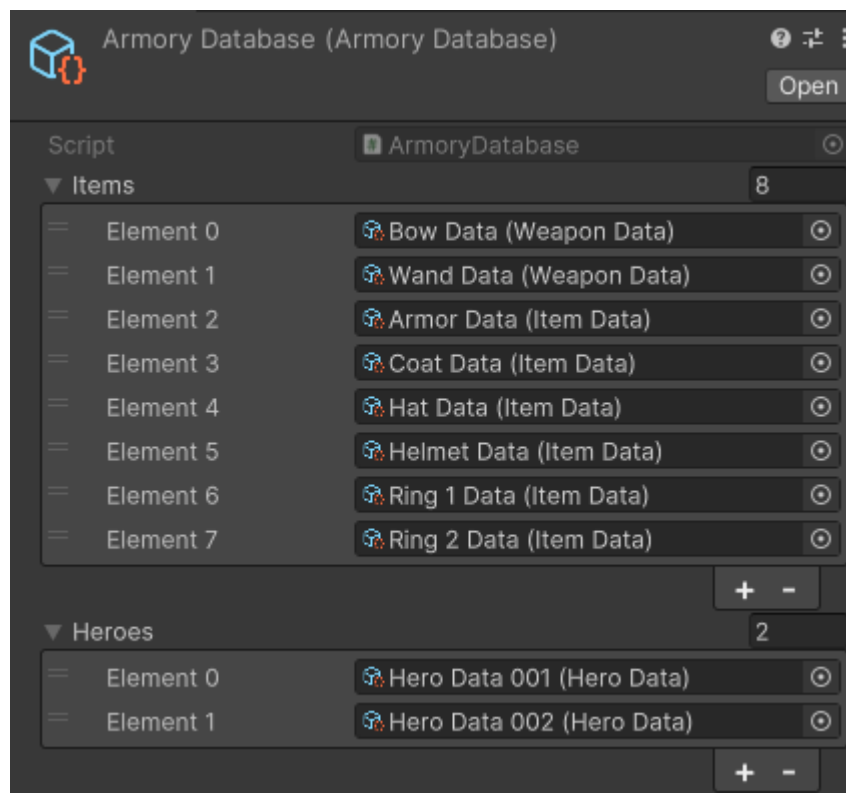
1. Open **Assets/The Archer/Scriptables/Armory/Heroes**.
2. Clone an existing **Hero Data** asset, or create a new one via:
Right-click → Create → October → Armory → Hero Data.
3. Fill in the hero's properties:
 - **Name**
 - **Icon** (the Armory sprite you imported)

- **Prefab** (your newly created hero prefab)
- **Upgrade levels, HP, and Damage** stats (you can fill these in now or later)

Ensure at least **one upgrade level** is defined.



4. Open **Assets/The Archer/Scriptables/Armory** and select the **Armory Database** asset.
5. Add your new **Hero Data** asset to the **Heroes List**.



6. Assign the **Animation Sets** you configured earlier in the grid at the bottom of the **Armory Database**, next to your hero's name.

	Signature Bow	Ceremonial Wand
Archer	Bow Anima	Wand Anim
Thief	Bow Anima	Wand Anim

That's it!

When you launch the game, your new hero should now appear in the **Armory Page** of the **Main Menu** and be fully playable.

Enemies

Enemies Overview

Included Enemies

The template features **14 unique enemies** and **6 unique bosses**, each with their own combat style and behavior patterns.

Enemies

- Cauldron - Fires arched projectiles that leave damaging puddles on the ground.
- Jelly - Shoots projectiles in all directions
- Plant - Shoots projectiles, remains stationary but teleports to new positions instead of moving.
- Wasp - Charges toward the player.
- Eye - Shoots lasers from it's eye.
- Slime - Splits into smaller slimes upon defeat.
- Bomb - Chases the player and explodes on contact.
- Pumpkin - Shoots bursts of seed projectiles.
- Bat - Fires small projectiles from range.
- Hand - Rushes straight toward the player.
- Mole - Follows the player underground.
- Spike - Charges toward the player with noticeable inertia.
- Trap - Follows the player persistently.
- Candle - Fires flame projectiles from a distance.

Bosses

- Queen Wasp - Charges the player, fires honey projectiles, and spawns sticky honey mines.
- Mighty Cactus - Spawns mini cacti, fires needle barrages in all directions, and uses a rapid-fire minigun attack.

- Cursed Totem - Fires lasers in multiple directions, summons tornadoes, and calls down giant fists that slam the ground.
- Ice Slime - Spawns ice spheres, shoots icicles, and creates freezing mines and puddles.
- Mysterious Mask - Fires dark projectiles and waves, and summons spectral hands that charge at the player.
- Ancient Golem - Charges the player and performs a variety of lightning-based attacks.

All enemy behavior scripts **inherit from the** `EnemyBehavior` **base class.**

This class manages most common functionality, such as:

- Spawning and death handling
- Taking damage
- Reacting to global events and effects

Bosses use the same base logic but feature **more complex, multi-phase behaviors** and attacks.

Enemy Data Structure

Candle

Name: Candle

Prefab: Candle Enemy

Enemy Type: Candle

Is Boss: ☐

▼ Drops: 2

Drop Type: XP Gem

HP Drop Zones: 1 2 3 4

0% 100%

Drop Amount: 3 4

Chance: 100

Drop Type: Heal

HP Drop Zones: 1 2 3 4

0% 15% 40% 70% 100%

Drop Amount: 1

Chance: 12

+ -

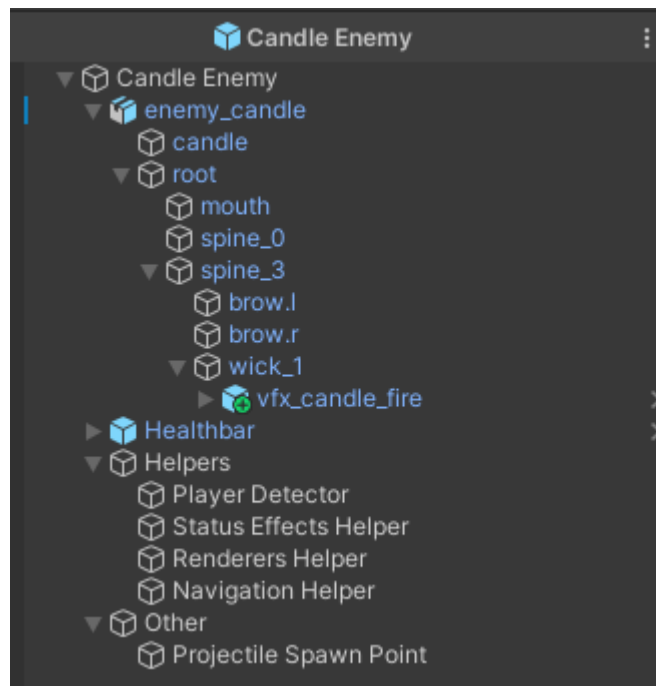
Enemy Data

Each enemy's data is defined in the **Enemy Data** asset, which includes the following fields:

- Name - Used for the boss health bar display.
- Prefab - The prefab reference for the enemy.
- Enemy Type - The unique type identifier of the enemy.
- Is Boss - When enabled, waves containing this enemy display the boss health bar.
- Drops - A list of items the enemy can drop upon defeat.

Enemy Prefab Structure

Example: **Candle Enemy**



- **Root GameObject**
 - Contains the `CandleEnemyBehavior` component (core logic and interactions)
 - Includes **Rigidbody**, **Collider**, and **NavMesh Agent** components
- **Enemy Model**
 - The 3D model of the enemy
 - Includes an **Animator** and a component derived from `EnemyEventsHandler`
- **Healthbar**
 - Displays the enemy's current HP
- **Helpers**
 - **Player Detector** – Detects player proximity for engagement
 - **Status Effects Helper** – Defines how the enemy reacts to status effects
 - **Renderers Helper** – Applies rim visual effects to the enemy's renderers
 - **Navigation Helper** – Handles enemy movement and navigation
- **Projectile Spawn Point** – Defines the location where projectiles are instantiated

 The Rim effect is only visible if the materials use **October.shader** or a shader that implements the rim effect in the same way.

Enemy Behavior

Each enemy behavior must override the coroutine:

```
IEnumerator BehaviorCoroutine();
```

This coroutine starts when the enemy is spawned and continues to run until it is defeated.

It defines the enemy's attack patterns, movement loops, and decision-making logic.

The Candle enemy's `BehaviorCoroutine`:

```
protected override IEnumerator BehaviorCoroutine()
{
    while (IsAlive)
    {
        yield return LocateTarget(0.3f);

        yield return IdleCorountie(idleDuration, false);

        yield return MovementCoroutine();

        yield return IdleCorountie(delayBetweenWalkAndAttack, true);

        yield return AttackCoroutine();
    }
}
```

Bosses typically use **more elaborate coroutines**, often chaining multiple attacks or summoning phases.

The **Cursed Totem** bos's `BehaviorCoroutine`:

```
protected override IEnumerator BehaviorCoroutine()
{
    LookAtPlayer = false;

    // Disabling collider to prevent the player from hitting the totem
    while it is spawning
        enemyCollider.enabled = false;

    waitForSpawnToEnd.Reset();
    yield return waitForSpawnToEnd;

    enemyCollider.enabled = true;

    int i = 0;
    while (IsAlive)
    {
        yield return new WaitForSeconds(idleDuration);

        switch(attacksSequence[i % attacksSequence.Count])
        {
            case TotemAttackType.Laser:
                yield return LaserAttackCoroutine();
                break;
            case TotemAttackType.Tornado:
                yield return TornadoAttackCoroutine();
                break;
            case TotemAttackType.Fist:
                yield return FistAttackCoroutine();
                break;
        }

        i++;
    }
}
```

Each attack:

- Triggers different attack animations,
- Waits for animation events to spawn projectiles or effects,
- Repeats each attack sequence 2–3 times,
- Then returns control back to the main `BehaviorCoroutine()` loop.

How to add a new Enemy

This guide explains how to add a new enemy to the game.

As an example, we'll create and configure a completely new enemy from scratch - including assets, animations, behavior, and data setup.

1. Importing Assets

- Inside `Assets/The Archer/Models/Enemies`, create a **new folder** named after your enemy.
Import the 3D model and animation clips there.
- Inside `Assets/The Archer/Textures/Enemies`, import your enemy's texture.

You'll need the following assets for your enemy:

- **3D Model**
- **Animations**
- **Texture**

Folder Setup

1. In `Assets/The Archer/Enemies`, create a new folder named after your enemy.
Import your **3D model** and **animation clips** there.
2. In `Assets/The Archer/Textures/Enemies`, import your enemy's **texture**.

2. Configuring Animator and Animations

Enemy animator setup is **more flexible** than the hero's - it depends heavily on the specific behavior of your enemy.

The template already includes a wide variety of enemies, which you can use as **references**.

Animation Configuration

1. Ensure each animation clip is properly configured (looped if needed).

2. Add commonly used animation events such as:

- `OnDefeatAnimationEnded`
- `OnAttack`
- `OnAttackEnded`

3. Clone an existing animator or create a new one and place it in the same folder as the 3D model.

Required Animator Parameters

The abstract class **EnemyBehavior** expects the following animator parameters:

Parameter	Type	Description
Movement Blend (EnemyBehavior.MOVEMENT_BLEND_FLOAT)	Float	Controls movement blend tree (idle, walk, run). Skip if your enemy doesn't move.
Defeat (EnemyBehavior.DEFEAT_TRIGGER)	Trigger	Plays the defeat animation.
Hit (EnemyBehavior.HIT_TRIGGER)	Trigger	Plays the hit reaction animation.
Attack (EnemyBehavior.ATTACK_TRIGGER)	Trigger	Plays the attack animation.
Attack Ended (EnemyBehavior.ATTACK_ENDED_TRIGGER)	Trigger	Ends a looping attack animation.
Hide (EnemyBehavior.HIDE_TRIGGER)	Trigger	Plays a hide animation (used, for example, by enemies that burrow).
Show (EnemyBehavior.SHOW_TRIGGER)	Trigger	Plays a show animation (usually paired with Hide).
Attack Type (EnemyBehavior.ATTACK_TYPE_INT)	Int	Defines which attack animation to play when the enemy has multiple attacks.

Enemy animators typically have **at least two layers**:

- **Base Layer** - main animations (idle, movement, attack).
- **Hit Layer** - hit reactions, allowing the enemy to play "hit" animations without interrupting other actions.

3. Creating Enemy Behavior

1. Navigate to:

- `Assets/The Archer/Scripts/Enemies/Simple Enemies` (for regular enemies), or
- `Assets/The Archer/Scripts/Enemies/Bosses` (for boss enemies).

2. Create a new folder named after your enemy.

3. Inside it, create a new script named `[Name]EnemyBehavior.cs`.

Base Setup

Open the script and inherit from **EnemyBehavior**.

Override the `BehaviorCoroutine` method - this is the main loop that defines the enemy's logic:

```
protected override IEnumerator BehaviorCoroutine()
{
    // Wait for spawn animation to finish (if any)
    yield return new WaitForSeconds(0.5f);

    while (IsAlive)
    {
        // Move or idle
        ...

        // Try to find a target
        ...

        // If target found - perform an attack
        ...
    }
}
```

Use existing enemies as examples to learn about structure and pacing.

Typical Attack Example


```
protected IEnumerator AttackCoroutine()  
{  
    animator.SetTrigger(ATTACK_TRIGGER);  
    waitForAttackToEnd.Reset();  
    yield return waitForAttackToEnd;  
}
```

`waitForAttackToEnd` is an instance of a custom `WaitUntilTrue` class, which blocks coroutine execution until its `Complete()` method is called.

Usually, the attack animation triggers `OnAttackEnded` at the end - this signals the coroutine to continue.

If your attack involves projectiles, add an animation event to call the projectile-spawn logic at the right frame.

Animation Events Handling

The **EnemyEventsHandler** component is responsible for catching animation events and forwarding them to the enemy script.

By default, it provides the following callbacks:

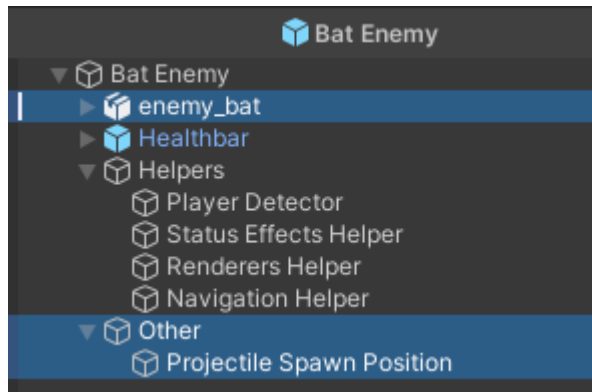
Animation Event	Calls Enemy Method
<code>OnDefeatAnimationEnded</code>	<code>OnDefeatEndedEventFired()</code>
<code>OnAttack</code>	<code>OnAttackEventFired()</code>
<code>OnAttackEnded</code>	<code>OnAttackEndedEventFired()</code>
<code>OnHidden</code>	<code>OnHideEndedEventFired()</code>
<code>OnSpawnEnded</code>	<code>OnSpawnEndedEventFired()</code>

If your enemy requires **additional events**, create a new class that **inherits from** `EnemyEventsHandler` and extend it with your custom event logic.

4. Creating Enemy Prefab

1. Navigate to:

- `Assets/The Archer/Prefabs/Enemies/Simple Enemies` (for regular enemies), or
 - `Assets/The Archer/Prefabs/Enemies/Bosses` (for bosses).
2. Clone an existing enemy prefab and rename it to your enemy's name.
 3. Remove the old model and all components **except** the **Healthbar** and **Helpers** GameObject (with its children).



What to remove

4. Drag your new **enemy model** into the prefab.
5. Assign your **Animator Controller** to the **Animator** component.
6. Add the **EnemyEventsHandler** component (or your custom subclass if you extended it).

Renderers Helper Setup

1. Select the **Renderers Helper** GameObject.
2. Add all your model's **Mesh Renderers** to its **Renderers** list.
3. Remove any unassigned elements.

Enemy Behavior Setup

1. Select the **Root GameObject**.
2. Without removing the existing **EnemyBehavior**, add your new **Enemy Behavior** component.
3. Copy all commonly used field values from the old component to your new one, then delete the old one.
4. Assign:

- **Animator**
 - **Graphic Object** (the model's root GameObject)
5. Adjust any custom parameters controlling your enemy's behavior.
 6. Resize the **collider** and adjust the **healthbar height** to fit your model.

5. Assigning Enemy Data

1. Open:
`Assets/The Archer/Scriptables/Enemies Database`
2. Add a new entry to the **Enemies List** and fill in:
 - **Enemy Name**
 - **Prefab**
 - **Sprite**
 - **Is Boss**
 - **Item Drops** (and the player HP percentages at which they drop)
3. Press the button next to the **Enemy Type** value.



4. In the window that opens, click **Add**.
5. Enter your enemy's name and assign a unique index.
6. Press **Save**.

Enemy Type		⋮
Cauldron	0	Delete
Jelly	1	Delete
Plant	2	Delete
Wasp	3	Delete
Eye	4	Delete
Slime	5	Delete
SmallSlime	6	Delete
TinySlime	7	Delete
Bomb	8	Delete
Pumpkin	9	Delete
Bat	10	Delete
Hand	11	Delete
Mole	12	Delete
Trap	13	Delete
Candle	14	Delete
Spike	15	Delete
QueenWasp	100	Delete
Cactus	101	Delete
Totem	102	Delete
IceSlime	103	Delete
Mask	104	Delete
Golem	105	Delete
Add		Reset
		Save

That's it!

Your new enemy should now appear in the **Stage Creator**.

Add it to any wave and start the game — it should spawn and behave according to your configuration.

ABILITIES

Abilities Overview

This template includes 80 abilities (72 main + 8 endgame). They are sorted into these categories:

 Buff Abilities	Temporary power ups, used mostly to gain attack power.
 Defensive Abilities	Help to reduce upcoming damage and restore HP.
 Elemental Abilities	Enhance your attacks with elemental powers (fire, ice, lightning, etc.).
 Orb Abilities	Spawn circling orbs around the player with various effects.
 Spirit Abilities	Summon helping spirits that attack independently.
 Stat Abilities	Increase player attributes in the game (damage, HP, etc).
 Weapon Abilities	Enhance weapon's projectiles.
 Endgame Abilities	Give additional gold and heal.

Now, let's take a look at the abilities in these categories

Buff Abilities



Combo

Killing an enemy multiplies the player's damage for a short duration. Each consecutive kill within this duration multiplies the damage even more.



Invulnerability Star

Randomly spawns a Star that gives the player invulnerability for a short time upon pickup.



Painful Power

Getting hit temporarily boosts the player's damage.



Perfect Stance

Attack speed is increased while standing still for some time.



Rage

When the player's HP is below a certain percent, they gain increased attack speed and damage.



Rage Star

Randomly spawns a Star that gives the player increased attack speed and damage for a short time upon pickup.

Defensive Abilities

**Cure**

Heals up fully and increases the player's max HP.

**Heal**

Heals a random amount of HP.

**Heal Enhancement**

Heal drop pickups become stronger and increase the player's max HP.

**Kill To Heal**

Killing an enemy has a random chance to recover HP.

**Power Heal**

Heal drop pickups have a chance to increase the player's damage.

**Protector**

Player gains invulnerability after receiving damage.



Safety Bubble

Casts a bubble shield that absorbs one hit.



Second Chance

Gives another life after a defeat.



Strong Aura

Gives a force field that damages enemies upon entrance and decreases their defense.

Elemental Abilities



Freeze

Attacks have a chance to ❄️ freeze enemies. They are completely stunned for a short duration



Freeze Enhancement

Increases ❄️ freeze damage.



Ignite

Attacks have a chance to apply 🔥 **burning**. Enemies on fire get additional burn damage over time.



Ignite Enhancement

Increases 🔥 **burning** damage.



Meteor Strike

Casts meteors that fall on enemies from above, dealing damage.



Meteor Rain

Meteors spawn at a faster rate



Poison

Attacks have a chance to ● **poison** enemies. Poison deals damage over time and decreases enemy damage.



Poison Enhancement

Increases ● **poison** damage.



Shock

Attacks have a chance to ⚡ **shock** enemies. Shock projectiles spread to nearby enemies.



Shock Enhancement

Increases ⚡ **shock** damage.



Ultimate Meteor

Casts devastating meteors that fall from above. These meteors have a bigger range and damage.

Orb Abilities



Fire Orbs

Grants circling fire orbs that apply 🔥 **burning** on impact.



Ice Orbs

Grants circling ice orbs that apply ❄️ **freeze** on impact.



Laser Orbs

Grants circling light orbs that damage on impact, and sometimes strike enemies with a laser in a large range.



Orb Enhancement

Increases damage for all orbs.



Poison Orbs

Grants circling poison orbs that apply  **poison** on impact.



Shock Orbs

Grants circling ice orbs that apply  **shock** on impact.



Ultimate Orbs

Grants circling orbs in a bigger radius that damage strongly on impact.



Vampiric Orbs

Grants circling orbs that steal enemy HP.

Spirit Abilities



Fire Spirit

Summons a friendly creature - Fire Spirit. It shoots at enemies independently and applies 🔥 **burning**.



Freeze Spirit

Summons a friendly creature - Freeze Spirit. It shoots at enemies independently and applies ❄️ **freeze**.



Poison Spirit

Summons a friendly creature - Poison Spirit. It shoots at enemies independently and applies 🟢 **poison**.



Shock Spirit

Summons a friendly creature - Shock Spirit. It shoots at enemies independently and applies ⚡ **shock**.



Spirit Enhancement

All summoned spirits gain increased damage.



Spirit Rage

All summoned spirits gain attack speed.



Ultimate Spirit

Summons a powerful, friendly creature - the Ultimate Spirit. Its damage is unmatched.

Stat Abilities



Boss Slayer

Increases player damage when facing bosses.



Critical Strike

Attacks can randomly deal devastating damage.



Four Leaf Clover

Increases the chance of getting better abilities and dodging attacks.



Gnome Form

Shrinks player, giving them more attack speed, movement speed, and a higher dodge chance.



Lucky Charm

Heal drop pickups can increase max HP.



Major Health Boost

Hugely increases max HP.



Major Power Boost

Hugely increases attack damage.



Major Swiftiness

Hugely increases attack speed.



Medium Power Boost

Moderately increases attack damage.



Minor Attack Speed Boost

Slightly increases attack damage.



Minor Health Boost

Slightly increases max HP.



Minor Power Boost

Slightly increases attack damage.



Minor Speed Increase

Slightly increases movement speed.



Minor Swifttness

Slightly increases attack and movement speed.



Starfall

Stars spawn more frequently.



Titan Form

Increases player size, but gives them much more HP and attack damage.



Trifecta

Grants higher attack damage, attack speed, and max HP.

Weapon Abilities



Bouncy Shot

Player's projectiles bounce off walls and obstacles.



Bullseye

Grants slight magnetism to the player's projectiles, so they miss less often.



Close Combat

Attack range is restricted within a small circle, but attack speed and damage are significantly increased.



Diagonal Shot

Grants two additional diagonal arrows that have slightly lower damage.



Flying Weapon

Weapon flies and attacks on its own, but loses some of its damage and attack speed. Allows the player to move while shooting.



Front Projectile

Grants an additional front projectile, but with slightly lower damage.



Multi Shot

Allows the player to shoot twice in a row, but with slightly lower damage.



Piercing Strike

Attacks pierce through enemies.



Quick Shot

Grants increased attack speed, but projectiles become smaller and imprecise.



Rear Shot

Grants an additional projectile from behind.



Ricochet

Projectiles bounce off enemies.



Sniper

Grants increasingly more damage on distant shots. The longer the distance, the bigger the damage.



Split Shot

Projectiles split into several smaller ones.



Traversing Shot

Projectiles pierce obstacles.

Endgame Abilities



Coins

Grants small, moderate, high, and huge amounts of coins, depending on ability rarity.



Heal

Grants small, moderate, high, and huge amounts of health, depending on ability rarity.

Abilities System Structure

Abilities Manager

The **AbilitiesManager** is located in the **Game** scene hierarchy under the **Stage Controller** GameObject.

It provides a variety of useful methods for manipulating abilities, such as:

- Adding or removing an ability
- Increasing or decreasing an ability's level
- Retrieving all acquired abilities
- Retrieving all available abilities

You can access it in code via the property:

```
StageController.AbilitiesManager
```

All methods of **AbilitiesManager** are declared as `virtual`.

If you want to modify or extend its functionality, it is recommended to create a new class that inherits from it rather than editing the original directly.

Ability Database

The **Ability Database** asset stores references to all **Ability Data** assets used in the game.

It serves as a central point for managing abilities and their rarities. Other tools, such as the [Abilities Tester](#), can only operate on abilities included in this database.

Fields:

- **Abilities** - A list of all **Ability Data** assets available in the game. Only abilities from this list can be accessed by other systems (e.g., the [Abilities Tester](#) or [Abilities Selector](#)).
- **Rarities** - A list of **Rarity Data** entries. Each entry defines a rarity type, its display name, color, and the probability of being selected by the [Abilities Selector](#).
- **Reroll Abilities Cost** - The amount of gold the player must spend to reroll the available abilities in the [Abilities Selector](#).

Abilities Database (Abilities Database)

Open

Script

AbilitiesDatabase

Abilities

80

Element 0

Front Projectile Ability Da

Element 1

Rear Shot Ability Data (Re

Element 2

Diagonal Shot Ability Data

Element 3

Bouncy Shot Ability Data

Element 4

Shock Orbs Ability Data (:

Element 5

Laser Orbs Ability Data (L

Element 6

Fire Orbs Ability Data (Fir

Element 7

Ice Orbs Ability Data (Ice

Element 8

Poison Orbs Ability Data (

Element 9

Ultimate Orbs Ability Data

Element 10

Vampiric Orbs Ability Dat

Element 11

Orb Enhancement Ability

Element 12

Freeze Ability Data (Freez

Element 13

Ignite Ability Data (Ignite

Element 14

Poison Ability Data (Poiso

Element 15

Shock Ability Data (Shock

Element 16

Fire Spirit Ability Data (Fir

Element 17

Freeze Spirit Ability Data

Element 18

Poison Spirit Ability Data

Element 19

Shock Spirit Ability Data (

Element 20

Spirit Enhancement Abilit

Element 21

Spirit Lord Ability Data (S

+

-

Rarities

4

Element 0

Rarity

Common

Color

Name

Common

Chance

50

Element 1

Element 2

Element 3

+

-

Reroll Abilities Cost

200

Ability Data

Every ability in the template has its own **Ability Data class**, which inherits from `GenericAbilityData<T>`, itself derived from the abstract `AbilityData` class.

`AbilityData.cs` contains the following key fields:

Property	Description
AbilityType type	An enum value that uniquely identifies the ability. There must not be two abilities with the same type. This value is usually assigned in the <code>Awake</code> and <code>OnValidate</code> methods of the specific Ability Data class to prevent accidental modification.
List<AbilityType> prerequisites	A list of required abilities. The Abilities Selector will only consider showing this ability if all prerequisite abilities have already been acquired.
AbilityRarity rarity	An enum that defines the rarity of the ability. Possible values: Common, Rare, Mystical, or Legendary. The rarer the ability, the less likely it is to appear. The Abilities Selector shows only abilities of the same rarity at a time, and the Ability Card UI uses different visuals for each rarity.
bool isRepeatedAbiltiy	If true, this ability can always be proposed to the player, even if it has already been acquired or is at its maximum level. Examples include Heal Ability or other endgame abilities.
bool isEndgameAbility	If true, this ability will only be considered by the Abilities Selector when no other abilities of the same rarity remain available.
string title	The display name of the ability. Shown on the Ability Card UI and above the player when the ability is acquired.
string description	The display description of the ability. Shown on the Ability Card UI.
Sprite icon	The ability's icon, shown on the Ability Card UI and in the Pause Menu if the ability is active.

GameObject prefab

The prefab that represents this ability. Its root object must include an Ability Behavior component that defines the ability's functionality.

You can subscribe to the `onAbilityUpgraded` event to execute custom code when the ability is upgraded.

`GenericAbilityData` is a generic intermediary class between `AbilityData` and the actual implementation of an ability's data class.

Its generic parameter `T` must be a subclass of `AbilityLevel` - an empty marker class used to define individual ability level data.

Each **Ability Data** instance contains a list of **Ability Levels**, specific to that ability.

```
public class GenericAbilityData<T> : AbilityData where T : AbilityLevel
{
    [SerializeField] protected T[] levels;
    public override AbilityLevel[] Levels => levels;

    public virtual new T GetLevel(int index)
    {
        return levels[index];
    }
}
```

Here's an example of an actual Ability Data class:

```

// This attribute adds an item to the Create menu in Unity
// allowing you to create an instance of this ScriptableObject
[CreateAssetMenu(fileName = "Combo Ability Data", menuName =
"October/Abilities/Buff Abilities/Combo")]

// ComboAbilityData inherits from GenericAbilityData,
// with its generic parameter set to ComboAbilityLevel
public class ComboAbilityData : GenericAbilityData<ComboAbilityLevel>
{
    // When the asset is created, the type field is set to
    AbilityType.Buff_Combo
    protected virtual void Awake()
    {
        type = AbilityType.Buff_Combo;
    }

    // Prevents accidental modification of the type field
    protected virtual void OnValidate()
    {
        type = AbilityType.Buff_Combo;
    }
}

// This attribute allows Unity to display the class in the Inspector
[System.Serializable]
public class ComboAbilityLevel : AbilityLevel
{
    // Time before the combo meter resets
    [SerializeField] protected float abilityDuration;
    public float AbilityDuration => abilityDuration;

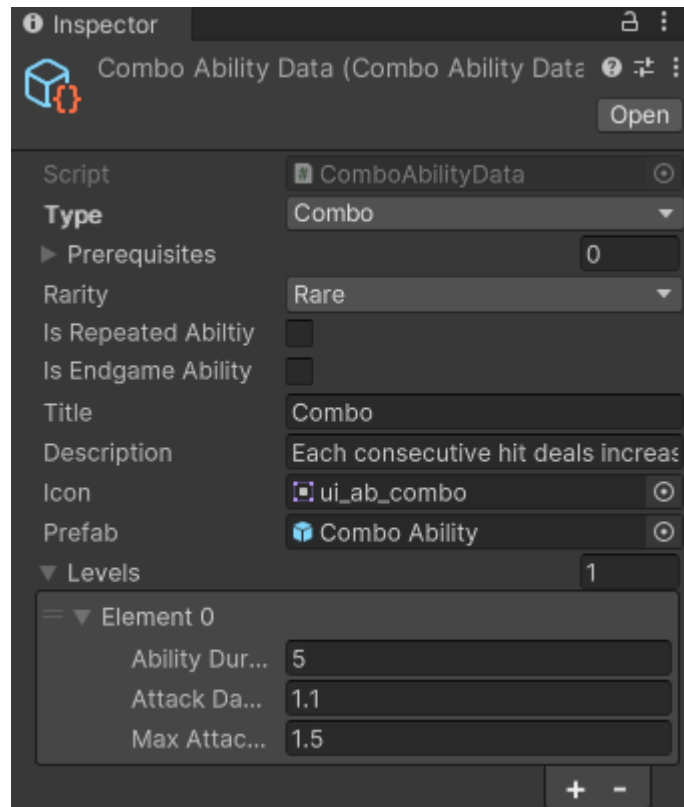
    // Each defeated enemy increases attack damage by this multiplier.
    // The effect stacks and resets after abilityDuration seconds.
    [SerializeField] protected float
attackDamageMultiplierPerEnemyKilled;
    public float AttackDamageMultiplierPerEnemyKilled =>
attackDamageMultiplierPerEnemyKilled;

    // Maximum cap for the attack damage multiplier
    [SerializeField] protected float maxAttackDamageMultiplier;
    public float MaxAttackDamageMultiplier =>
maxAttackDamageMultiplier;
}

```

Note: The underscore symbol in `AbilityType.Buff_Combo` separates the **category** ("Buff") from the **ability name** ("Combo").
The template includes 80 abilities, so a custom property drawer is used to organize them into categories for easier access.

Here's how the Combo Ability Data inspector looks like



Ability Behavior

Each ability must have its own prefab with a component that inherits from the generic abstract class:

```
AbilityBehavior<T, K>
```

where:

- **T** - the ability data class
- **K** - the ability level class

This base class provides several commonly overridden methods used to control ability initialization, level updates, and cleanup.

Commonly Overridden Methods

```
public virtual void Init(AbilityData data, int levelId)
```

Called once when the ability is activated.
Subsequent level changes do **not** trigger this method.

```
protected virtual void SetData(T data)
```

Called once together with `Init`.
Use this when you need to ensure that the ability data is initialized before performing setup logic.

```
protected virtual void SetAbilityLevel(int levelId)
```

Called when the ability is first activated and every time its level changes.
Override this method to adjust the ability's behavior or values when its level changes.

```
public virtual void Clear()
```

Called when the ability is removed.
By default, it destroys the ability's `GameObject`.
If you override this method, make sure to call `base.Clear()` at the end to ensure proper cleanup.

Below is an example of a Major Power Boost Ability behavior class that modifies the player's attack damage stat.

```
public class MajorPowerBoostAbilityBehavior :
AbilityBehavior<MajorPowerBoostAbilityData,
MajorPowerBoostAbilityLevel>
{
    // Multiplier applied to the player's attack damage stat
    protected StatMultiplier attackDamageMultiplier = 1;

    public override void Init(AbilityData data, int levelId)
    {
        base.Init(data, levelId);

        // Add multiplier to the player's attack damage stat
        StageController.Player.Stats.AttackDamageStat.AddMultiplier(attackDamageMultiplier);
    }

    protected override void SetAbilityLevel(int levelId)
    {
        base.SetAbilityLevel(levelId);

        // Update multiplier value based on the current ability level
        attackDamageMultiplier.Value =
        AbilityLevel.AttackDamageMultiplier;
    }

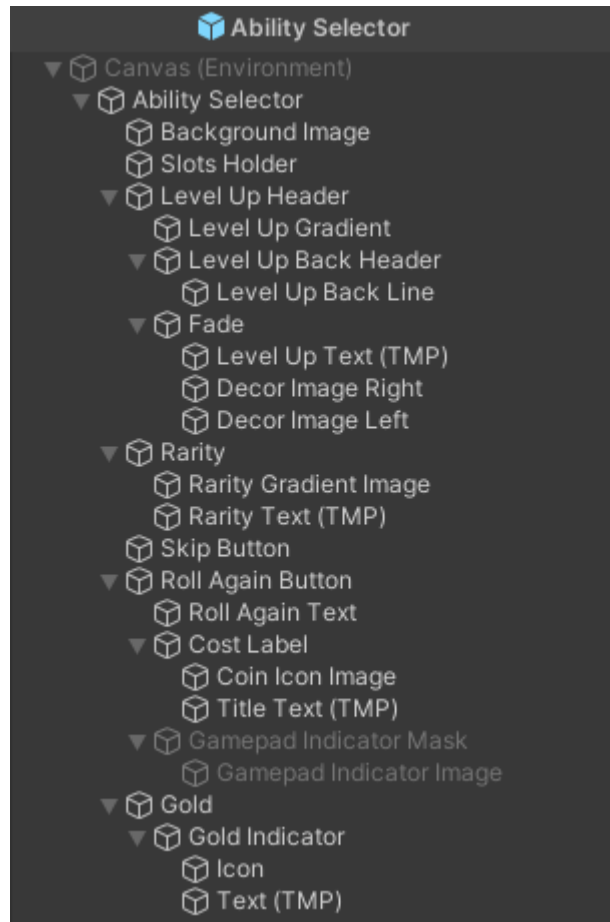
    public override void Clear()
    {
        // Remove multiplier before destroying the ability
        StageController.Player.Stats.AttackDamageStat.RemoveMultiplier(attackDamageMultiplier);

        base.Clear();
    }
}
```

Abilities Selector Window

The **Abilities Selector** is a complex window that displays up to three abilities for the player to choose from.

Abilities Selector Prefab Structure

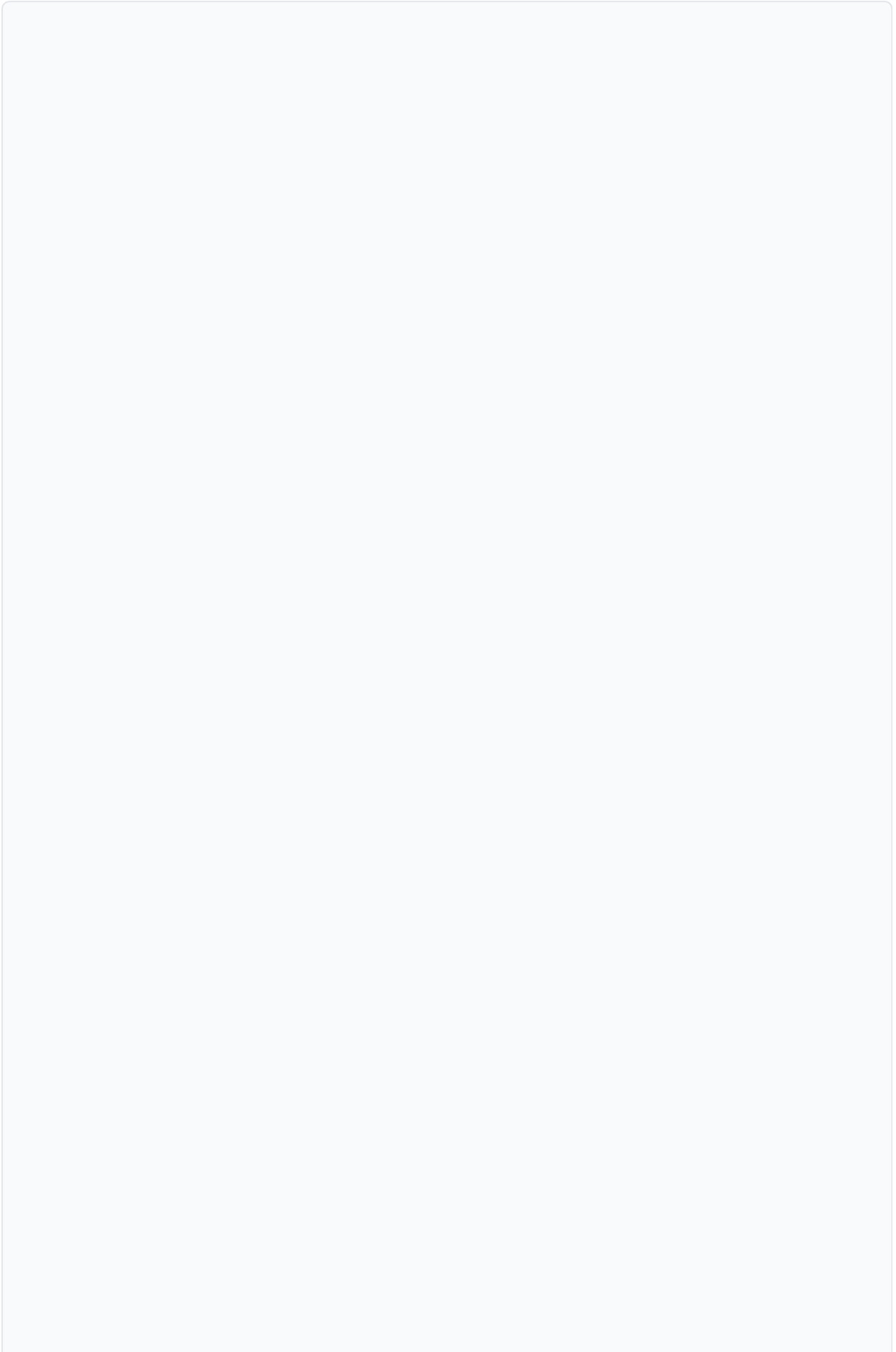


- Background Image - A dark, full-screen image that dims the background and blocks raycasts to all elements behind the Ability Selector.
- Slots Holder - The parent object that contains all **Ability Selector Slots**.
- Level Up Header - The visual header of the Ability Selector. It includes the *Level Up* text, decorative images, and an animated shine line that plays when the Ability Selector is shown.
- Rarity - A visual element representing the rarity tier of the abilities currently shown to the player. It contains **Rarity Text** and a **Gradient Image**.
- Skip Button - A full-screen invisible button that allows the player to skip the Ability Selector's animation.

- Roll Again Button - A button that lets the player reroll the available abilities for a small gold cost.
- Gold - Displays the player's current gold amount.

Abilities Selector Animation flow

The Ability Selector window features a complex multi step animation. It's controlled by ShowCoroutine `()` method. Let's break it down




```
// Method is public virtual for easier modification
public virtual IEnumerator ShowCoroutine(List<AbilityData> abilities,
AbilityRarity rarity)
{
    IsOpen = true;
    Rarity = rarity;

    // Tell the Game Screen to hide the Pause button and temporary gold
indicator
    StageController.GameScreen.HideSideUI();

    // Initialize audio (plays popup and ambient scroll sounds)
    InitAudio();

    // Reset all UI parameters and components to default values
    ResetUI();

    // Gradually slow down time to 0
    timeScaleEasingCoroutine = EasingManager.DoTimeScale(0,
timescaleStopDuration);

    // Fade in background image
    backgroundAlphaEasingCoroutine =
        backgroundImage.DoAlpha(1, backgroundFadeInDuration)
        .SetUnscaledTime(true);

    // Show Abilities Selector header
    title.Show();

    // Wait briefly before initializing slots
    yield return new WaitForSecondsRealtime(slotInitialDelay);

    // Fetch slots from the pool and initialize them with ability data
and IDs
    InitSlots(abilities);

    // Allow player to skip the remaining animation
    skipButton.gameObject.SetActive(true);

    // Wait before showing rarity flash
    yield return new WaitForSecondsRealtime(flashInitialDelay);

    // Fade in rarity header (initially gray/undefined)
    rarityAlphaEasingCoroutine =
        rarityCanvasGroup.DoAlpha(1, 0.5f)
        .SetUnscaledTime(true);

    // Flash rarity until desired level is reached
```

```
// Order: Common -> Rare -> Mystical -> Legendary
var rarityIndex = (int)rarity;
for (int i = 0; i <= rarityIndex; i++)
{
    var data = rarityData[i];
    DoRarityFlash(data); // Trigger rarity flash animation

    if (i != rarityIndex)
        yield return new WaitForSecondsRealtime(flashStepDelay);
}

// Wait before stopping the slot scroll
yield return new WaitForSecondsRealtime(stopScrollingDelay);

// Stop scrolling slots, show abilities, reroll button, and gold
indicator
yield return CompleteShowCoroutine();
}
```

The **Ability Selector Component** contains many variables that control the animation flow, durations, and delays of different UI elements.

If the player clicks **Skip**, the animation immediately jumps to the

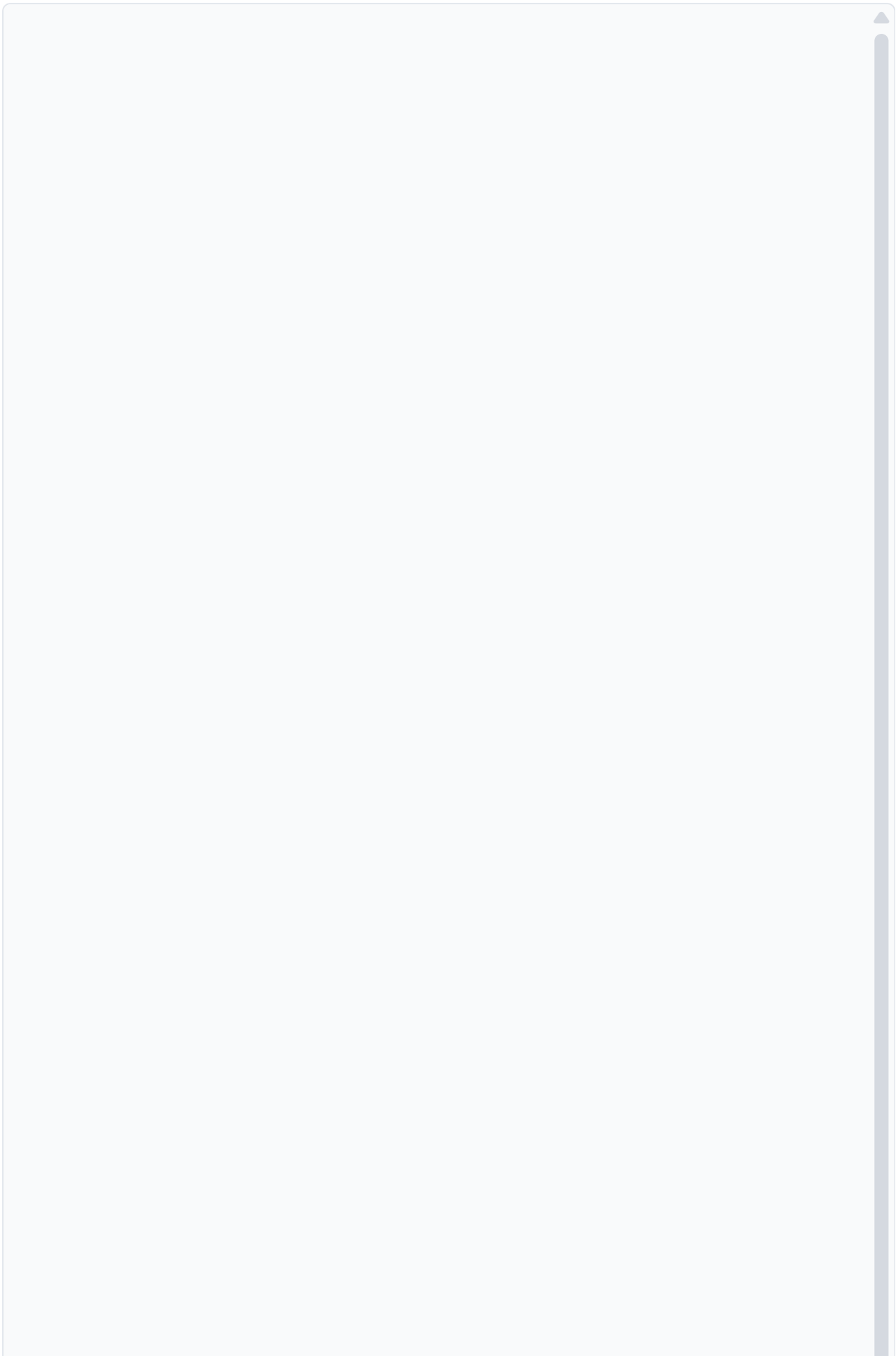
`CompleteShowCoroutine()` method.

Ability Selector Slot Animation Flow

Each slot scrolls cards vertically in a continuous loop, controlled by

`ScrollingCoroutine()`.

Here's a breakdown:



```
protected virtual IEnumerator ScrollingCoroutine( AbilityData data, int
id)
{
    isStopped = false;

    // Reset gradient image to gray (undefined) and make it transparent
    gradientImage.color = initialGradientColor;
    gradientImage.SetAlpha(0);

    // Fade in the gradient background
    gradientImage.DoAlpha(initialGradientColor.a,
0.3f).SetUnscaledTime(true);

    // Get an Ability Card from the pool and show its back side
    highestCard = cardsPool.GetEntity();

    // Telling card to initialize showing it's back.
    // hiddenCardMargin is used to calculate card's size
    highestCard.ShowHidden(hiddenCardMargin);

    var scrollContentSize = scrollContentRect.sizeDelta;
    var cardSize = highestCard.RectTransform.sizeDelta;

    // Calculating first card's position.
    // Adjusted card's position adds offset from slot's id.
    cardPosition = new Vector2(0, scrollContentSize.y / 2f + cardSize.y
/ 2f - hiddenCardMargin);
    var adjustedCardPosition = new Vector2(0, scrollContentSize.y / 2f
+ cardSize.y / 2f + cardSize.y / 4f * id - hiddenCardMargin);

    highestCard.RectTransform.anchoredPosition = adjustedCardPosition;

    var cardHeightOffset = new Vector2(0, cardSize.y);

    // Card image is faded by a custom gradient, not by a single alpha
value.
    // We use custom shader for that, and this are the parameters that
are provided
    // To the material
    highestCard.SetAlphaParameters(200, 200 - fadeSize, true);
    highestCard.CanvasGroup.alpha = 1;

    // Calculating the position the highest card needs to be in order
to spawn next card
    // and the position card will be completely invisible and can be
returned to the pool.
    nextCardSpawnTriggerY = cardPosition.y - cardSize.y +
hiddenCardMargin * 2 - scrollingSpacing;
```

```

var cardDisappearTriggerY = -cardPosition.y;
var flipCardTriggerY = nextCardSpawnTriggerY - fadeSize;

// Creating and adding to a list Card data - a simple wrapper for
Ability card
// with an additional flipped value
// Card is "Flipped" when it passes a middle point of a slot
var highestCardData = new CardData
{
    Card = highestCard,
    flipped = false
};

cards.Add(highestCardData);

var scrollSpeed = this.scrollSpeed;

MainCard = null;

LastCardSpawnTime = Time.unscaledTime;

// Scroll is continuous until Abilities Selector doesn't stop it.
while (true)
{
    // We run this loop every frame
    yield return null;

    bool shouldBreak = false;

    // If we stopping scrolling and showing main card
    if (isStopped && MainCard != null)
    {
        var time = Time.unscaledTime - mainCardSpawnTime;
        var t = time / mainCardMovementDuration;
        if(t > 1)
        {
            t = 1;
            shouldBreak = true;
        }

        // We moving top card using a animationCurve you can adjust
        in the inspector
        var y = Mathf.LerpUnclamped(cardPosition.y, 0,
mainCardMovementCurve.Evaluate(t));
        var difference = MainCard.RectTransform.anchoredPosition.y
- y;

        // At this point scrolling speed is determined by the curve

```

```

        scrollSpeed = difference / Time.unscaledDeltaTime;

        // Hiding other card by another adjustable curve aswell
        var otherCardsAlpha = otherCardsFadeCurve.Evaluate(t);

        for (int i = 0; i < cards.Count; i++)
        {
            var card = cards[i];

            if(card.Card != MainCard)
            {
                card.Card.CanvasGroup.alpha = otherCardsAlpha;
            }
        }

        for (int i = 0; i < cards.Count; i++)
        {
            var cardData = cards[i];
            var card = cardData.Card;

            // Moving all card down
            card.RectTransform.anchoredPosition += Vector2.down *
scrollSpeed * Time.unscaledDeltaTime;

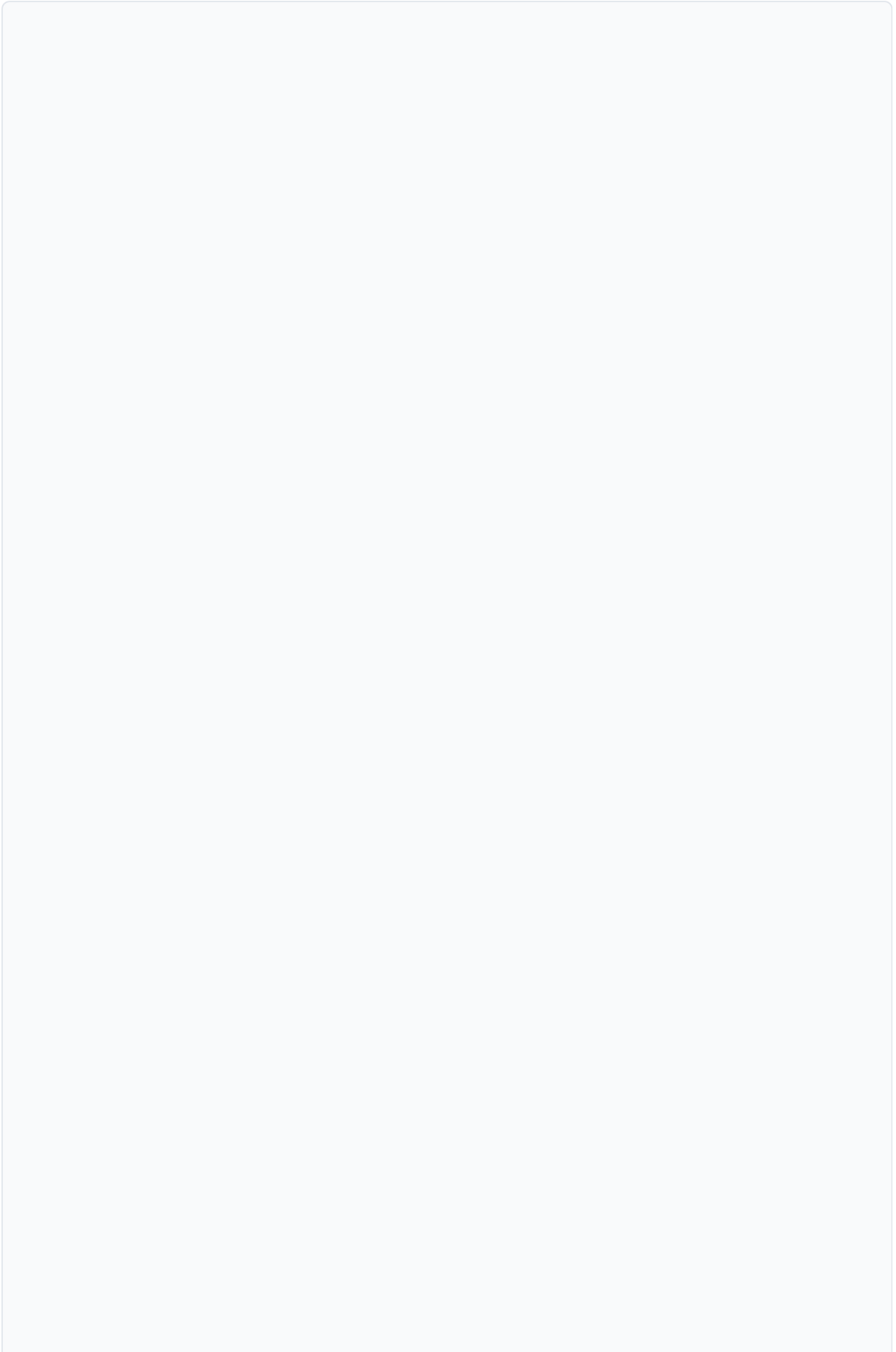
            if(card != MainCard)
            {
                // reseting the scale of all cards but the main one
                card.transform.localScale = Vector3.one * cardsScale;
            }

            // Hiding card if it is bellow cardDisappearTriggerY
            if (card.RectTransform.anchoredPosition.y <
cardDisappearTriggerY)
            {
                card.gameObject.SetActive(false);
                cards.RemoveAt(i);
                i--;
            }
        }
    }
}

```

Ability Card Animation Flow

The animation for revealing an ability card is handled by the `Init()` method in `AbilityCardBehavior`:



```
public virtual void Init(AbilityData data, float duration, float delay
= 0)
{
    IsAnimationActive = true;
    button.enabled = false;

    // Initializing Ability Card UI, such as Icon, name and description
    SetData(data);
    // Setting position of card's front rect
    cardFrontRect.SetStretchedOffset(Vector2.zero, Vector2.zero);

    // By default cardBackSideImage.sprite is set to blurry version of
    a sprite to
    // better show movement.
    // Here we switch it to a clear version
    cardBackSideImage.sprite = cardBackSprite;

    var offsetMin = cardBackRect.offsetMin;
    var offsetMax = cardBackRect.offsetMax;

    var scaleX = (RectTransform.sizeDelta.x - offsetMin.x +
offsetMax.x) / RectTransform.sizeDelta.x;
    var scaleY = (RectTransform.sizeDelta.y - offsetMin.y +
offsetMax.y) / RectTransform.sizeDelta.y;

    CardBackRect.SetStretchedOffset(Vector2.zero, Vector2.zero);

    transform.localScale = new Vector3(scaleX, scaleY, 1);
    // Scaling card to a final size
    scaleEasingCoroutine =
        transform.DoLocalScale(Vector3.one, scaleDuration)
            .SetDelay(duration - scaleDelayReduction + delay)
            .SetEasingCurve(scaleCurve)
            .SetUnscaledTime(true);

    // Rotating card's back
    backRotationEasingCoroutine =
        cardBackRect.DoLocalEulerAngles(new Vector3(0, 90, 0),
backRotationDuration)
            .SetDelay(duration - rotationDelayReduction + delay)
            .SetUnscaledTime(true)
            .SetEasingCurve(backScaleCurve)
            .SetOnFinish(OnCardBackScaledToZero);

    // Rotating card's front
    frontRotationEasingCoroutine =
        cardFrontRect.DoLocalEulerAngles(Vector3.zero,
frontRotationDuration)
```



```
        .SetDelay(duration - rotationDelayReduction + delay +
backRotationDuration)
        .SetEasingCurve(frontScaleCurve)
        .SetUnscaledTime(true);

    // Scale the gradient image to its original size
    gradientStretchEasingCoroutine =

gradientImage.rectTransform.DoStretchedOffset(gradientOffsetMin,
gradientOffsetMax, gradientStretchDuration)
        .SetDelay(duration - gradientStretchDelayReduction + delay)
        .SetEasing(EasingType.CubicOut)
        .SetUnscaledTime(true)
        .SetOnFinish(OnGradientStretched);

    // If ability is already acquired than we activate upgrade
indicator
    if (shouldShowUpgrade)
    {
        upgradeIndicator.gameObject.SetActive(true);
        upgradeIndicator.DoLocalScale(Vector3.one, 0.4f)
            .SetUnscaledTime(true)
            .SetEasing(EasingType.CubicOut);
    }

    // Playing reveal sound
    EasingManager.DoAfter(cardRevealSoundDelay + delay, () =>
GameController.AudioManager.PlayAudio(cardRevealedSound)).SetUnscaledTi
me(true);
}
```

How to add a new Ability

This guide explains how to add a new ability to the game.

As an example, we'll create an ability that **increases the player's damage and attack speed** but **reduces HP**, called **Glass Cannon**.

1. Importing assets

1. Import the ability icon into

`Assets/The Archer/Sprites/UI/Abilities`

2. Set its **Texture Type** to **Sprite (2D and UI)**.

Note: Corners of the sprite will be masked out in the UI.

3. If your ability requires additional assets such as **meshes**, **textures**, or **particles**, import them into the appropriate subfolders under `Assets/Common`.

2. Adding new Ability Type

1. Open the script:

`Assets/The Archer/Scripts/Abilities/Data/AbilityType.cs`

2. The enum values follow the naming pattern:

`[Group]_[AbilityName]`

The first part defines the **Group** (used for organization in the property drawer), and the second is the **Ability Name**.

Groups are only for editor organization and have no gameplay impact.

3. Add a new entry for your ability under a new or existing group.

For this guide, we'll create a new **Custom** group.

In an unmodified template, the last index is `79`, so use `80`:

```
Endgame_CommonCoins = 76,  
Endgame_RareCoins = 77,  
Endgame_MysticCoins = 78,  
Endgame_LegendaryCoins = 79,  
Custom_GlassCannon = 80
```

3. Creating Ability Data script

1. Navigate to:

Assets/The Archer/Scripts/Abilities/Behaviors

2. Create a new folder named **Custom** (or open the existing group folder if you're adding to an existing group).
3. Inside it, create a new folder named **GlassCannon**.
4. Create a script named **GlassCannonAbilityData.cs** and add the following code:

```
[CreateAssetMenu(fileName = "Glass Cannon Data", menuName =
"October/Abilities/Custom/Glass Cannon")]
public class GlassCannonAbilityData :
GenericAbilityData<GlassCannonAbilityLevel>
{
    protected virtual void Awake()
    {
        type = AbilityType.Custom_GlassCannon;
    }

    protected virtual void OnValidate()
    {
        type = AbilityType.Custom_GlassCannon;
    }
}

[System.Serializable]
public class GlassCannonAbilityLevel : AbilityLevel
{
    [SerializeField] protected float attackDamageMultiplier;
    public float AttackDamageMultiplier => attackDamageMultiplier;

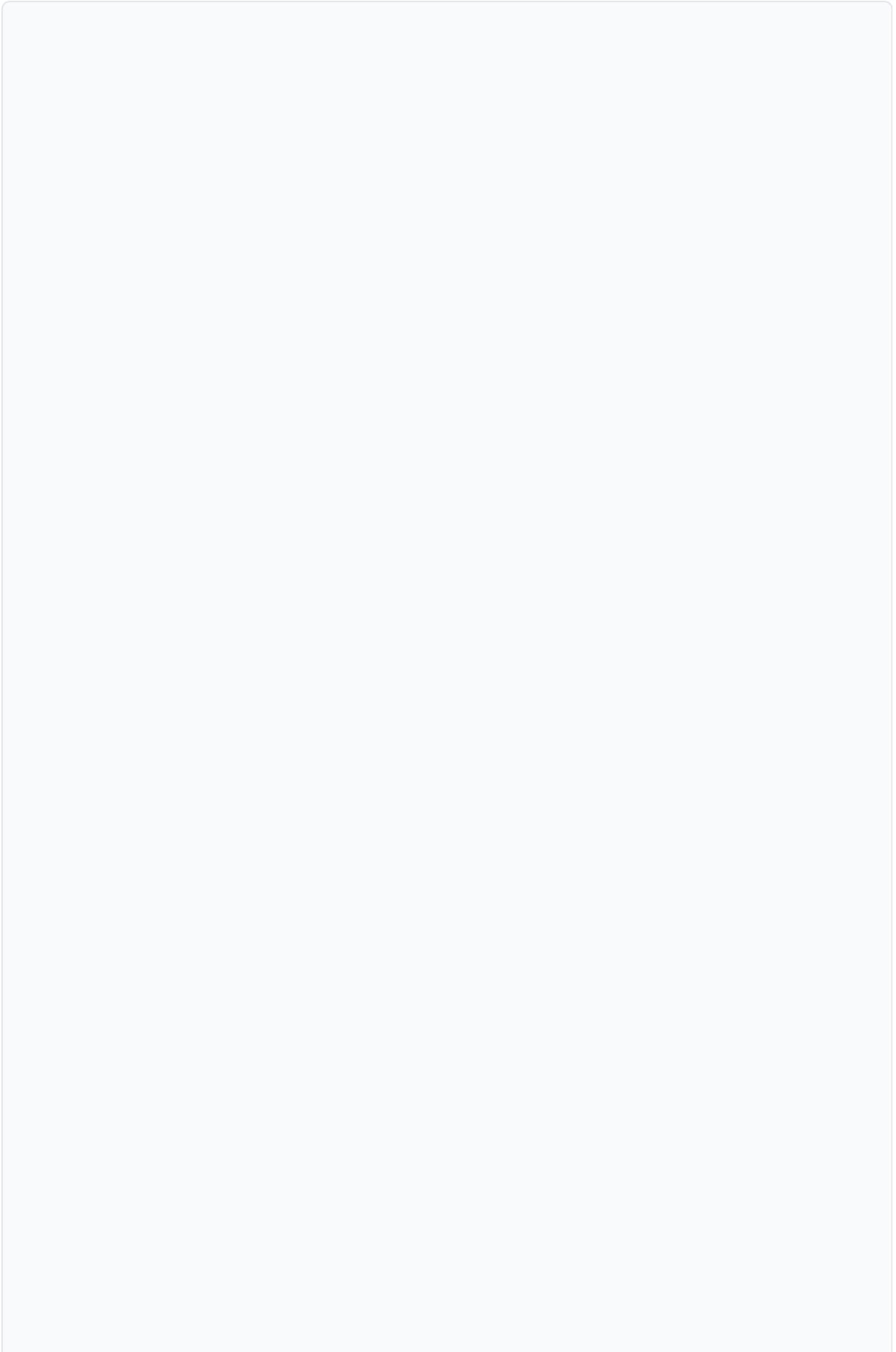
    [SerializeField] protected float attackSpeedMultiplier;
    public float AttackSpeedMultiplier => attackSpeedMultiplier;

    [SerializeField] protected float hpMultiplier;
    public float HPMultiplier => hpMultiplier;
}
```

4. Creating Ability Behavior script

1. In the same folder, create a new script named **GlassCannonAbilityBehavior.cs**

2. Add the following code:



```
public class GlassCannonAbilityBehavior :
AbilityBehavior<GlassCannonAbilityData, GlassCannonAbilityLevel>
{
    // Multipliers that modify the player's stats
    protected StatMultiplier attackDamageMultiplier = 1;
    protected StatMultiplier attackSpeedMultiplier = 1;
    protected StatMultiplier hpMultiplier = 1;

    public override void Init(AbilityData data, int levelId)
    {
        base.Init(data, levelId);

        // Register multipliers to the corresponding stats

        StageController.Player.Stats.AttackDamageStat.AddMultiplier(attackDamag
eMultiplier);

        StageController.Player.Stats.AttackSpeedStat.AddMultiplier(attackSpeedM
ultiplier);

        StageController.Player.Stats.MaxHPStat.AddMultiplier(hpMultiplier);
    }

    protected override void SetAbilityLevel(int levelId)
    {
        base.SetAbilityLevel(levelId);

        // Update multiplier values each time the ability level changes
        attackDamageMultiplier.Value =
AbilityLevel.AttackDamageMultiplier;
        attackSpeedMultiplier.Value =
AbilityLevel.AttackSpeedMultiplier;
        hpMultiplier.Value = AbilityLevel.HPMultiplier;
    }

    public override void Clear()
    {
        // Remove multipliers when the ability is removed or the game
ends

        StageController.Player.Stats.AttackDamageStat.RemoveMultiplier(attackDa
mageMultiplier);

        StageController.Player.Stats.AttackSpeedStat.RemoveMultiplier(attackSpe
edMultiplier);

        StageController.Player.Stats.MaxHPStat.RemoveMultiplier(hpMultiplier);
```

```
        base.Clear();  
    }  
}
```

5. Creating Ability Prefab

1. Go to:
`Assets/The Archer/Prefabs/Abilities`
2. Create a new folder named **Custom**, or open the existing one.
3. Right-click → **Create** → **Prefab**
Name it **Glass Cannon Ability** and open it.
4. Select the root object and add the **GlassCannonAbilityBehavior** component.
5. (Optional) If you have a particle effect that plays when the ability is activated, drag it into the prefab and assign it to the **Ability Acquired Particle** field.

6. Creating Ability Data

1. Go to:
`Assets/The Archer/Scriptables/Abilities`
2. Create a new **Custom** folder (or use an existing one).
3. Right-click → **Create** → **October** → **Abilities** → **Custom** → **Glass Cannon**
This option comes from the `CreateAssetMenu` attribute.
4. Select the newly created asset and fill in its fields according to the [Ability Data Structure](#) documentation.
 - Assign the prefab you created in the previous step to the **Prefab** field.
 - Add at least one **Ability Level**.
5. Finally, open the **Abilities Database** asset located in the same folder and add the **Glass Cannon** data asset to the **Abilities List**.

7. Testing

1. Start the game and enter any stage.
2. Press **Shift + A** to open the **Abilities Tester** window.
3. In the **Gameplay** tab, activate the **Glass Cannon** ability.

You should now see the player's **damage and attack speed increased**, and **HP reduced** according to your configuration.

Stage Creator

Stage Creator Overview

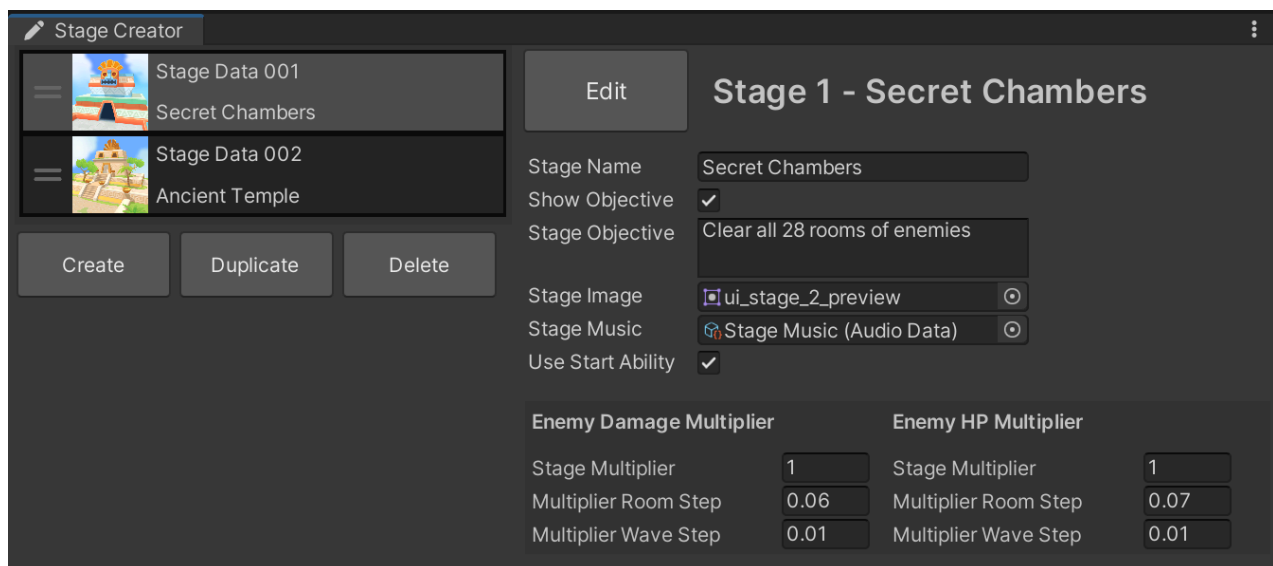
Stages Window Overview

This template features a robust solution that lets you create and edit stages, control game balancing and flow, and more.

Let's take a look at the Stage Creator.

To open it, in the top bar, go to *Tools* → *October* → *Stage Creator*. Or simply press **Shift + S**.

You will be presented with this window:



This window is split into two parts. *On the left*, you can see a list of all the stages, with options to add, copy, and delete. Additionally, items on this list are draggable, so you can sort your stages easily.

Upon selecting a stage, *the right part* will show all the available settings for stages:

- **Stage Name** - will be displayed in the main menu.
- **Show Objective** - an option to display a stage objective.
- **Stage Objective** - a short UI popup at the start of a stage, good for informing the player what to expect.

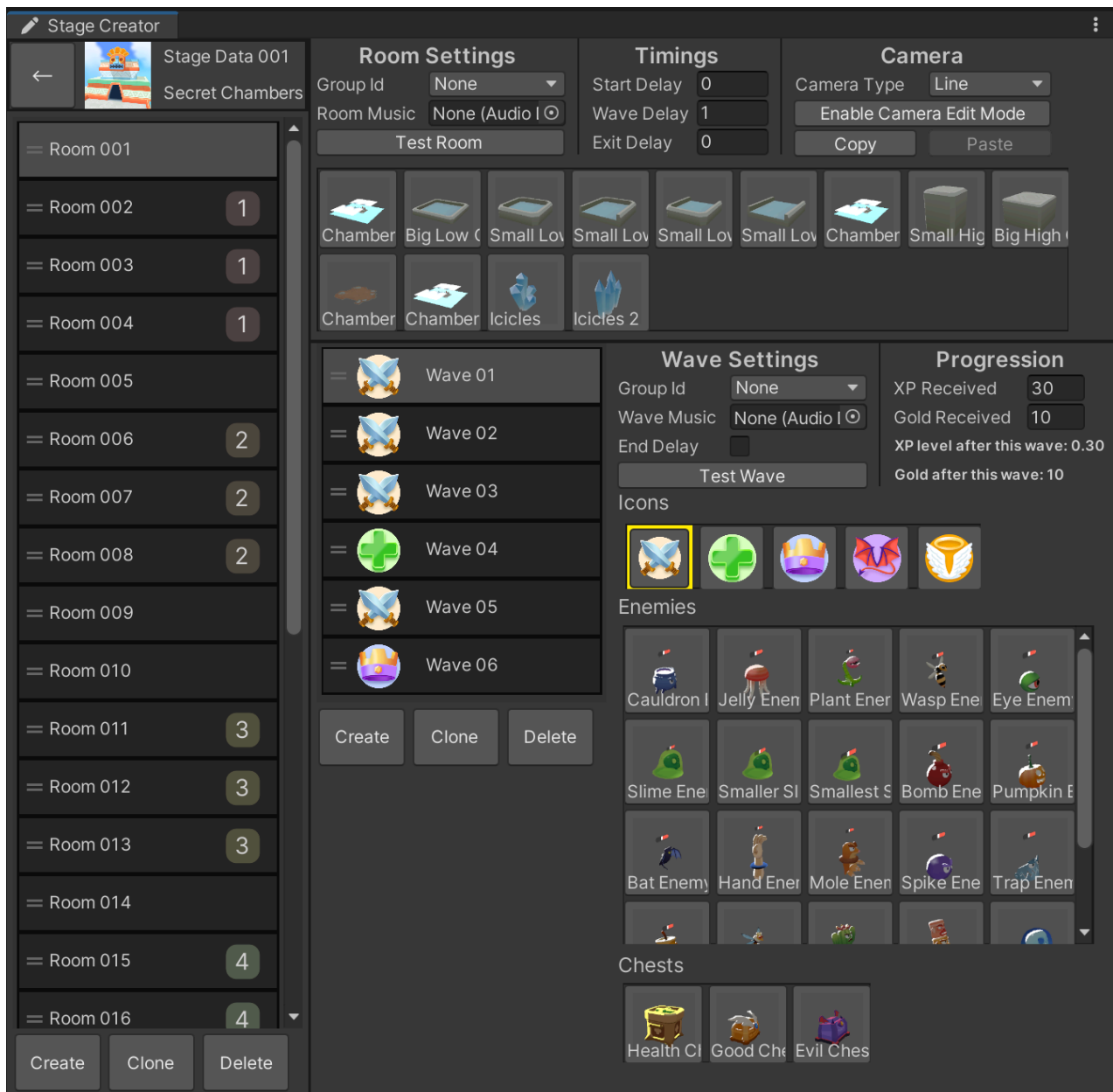
- **Stage Image** - a preview that will be displayed in the main menu.
- **Stage Music** - a main music file that will play on loop in the stage.
- **Use Start Ability** - an option to grant a "level up" at the start of the stage.

Below, you will find the 2 main sets of parameters for easy balancing - Enemy Stage and HP Multipliers. Set up those to gradually increase the difficulty inside this particular stage.

Stage Features

Now, let's take a look inside a stage. To edit a stage, select one and press the Edit button next to the list. Or simply double-click a stage.

You will be presented with a window that should look like this:



Let's dissect what each part does.

Rooms

The list of Rooms is located on the left. Each room can hold a different environment. These are used to give a sense of movement and progression to the game. Every room has a

Room Options

On top, next to this list, you will find several blocks of parameters:

- **Group ID** (Sorting Groups) gives you the ability to randomize all or certain rooms, in any order you like. If a room has a group ID assigned, it will display next to the room number in the list, like this: 
- **Room Music** is a composition that will play in that specific room, resetting to stage music if the next room has no dedicated music assigned.
- **Timings** are designed to give you more control over the flow of the game. For example, if you need a longer pause at the start or the end of the room to display something, etc.

Camera Control

Every room can have a unique shape, direction, etc. For example, you want the camera to move up and down in one room, but the next one has a completely different layout. This is when camera settings come in handy.

With those options, you can set up the camera movement with a few different types of camera constraints:

- **Free** - default unobstructed camera that moves with the player.
- **Line** - linear camera movement, best suited for straight, long rooms (like the default ones).
- **Fixed** - static camera, with a specified center point.
- **Polygon** - custom shape with multiple points for more complex rooms.

Prefab Palette

Below these options, you will find a **Prefab Palette**, a dynamic list of all prefabs used in rooms of this stage. This feature is very useful when you have a set of objects that appear in most rooms (like room environment or obstacles).

When you place a prefab in any room, it will appear in this list for easy access.

Waves

Next to the list of Rooms, you will find a similar list with **Waves**.

Waves are used to spawn various things, mainly enemies, to give the game a sense of flow and progression.

Wave Options

Right to the Waves list, you will find configurable options, similar to Rooms:

- **Group ID** (Sorting Groups) for the waves work separately, but exactly the same as for Rooms.
- **Wave Music** is self-descriptive, plays music on wave start until the wave dies or passes.
- **End Delay** - sets a delay for a particular wave, in case you want to space them out, for example, before a tough boss fight.

Wave Progression

Additionally, wave options feature progression fields - important parameters for balancing:

- **XP** - sets an XP amount received after passing a wave.
- **Gold** - amount of gold granted for passing a wave.

Below those fields, you will see statistics for how much Gold and XP are earned after this wave.

Wave Icon

Waves can feature an icon that is displayed on UI between waves - a way to inform the player of what's coming next.

By default, there are 5 distinct icons - for regular fight, boss, healing chest, good, and evil chests, respectively.

Wave Palettes

For ease of use, waves feature two types of quick access palettes - for enemies/bosses and chests. These palettes consist of every enemy or chest that's registered in their databases. Clicking on an item will place it on the scene.

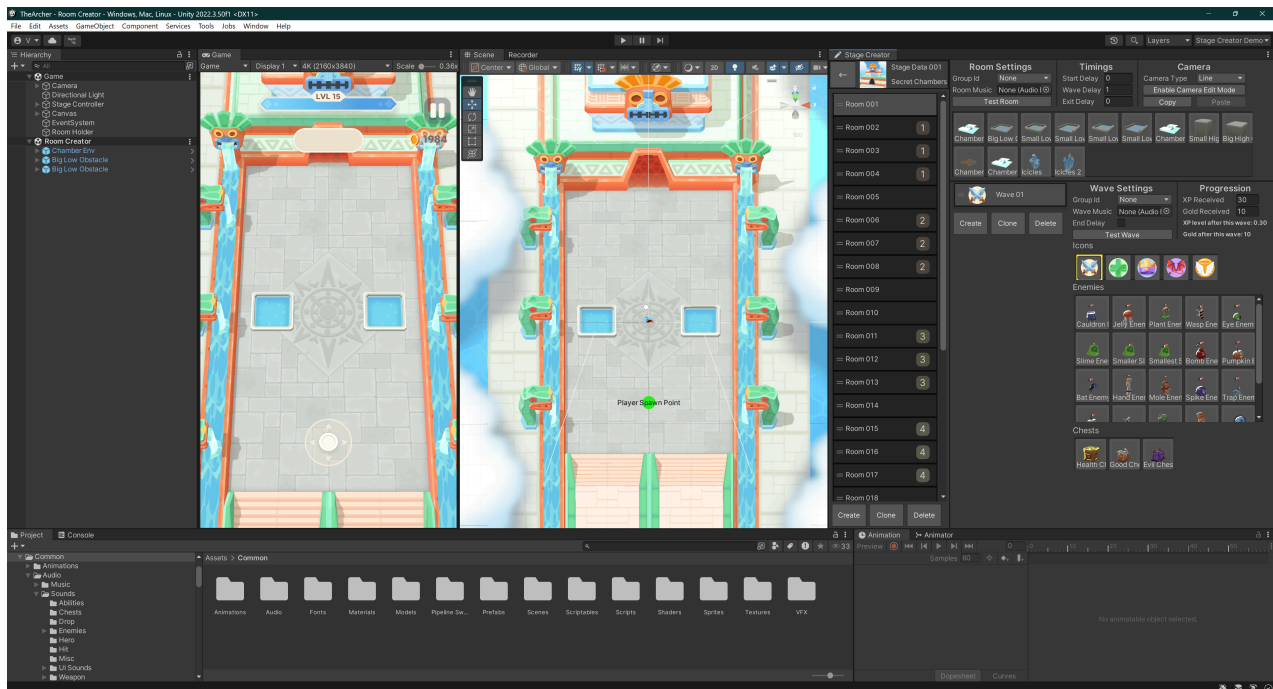
Stage Creator setup

To open Stage Creator window, navigate to the top bar, choose **Tools** → **October** → **Stage Creator**, or press **Shift + S**.

Stage Creator window will now pop up.

Optimal Layout

For the best convenience, its recommended that you create an editor layout. This can be personalized any way that's comfortable for you. This is the layout we used when creating stages:

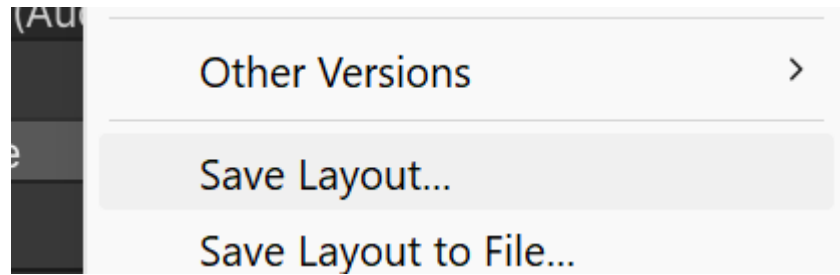
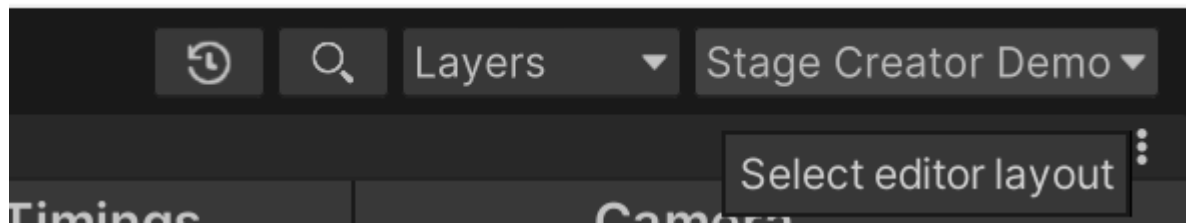


There are 4 windows that you will need the most:

1. Stage Creator window
2. Scene view
3. Hierarchy window
4. Game view

Additionally, Project (folders) and Inspector are also used sometimes.

When you finish setting up your windows, make sure to save this layout. To do it, go to the top right corner, click on the dropdown, select Save Layout, and name it



Room Environment

Learn how to create environments that work with this template.

Preparation

Before you begin, make sure that you have one or several environment prefabs that you plan on using. If you want to use the provided environments, or don't need any for now, you can skip this step.

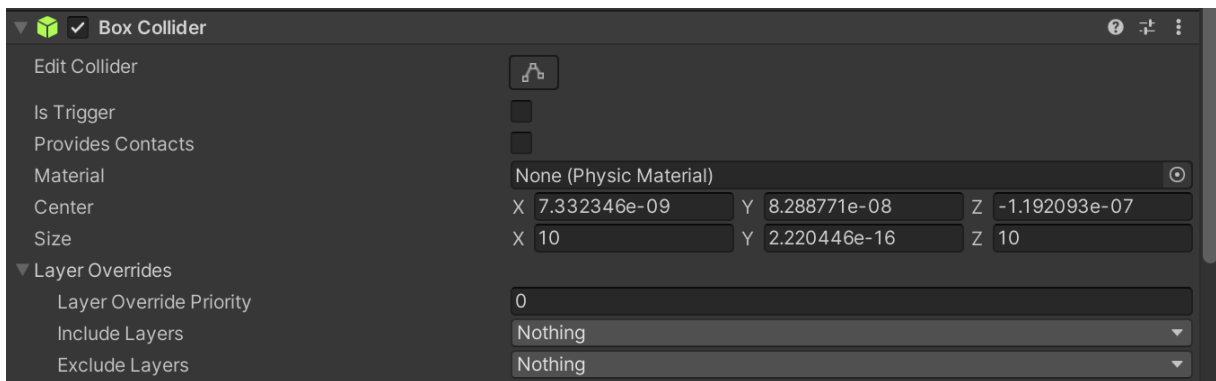
Environment Components

This project uses navmesh for navigation, so it's important to have your Rooms set up properly.

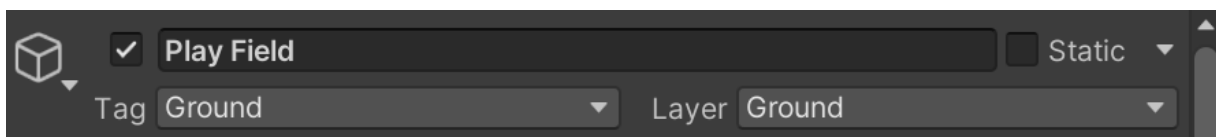
Ground

First things first, make sure that the **ground object** you are using has these things:

1. **Collider** component attached and configured to your ground.



2. **Tag** and **Sorting Layer** set to *Ground*.



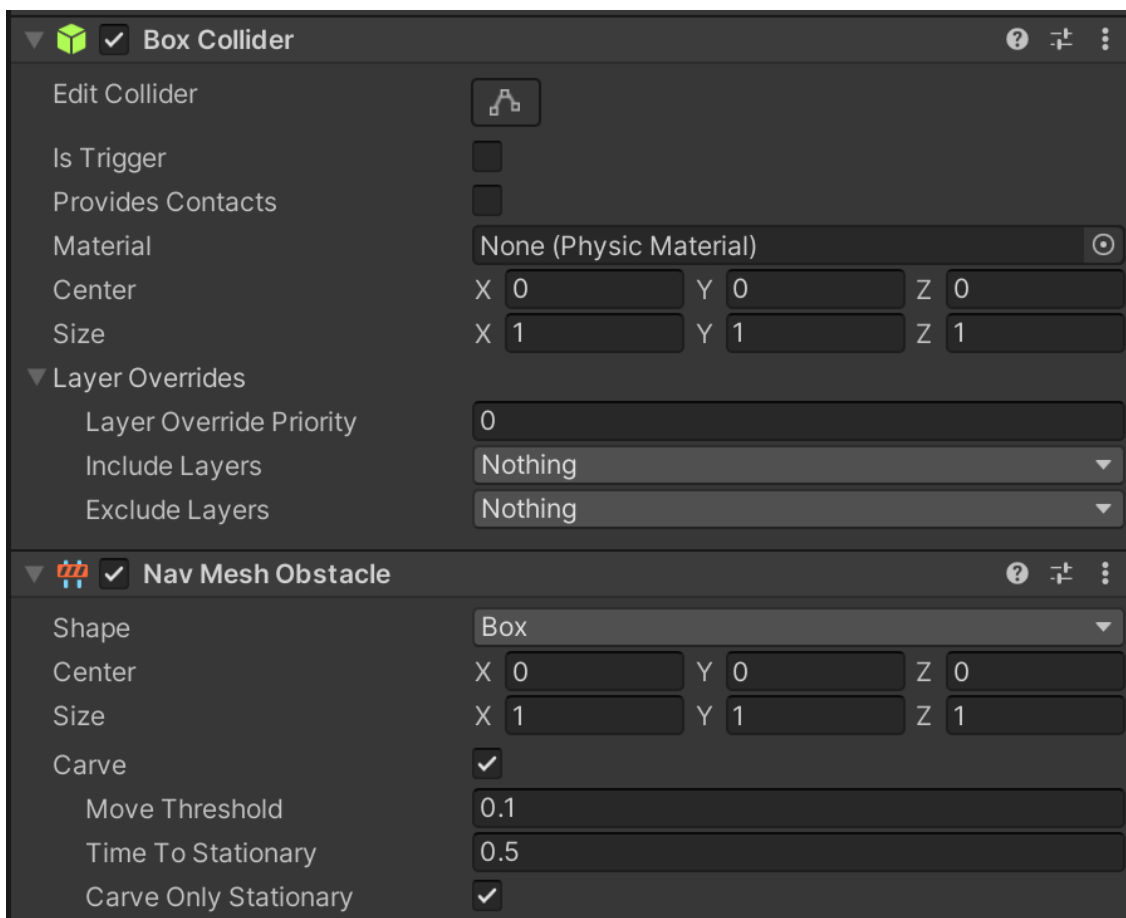
✓ You can use any 3D collider you need, but make sure to fit it to your ground object.

Walls

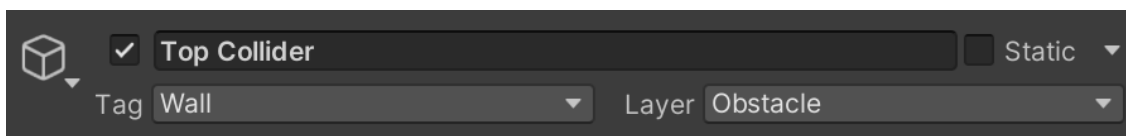
The wall setup is a bit different. Make sure that all of your wall objects have the following settings:

1. Collider and Nav Mesh Obstacle components are attached and configured to your desired sizes.

 Configure all the options for Nav Mesh Obstacle as on the screenshot below.

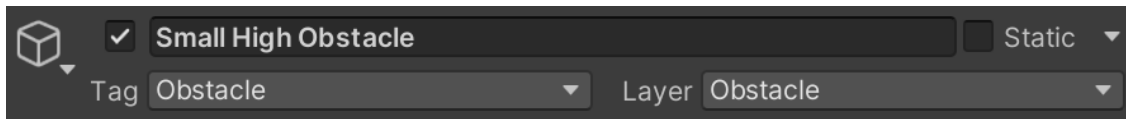


2. **Tag** set to *Wall* and **Sorting Layer** set to *Obstacle*



Obstacles

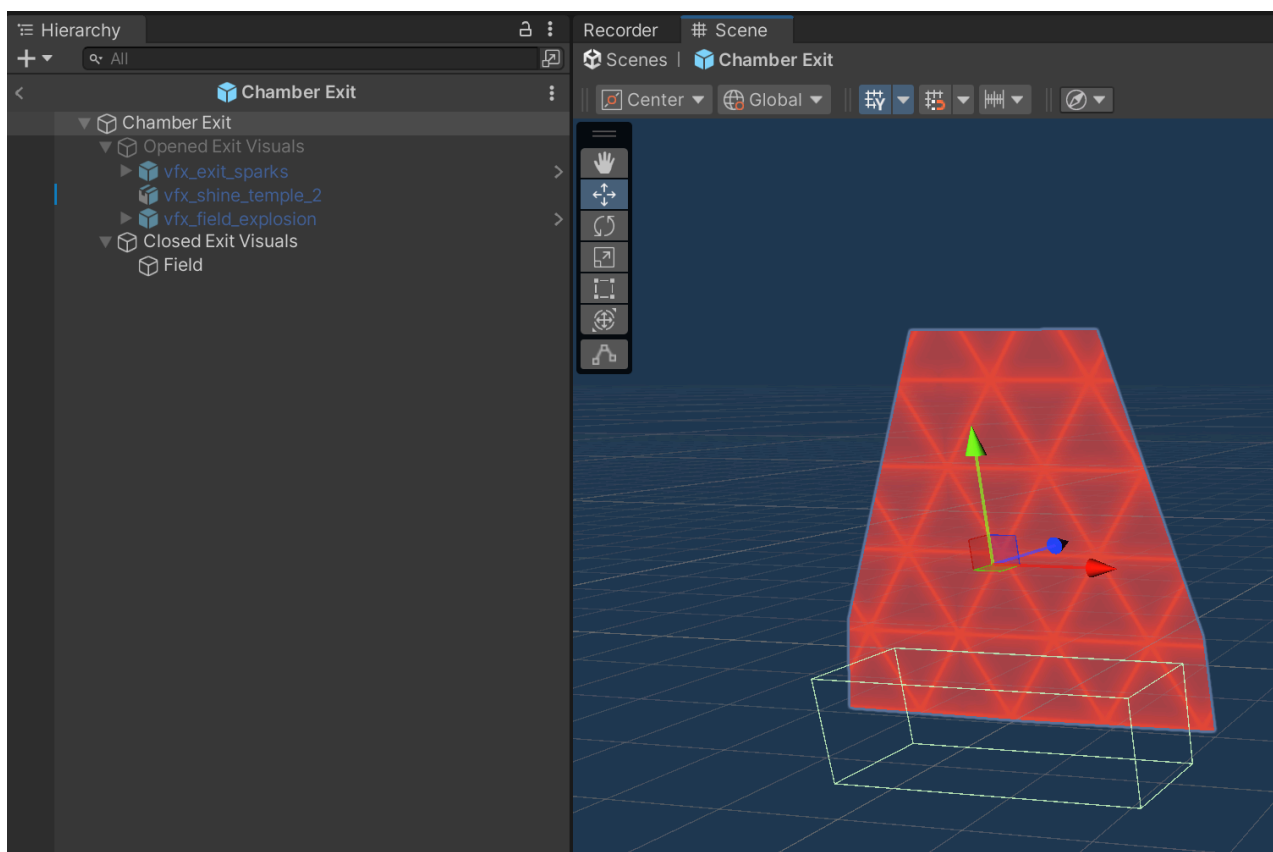
Obstacles are configured the same as walls, with one exception - **Tag** should be set to *Obstacle*



Exit

If you want your room environment to have an exit, you will need a special prefab for it.

Let's take a look at an exit prefab. Go to `Assets/The Archer/Prefabs/Prop/Chamber/Chamber Exit.prefab`



Exit Prefab consists of 3 main components:

1. Parent Object with a script and a collider
2. Opened Visuals
3. Closed Visuals

The logic of this structure is next:

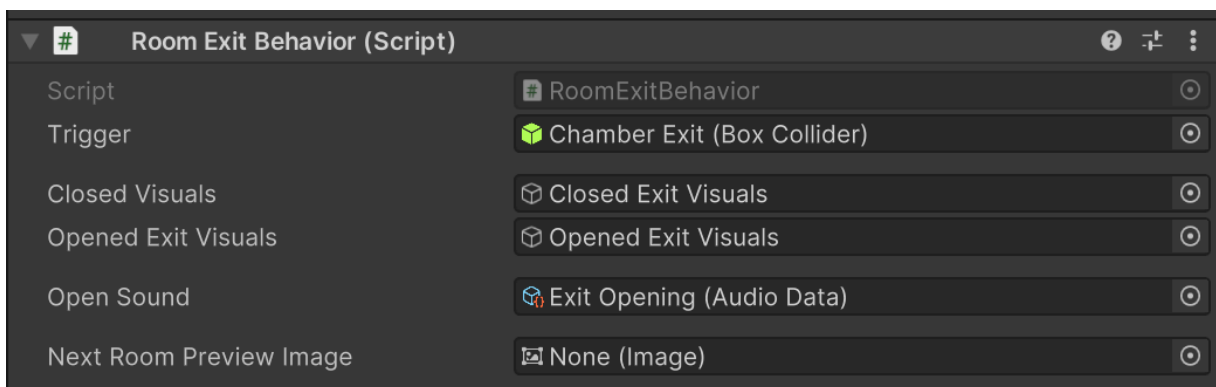
- While in gameplay, the exit displays closed visuals - energy field in this case.
- And at the end of the Room visuals swap to opened - explosion VFX that simulates the field's breaking.

Exit Setup

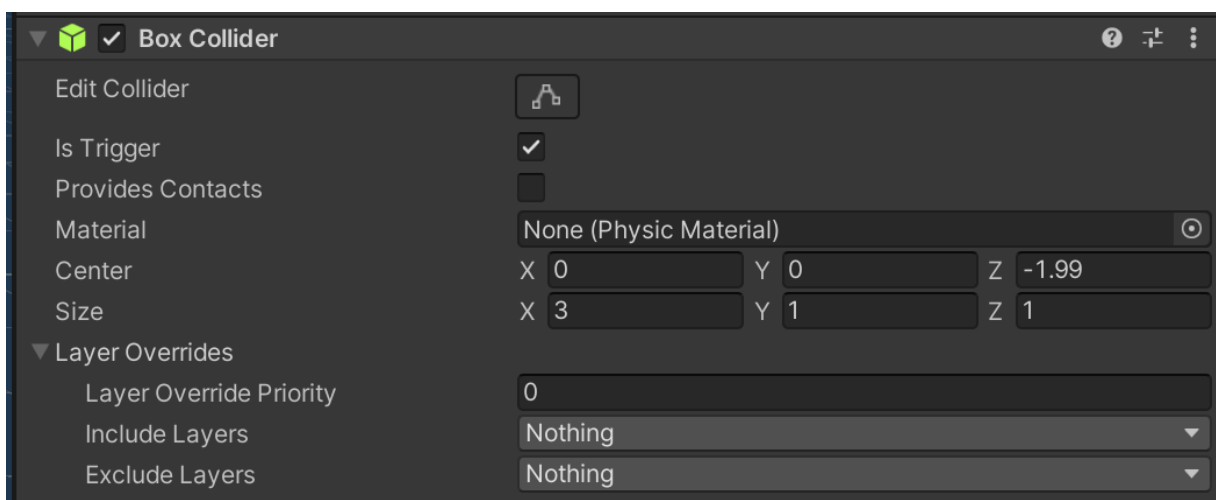
To set up an exit prefab, create a structure as described above - a parent object, and 2 child objects with appropriate visuals inside.

The parent object should have several things:

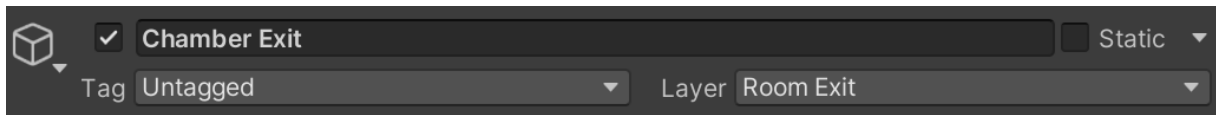
1. **Room Exit Behavior script** attached and configured as follows:



2. **Box Collider** (or other 3D Collider) with appropriate size. This collider will detect the player entering the exit. **Make sure to set it to Trigger as shown below.**



3. **Sorting Layer** is set to *Room Exit*

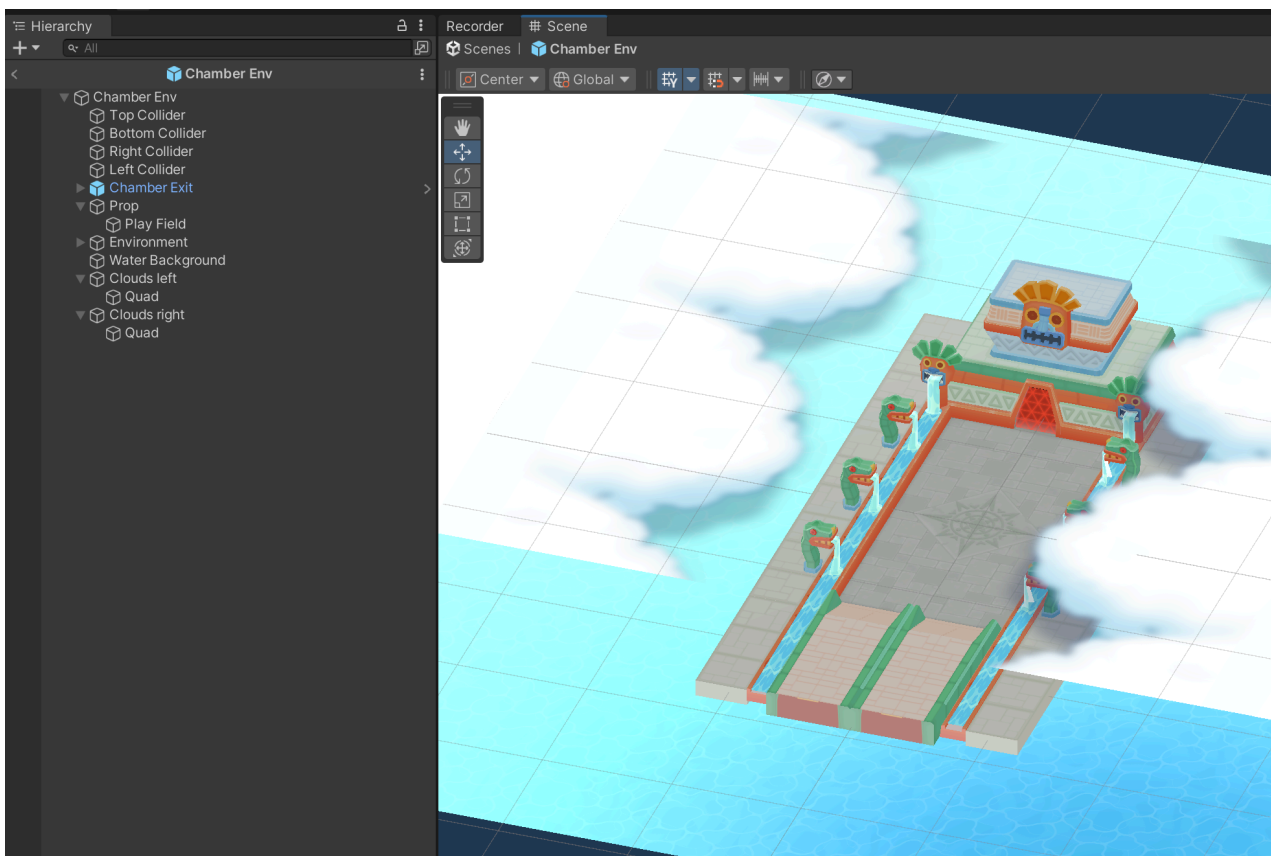


Environment Prefab

For the ease of use we recommend assembling one or several prefab for your Rooms (Environments).

Let's take a look at the provided prefab. Navigate to `Assets/The Archer/Prefabs/Prop`. There you will find several folders with prefabs. Open the Chamber folder and select and open Chamber Env.prefab

When opened, it should look like this:



At its core, this prefab consists of the mentioned objects: **Play Field (Ground), Wall Colliders, and Exit.**

The rest is purely decorative and should depend on the style you are going for.

In this case, there are no obstacles placed beforehand, since this environment will be reused. You can also make a different prefab for each room, but **reuse is recommended** to save on memory for both editor and build.

How to create a Stage

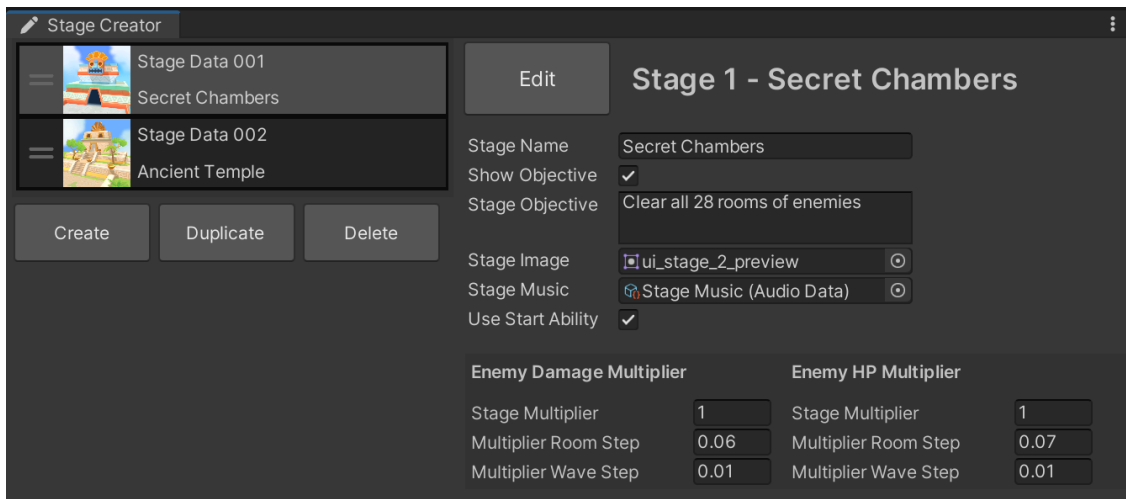
Learn more details about stages and how to create them.

Preparation

Before you begin, make sure that you read all the previous clauses related to Stage Creator and set up your layout and Room environments accordingly.

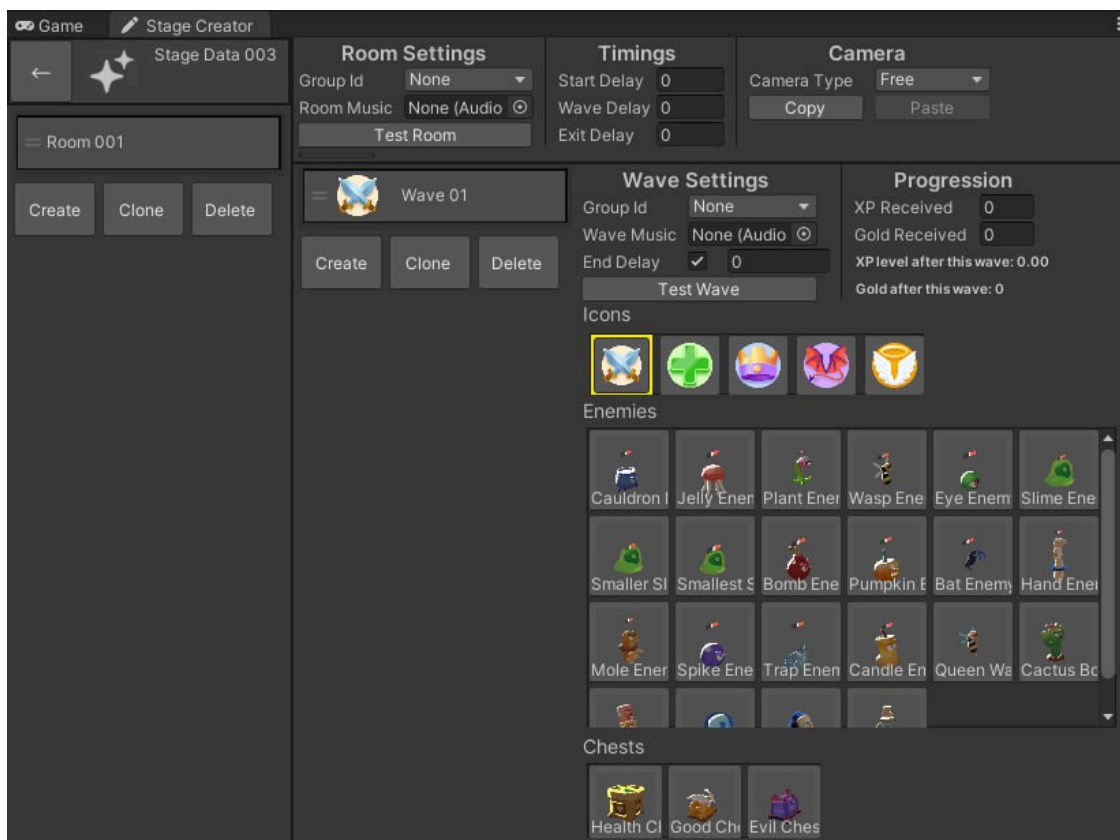
Add a new stage

1. **Open the Stage Creator.** Under the stages list, press *Create*



Optional: Set up all settings you like on the left. [You can return to this step at any time](#)

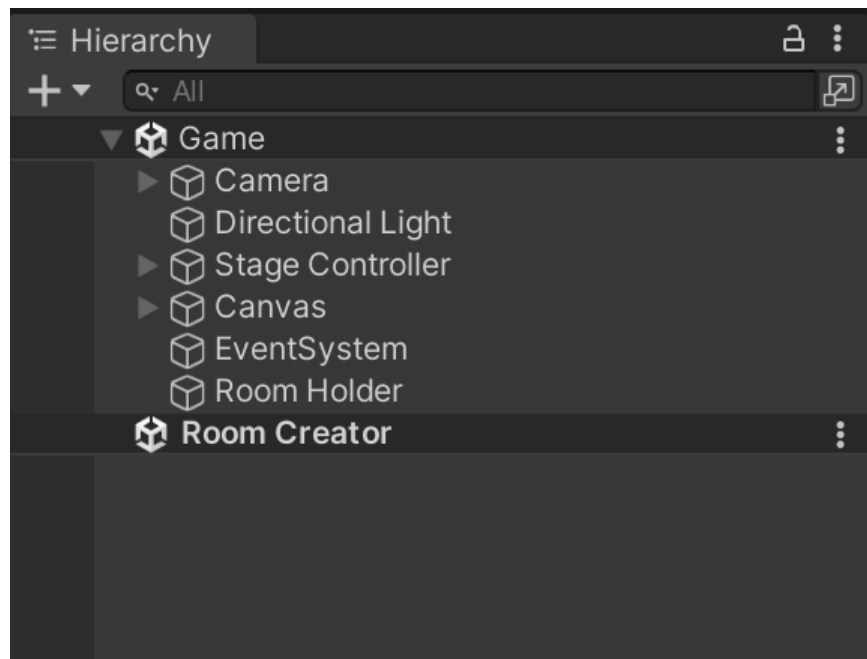
2. Select your newly created stage and press Edit or double-click it.
3. An empty stage will show up.



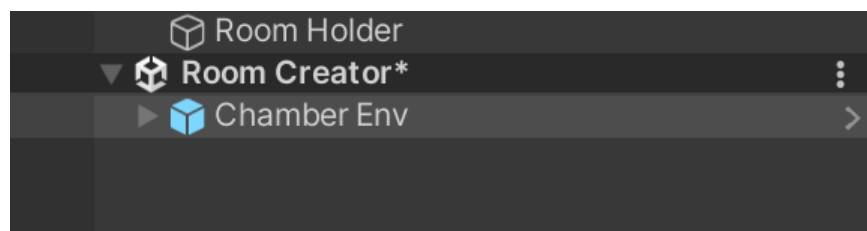
Add a room

The newly created stage will automatically have one empty room with one empty wave in it.

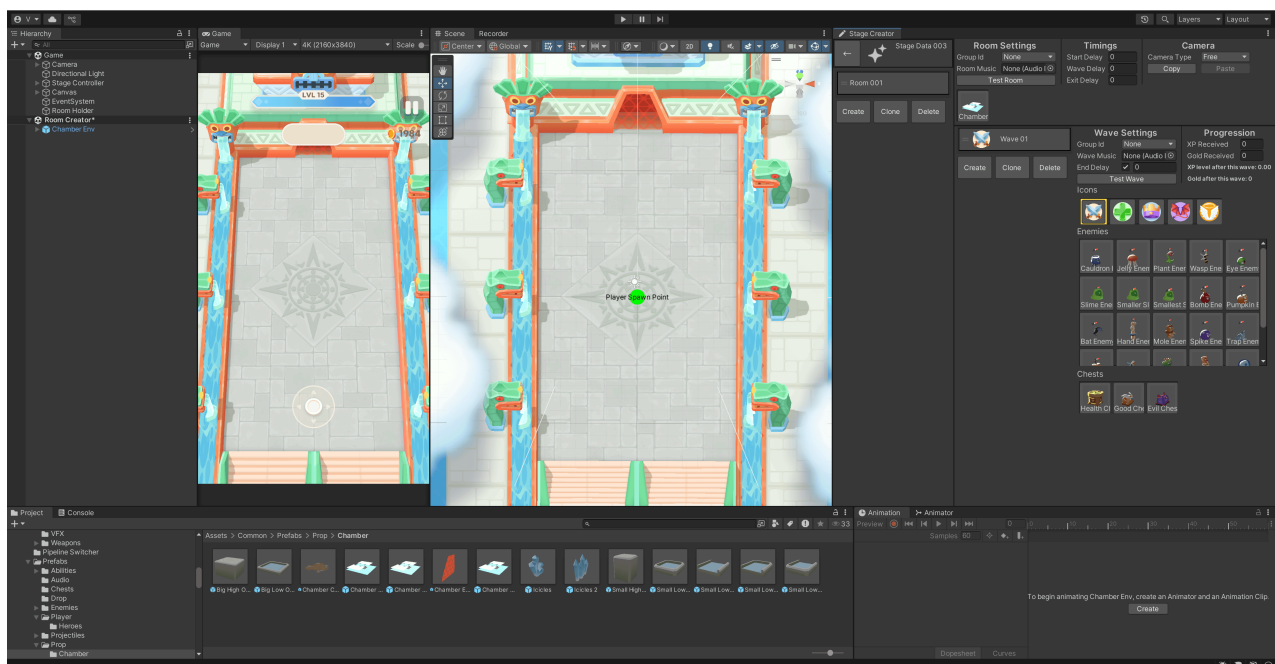
When Stage Edit mode is enabled, it creates an empty temporary subscene **Room Creator**



To place your Environment Room Prefab, simply drag and drop it from your folder



Now, you should see the environment you placed in the scene

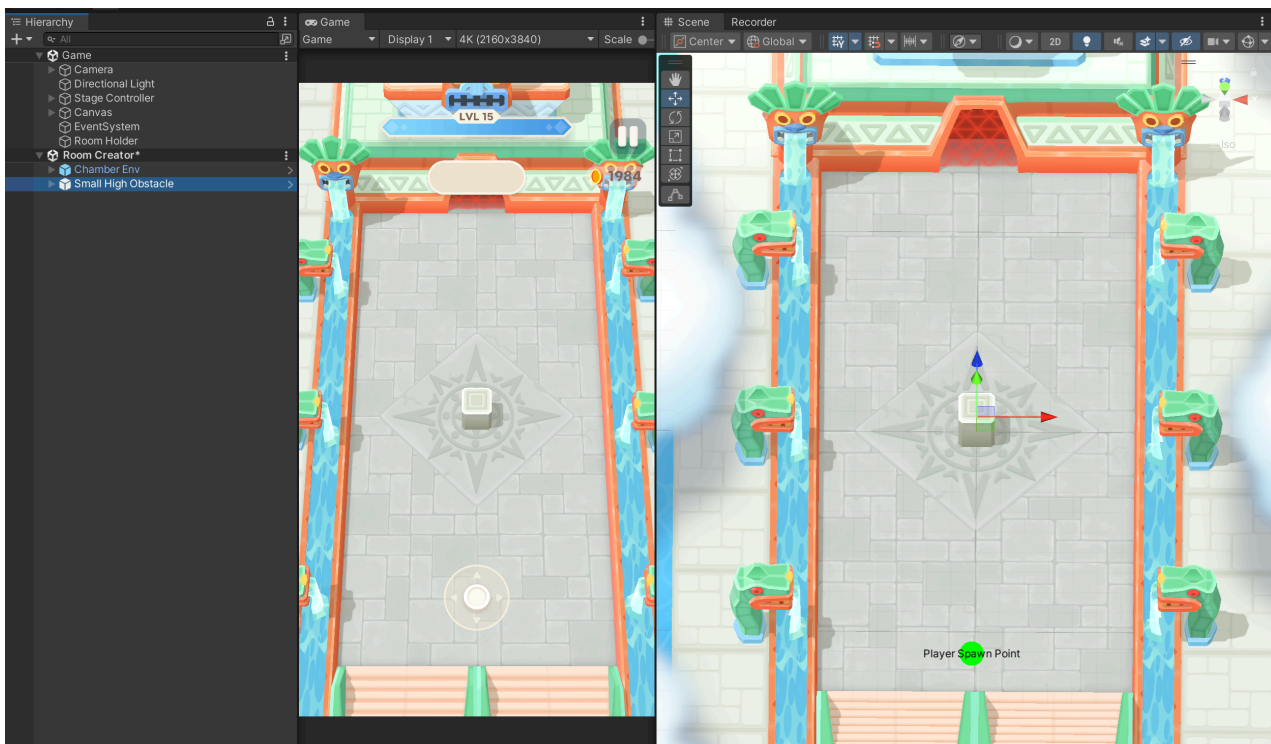


Place obstacles

Placing obstacles for the first time is the same as adding a room environment - drag and drop it from your folders

 Make sure that your environment and obstacles are configured correctly according to [Environment Components](#)

Added obstacle will be placed at the zero coordinates



Now you can add a bunch more obstacles and place them in the scene however you like.

 Stage Creator remembers prefab position, scale, and rotation, so you can adjust your obstacles on a per-room basis. This does not work for objects inside a prefab, however.

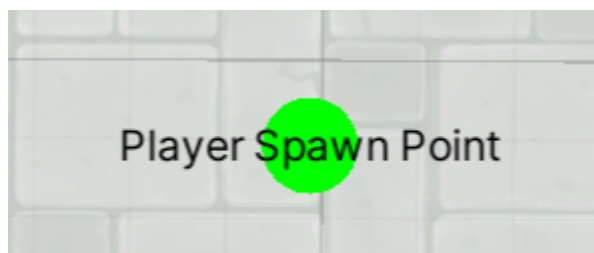
When you place a prefab inside the Room Creator subscene, it will appear in the Prefabs Palette in the Stage Creator:



So, if you added an obstacle once, it will appear there and can be easily spawned with a click. It will be there as long as it's present anywhere on the stage.

Adjust the spawn point

In the newly created room, you can notice an object that looks like a green sphere with "Spawn Point" text on it - move it wherever you like in every room.



Add waves

There are 3 main types of waves in this project:

- Enemy wave
- Boss wave
- Chest wave

These types are not strict and are not limited. You can create new wave types by adding and spawning custom objects, like a merchant; the only limit is your imagination.

- ✓ You can simultaneously spawn enemies, chests, and bosses in a single wave if you like.

Each wave has a different purpose. More on that below.

Enemy and Boss waves

Enemy waves are a most commonly used type, they are the main pillar of rogue-like gameplay.

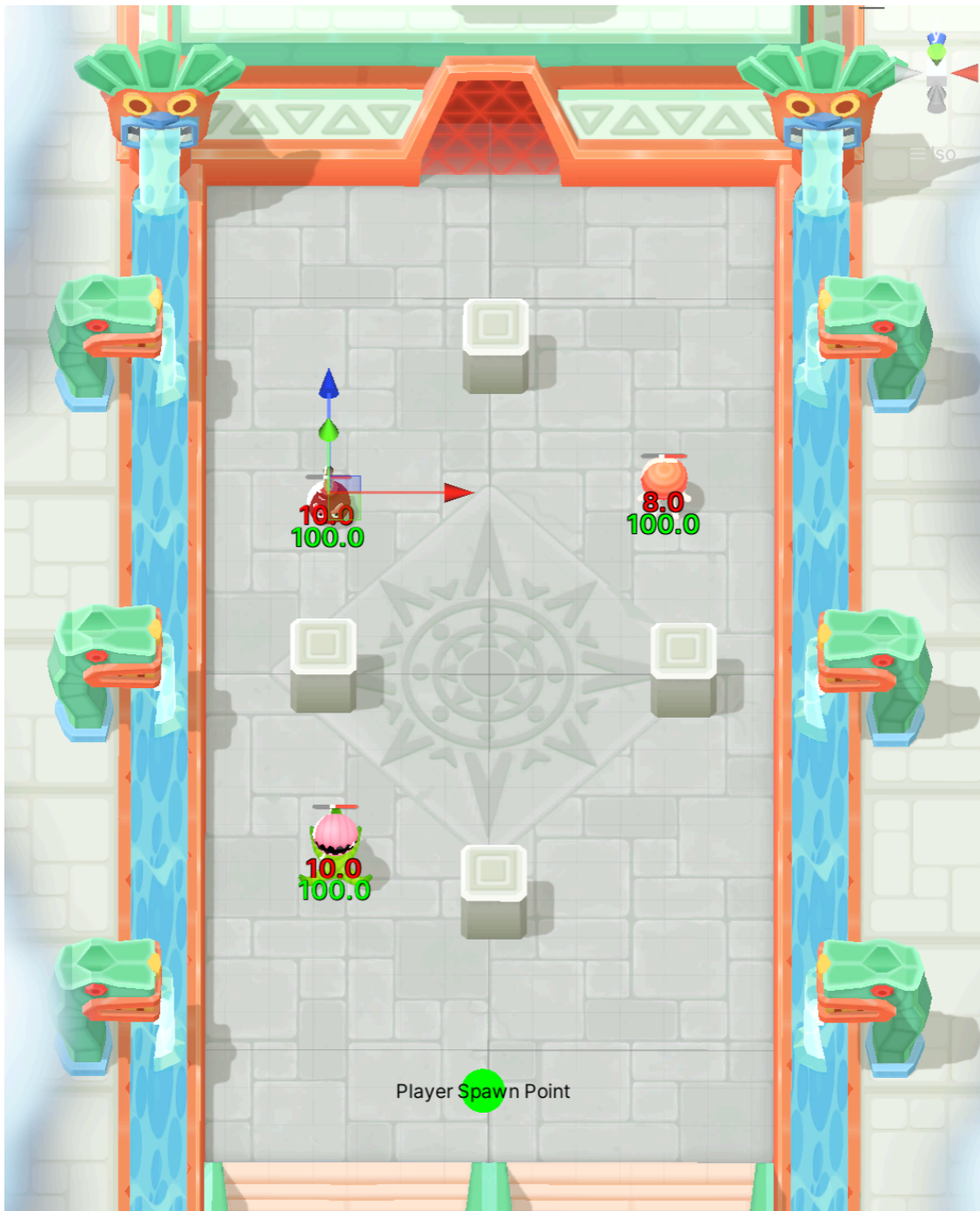
To add an enemy, click on the enemies you want to add on the palette:



When you click on an enemy, it will automatically spawn in the zero coordinates of the Room Creator subscene.

Drag the enemies however you like. Those will be the positions of enemy spawns at the start of the wave.

In the end, it can look like this, for example:



- ✓ If added from the palette, enemies will display their attack **Damage** and **HP** for the current wave. Those values are dynamic and will scale with the progression values you set up in the general Stage settings.

Chest wave

Chest waves are designed to reward the player, and also as a short pause between tense waves.

There are 3 types of chests in the template: **Heal** (offers healing or buff to Max HP), **Good** (Gives a high-tier ability selection), and **Evil** (Offers a valuable ability at a price of your Max HP).

Adding a chest to a wave is the same as for the enemies - simply click on the desired chest from the Palette and place it where you want:

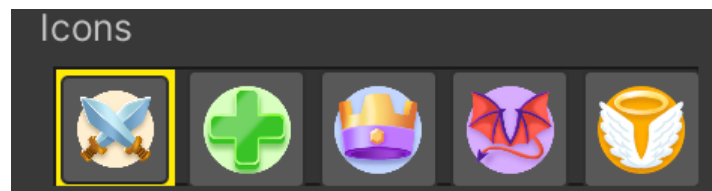


Wave icon

This template offers a carousel in the game UI to warn the player of what's coming next

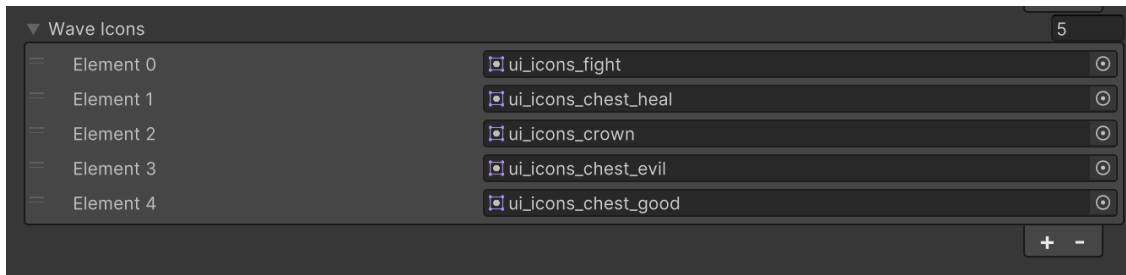


You can assign a wave icon by selecting one from the Icon Palette:



You can change or add these icons in the Stage Database scriptable.

To do that, go to `Assets/The Archer/Scriptables/Stages/Stage Database.asset` . When opened, it will show various stage parameters. Icons database is located at the bottom of this file:



Stage flow

Now, when you understand the basics of Rooms and Waves, it's important to imagine the approach you want to go with:

1. Single room with all the waves in it
2. One wave per room
3. Multiple waves per room
4. Hybrid of everything mentioned

Provided stages use a different approach for each: Secret Chambers have a single wave per room, but increasing to 2 per room in the late stage, while Ancient Ruins have only one room with all waves in it.

- ✓ For the game to feel dynamic, it's good to balance all the wave types. For example, you can create 5 fight waves, then a chest wave, followed by another 5 fight waves, finishing with a boss.

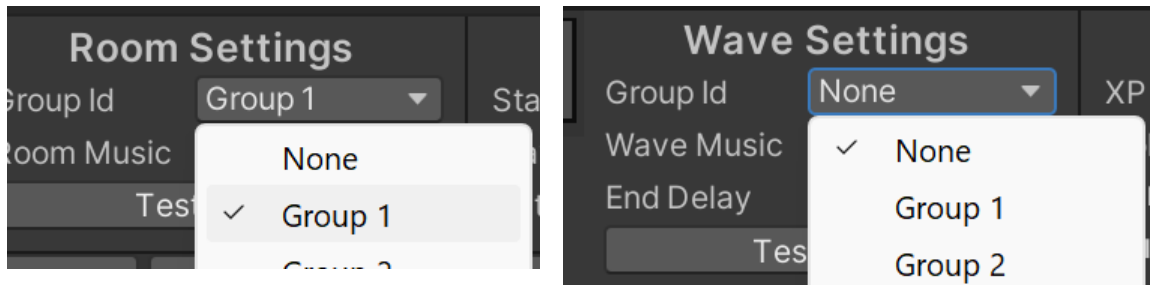
Sorting groups

One of the main pillars of rogue-like and -lite games is randomization. It can give a feeling of freshness and unpredictability even when the player repeatedly plays the same stages.

This is where Sorting Groups come in handy.

By default, all rooms and waves go one after another in a linear fashion, like they are placed in the Stage Creator.

Let's dissect how they work. You can assign a Group ID or both Rooms and Waves.



When multiple Rooms or Waves share the same Group, their order will be randomized once the player enters the first room in that group. This randomization only applies within the group and only if the Rooms or Waves appear consecutively in the Stage Creator.

 Sorting Groups only affect the order, so every Room or Wave from this group will still show up.

If a Room or a Wave has a Group ID set to *None*, it will play normally, without randomization. Lets take a look at the example below:

=		Wave 01	
=		Wave 02	1
=		Wave 03	1
=		Wave 04	1
=		Wave 05	
=		Wave 06	2
=		Wave 07	2
=		Wave 08	2
=		Wave 09	2
=		Wave 10	
=		Wave 11	
=		Wave 12	3
=		Wave 13	2
=		Wave 14	3

As you can see, if you want a Room or a Wave to always appear (like a guaranteed heal or a boss fight), you can leave its Group ID at *None*.

Testing

To quickly test a Room or a Wave, click on the buttons "Test room" or "Test wave" in the Stage Creator.

Alternatively, you can just press the regular Play button in the Editor, and it will play the selected Room or Wave. But keep in mind that it only works when Stage Creator is opened and in focus.

Tools

Abilities Tester

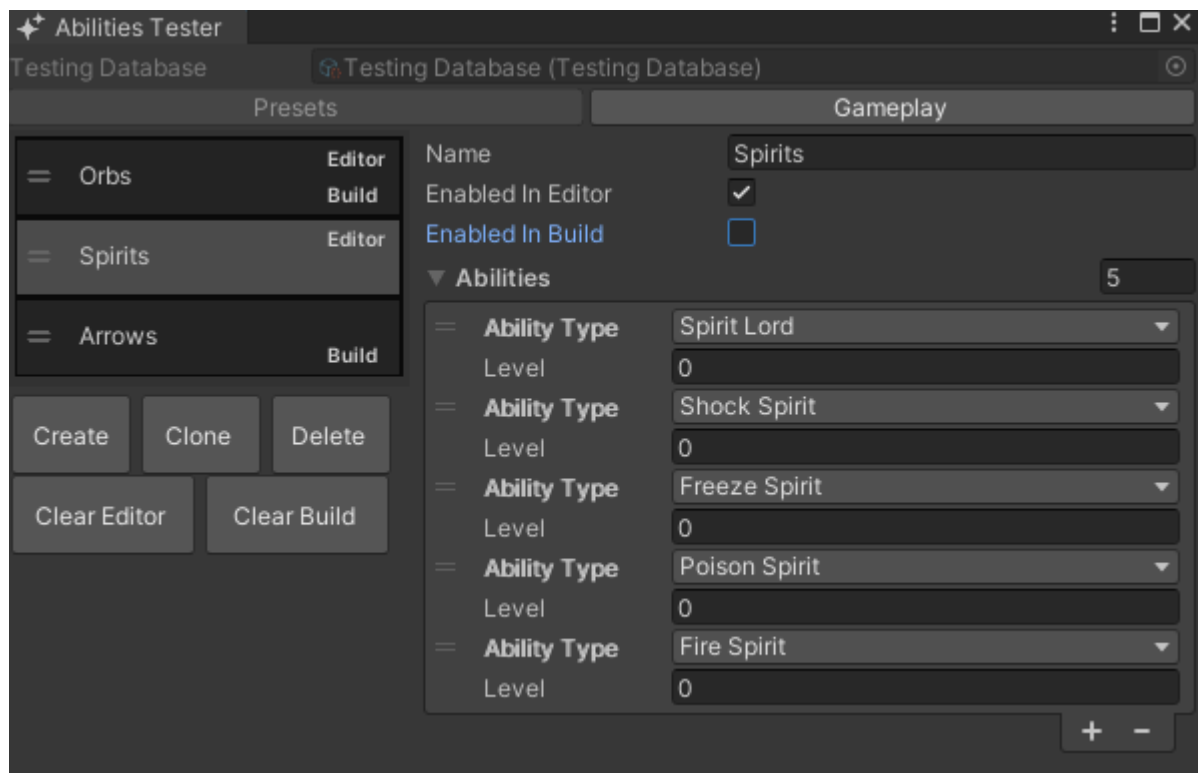
Overview

The **Abilities Tester** is a flexible tool that allows you to **test and balance abilities** with ease.

To open it, go to **Tools → October → Abilities Tester**, or press **Shift + A**.

The tool is divided into two main sections: **Presets** and **Gameplay**.

Ability Presets

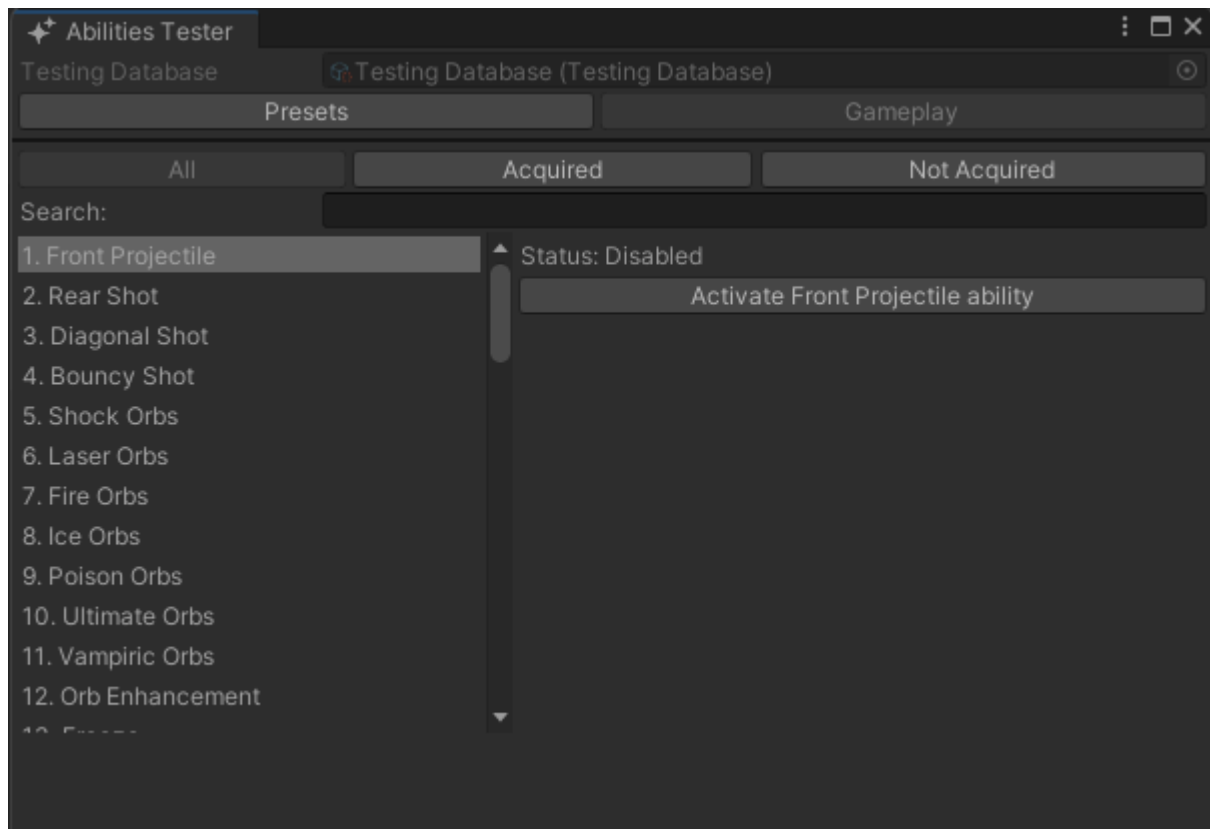


Presets Tab

In this section, you can **create and modify sets of abilities** that will automatically activate when the game starts - both in the **Editor** and/or in **builds**.

You can organize abilities into **presets** and **rename** them for easier management and comparison.

Gameplay Abilities Tester



Gameplay Tab

During gameplay, this tab allows you to **activate, disable, or change the level** of any ability in real time.

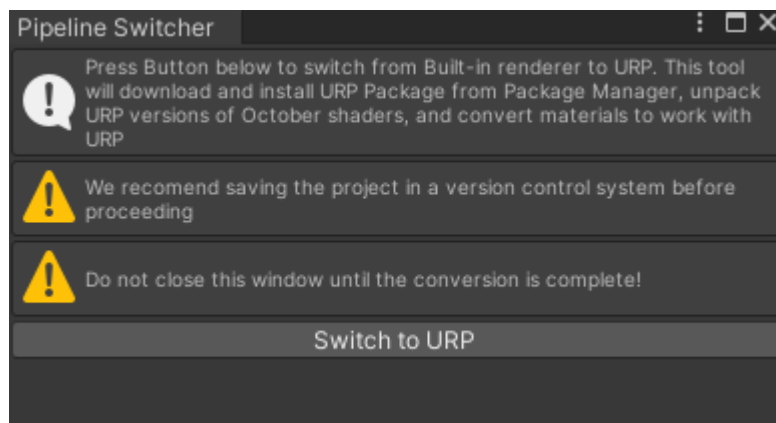
Since the template includes **80 abilities**, the tester also provides **filter options** to quickly find specific ones - by **name**, or by whether they are **acquired** or **not acquired**.

Pipeline Switcher

Overview

The **Pipeline Switcher** is a custom tool that allows you to switch from the **Built-in Render Pipeline** to the **Universal Render Pipeline (URP)** with a single click.

To open the tool, select **Tools → October → Pipeline Switcher**.



⚠ Before performing the switch, we **strongly recommend** saving your project progress using a **version control system** (e.g., Git). This will allow you to easily revert any changes if you decide to return to the Built-in Render Pipeline later.

How does it work

When you press the **"Switch to URP"** button, the tool performs the following steps in sequence:

1

Installs URP

Downloads and installs the **Universal RP package** from the Package Manager (if not already installed).

2 Recompiles Scripts

Unity recompiles all scripts to ensure compatibility with the new package.

3 Unpacks URP Assets

The tool automatically unpacks the **“The Archer URP Assets.unitypackage”**, which contains:

- Custom URP shaders
- A preconfigured **URP Asset**
- Material converters
- An updated version of the **Pipeline Switcher Window**

4 Recompiles Scripts Again

Another recompile occurs to register the newly unpacked assets.

5 Assigns URP Asset

The tool assigns the new **Universal Render Pipeline Asset** in **Graphics Settings**.

6 Converts Materials

All materials using the **October** and **StandardUnlitScrolling** shaders are converted to their URP equivalents.

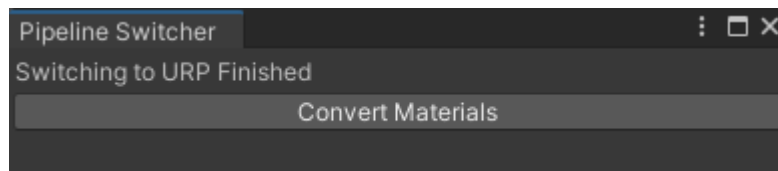


Do not close the Pipeline Switcher window until the entire process is complete.



If the **Universal RP package** is already installed, the tool will skip step 1 and begin from step 3.

If you’ve manually unpacked the **URP Assets package** or created new materials using Built-in **October** shaders after switching, you can use the **“Convert Materials”** button in the Pipeline Switcher to process them individually.



-  The Pipeline Switcher **only converts** materials that use **October** and **StandardUnlitScrolling** shaders. Materials using any other shaders will not be affected.

Armory

Armory Overview

This template features Inventory (called Armory in the game). It allows the player to unlock and equip new items and characters, upgrade them, and create stronger builds for easier runs.

Armory Features

Armory is presented as a part of the Main Menu:



Here are the main features:

- **3D Hero Preview**
- **Hero Stats**
- **4 Equip Slots** (Weapon, Headwear, Armor, and Accessories)

Below, there are 2 tabs:

1. **Items** - a grid of all acquired items, sorted by rarity
2. **Heroes** - all available Heroes (both locked and unlocked)

Each Individual item card has additional information:



- **Rarity** - represented by the background color.
- **The Item Type** is in the right corner.
- **Item Upgrade Level** is in the other corner.

Hero cards, on the other hand, feature only a Hero Level Indicator.



Expanded Armory Card

Clicking on any item in the Item or Hero grid will open the said item.

Let's take a look at the expanded Item view.



In the top portion, you will see all the main info for this Item:

- **Item or Hero Preview**
- **Rarity**
- **Name**
- **Upgrade Level**
- **Main Stats** and their value after the next upgrade.

✓ You can display different Main stats on a per-level basis.

Below, you will find an attached Ability. These abilities also feature an indicator on which level they will be unlocked.

✓ You can attach any number of abilities to any upgrade level. For example, you can have a certain ability granted on one level, but changed to a completely new one on the next upgrade.

And below it, there are the Equip and Upgrade buttons as well.

Armory System Structure

The **Armory** is a flexible and extensive system that manages **heroes and items**, their **stats**, **upgrade levels**, **equipped status**, and more.

The system is divided into three main parts: **Data**, **Management**, and **UI**.

Item Data

Each item in the game is defined by an **Item Data** asset, which contains the following fields:

- **Item Type** - Defines the item category. There are four types: **Weapon**, **Armor**, **Helmet**, and **Ring**.
- **Item Name** - The display name used in the UI and during item pickups in gameplay.
- **Icon** - The sprite shown in UI elements.
- **Is Default Item** - When enabled, this item is automatically selected the first time the player launches the game.
- **Is Unlocked by Default** - When enabled, the item is available from the start without requiring a purchase.
- **Levels** - A list of upgrade levels for the item.

Weapons are represented by the **Weapon Data** class, which inherits from **Item Data** and adds two extra fields:

- **Weapon Prefab** - The prefab that defines the weapon's model and behavior. It will be placed in the hero's hands during gameplay.
- **Is Right Hand Weapon** - Determines which hand holds the weapon. If enabled, it's placed in the **right hand**, otherwise, in the **left**.

Item Level

Each item level defines how the item improves as the player progresses.

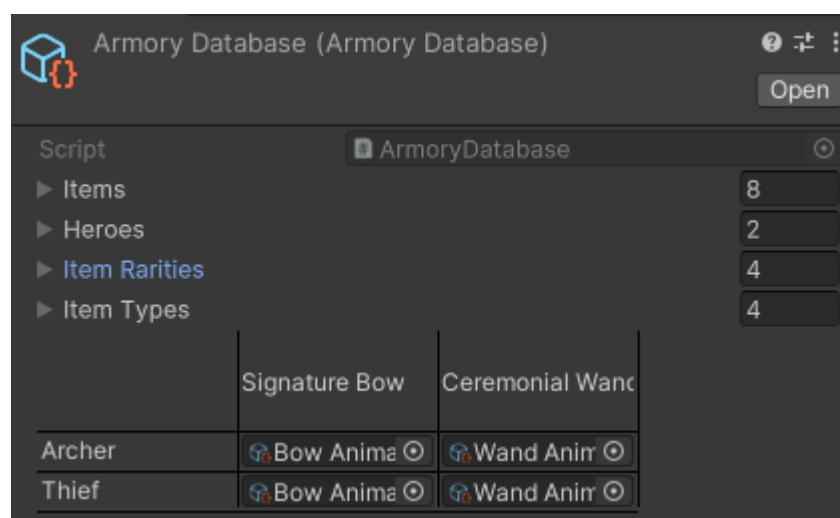
- **Cost** - The gold cost required to unlock or upgrade the item. The first level's cost defines the unlock price. If the item is unlocked by default, this can be left at 0.
- **Item Rarity** - One of: **Common**, **Rare**, **Mystic**, or **Legendary**. This affects UI sorting and visual appearance but not gameplay balance.
- **Stats** - A list of stat modifiers (HP, Damage, Attack Speed, etc.). When the item is equipped, these values are added to the hero's base stats.
- **Attached Abilities** - Abilities that automatically activate when the player starts a run with this item equipped.

i Item Level values **do not stack**.
 For example, if Level 1 grants +100 HP and Level 2 grants +120 HP, the item's HP bonus becomes **120**, not **220**.
 Similarly, if a stat or ability exists at Level 1 but is missing at Level 2, it will be **lost** after upgrading.

Hero Data

Hero data is described in detail in the [Heroes Overview](#) section.

Armory Database



Armory Database

The **Armory Database** is a central asset that stores all data used by the system. It contains the following fields:

- **Items** - A list of all Item Data assets available during gameplay. Only items added here will appear in the Armory and game.
- **Heroes** - A list of all Hero Data assets. Only heroes assigned here are accessible to the player.
- **Item Rarities** - A list used by the UI to retrieve colors and sprites for each rarity.
- **Item Types** - A list used by the UI to retrieve sprites for each item type.
- **Animatin Sets Grid** - A grid that defines which Animation Set should be used for each Hero–Weapon combination. (See [Heroes Overview](#) and [Adding a New Hero](#) for more details.)

 Each **Item** and **Hero** asset has a **unique ID**, generated automatically upon creation. If you delete and recreate an asset, even with the same name, it will be treated as a **new unique entity** by the system.

Armory Manager

The **Armory Manager** is the main class of the system and acts as a bridge between **data**, **saves**, **UI**, and **gameplay**.

- It can be accessed globally via `GameController.ArmoryManager`.
- It resides in the **Main Menu** scene, under the **Game Controller** GameObject.
- All of its methods are **public** or **protected**, and all are **virtual**, allowing easy extension of the system's functionality.

The Armory Manager handles:

- Loading and saving equipped items and heroes
- Providing data for the UI
- Managing upgrades, purchases, and unlocks
- Applying stat changes and ability bonuses during gameplay

How to add a new Weapon

This page provides a step-by-step guide on how to add a new weapon to the project.

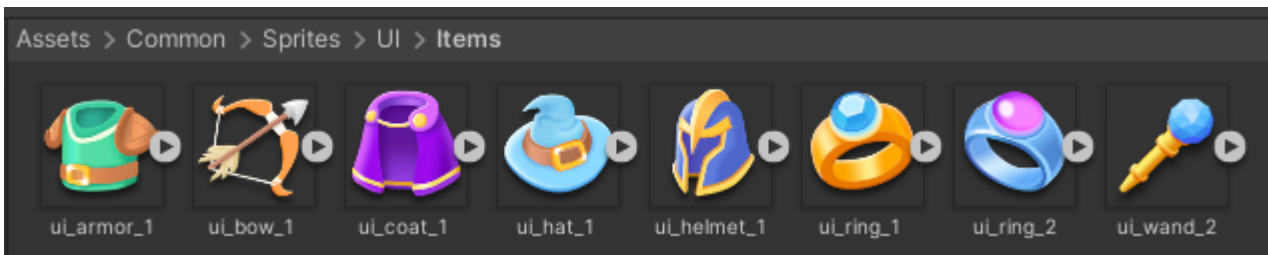
Importing assets

First, you'll need to import the following assets for your item:

- **Sprite (icon)**
- **3D model of weapon**
- **Shoot Animation (Optional)**
- **Texture**

Folder setup

1. Inside **Assets/The Archer/Sprites/UI/Items**, import your item's sprite and set its **Texture Type** to **Sprite (2D and UI)** in the **Inspector**.



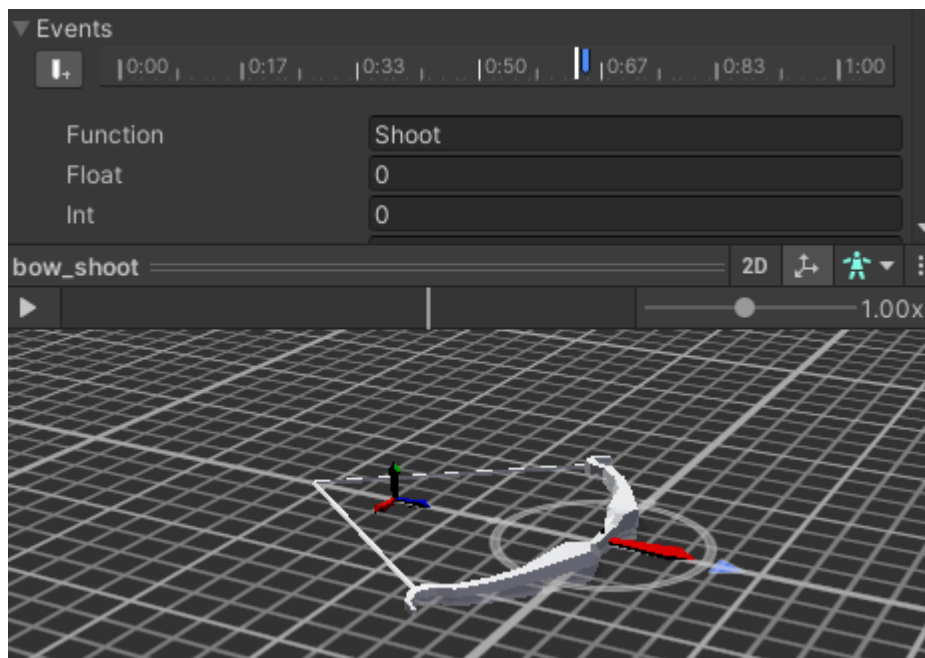
2. Inside **Assets/The Archer/Models/Weapons**, create a **new folder** named after your weapon. Import the 3D model and animation clip there.
3. Inside **Assets/The Archer/Textures/Weapons & Items**, import your weapon's texture.

Configuring Animation (Optional)

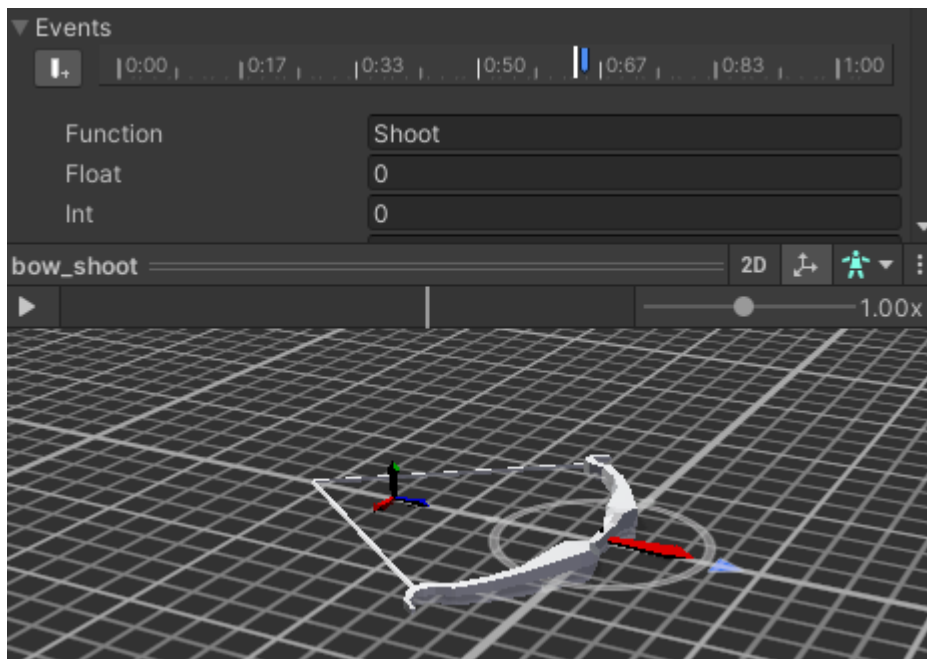
Weapons can function with or without animations.

The weapon's shooting animation should match the length of the hero's shooting animation.

1. Open the animation's **Import Settings**.
 - Ensure there are **no warnings or errors** in the *Import Messages* section.
2. Add an **animation event** at the desired frame:
 - Set the **Function** field to `"Shoot"`.
 - The weapon will handle projectile spawning **only when the Flying Weapon Ability is enabled**, otherwise, projectiles are spawned by the hero's animation.
- 3.

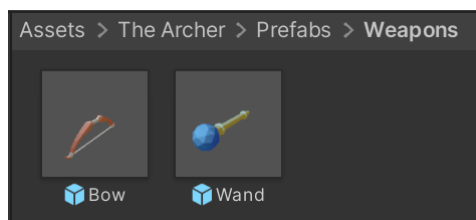


4. Clone an existing **bow animator**, or create a new one.
 - Place it in the folder you created in the previous step (next to the model asset).

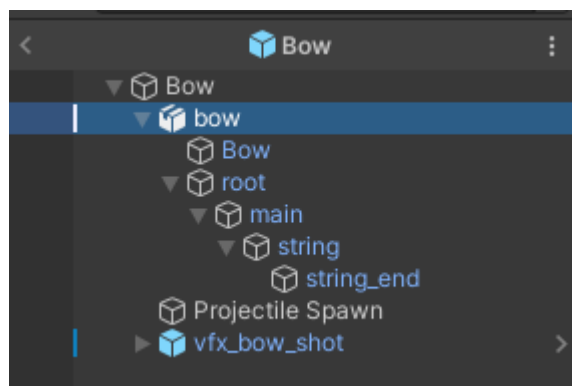


Creating Weapon Prefab

1. Navigate to **Assets/The Archer/Prefabs/Weapons**.

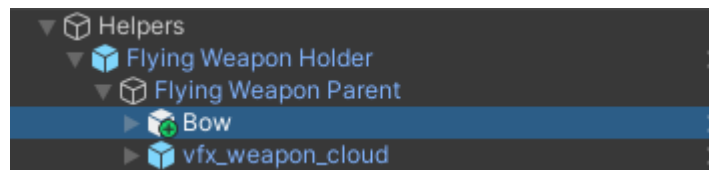


1. Clone an existing weapon prefab and open it.
2. Delete the old weapon model and drag your own model into the prefab.



4. If your weapon uses animations:
 - Add an **Animator** component.

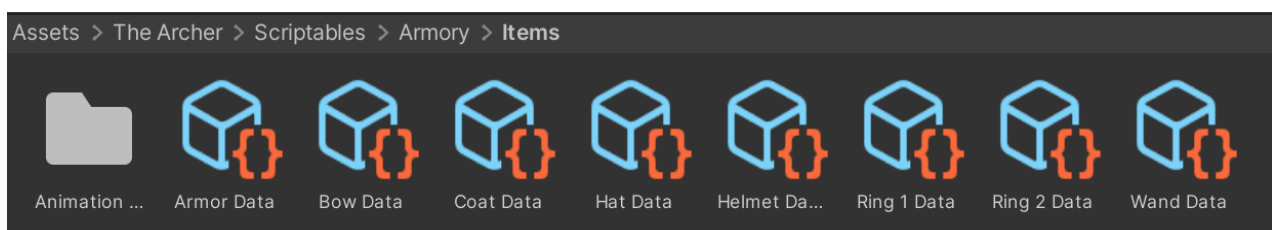
- Assign the animator from the previous step.
 - Add a **WeaponEventsHandler** component.
5. Select the **root object** of the prefab.
 - If your weapon has an animator, assign it here.
 6. To configure **Detached Position, Rotation, and Scale**:
 - Open any **Hero Prefab**.
 - Drag your new weapon prefab into the **Weapons Parent** GameObject inside the **Weapon Holder**.



- Adjust its position, rotation, and scale as desired.
 - Write down these transform values.
 - Remove the weapon prefab from the hero, close the hero prefab, and reopen the weapon prefab.
 - Enter the recorded values into the **Detached Position, Rotation, and Scale** fields.
7. Move the **Projectile Spawner** GameObject to the position where you want projectiles to spawn.

Creating Weapon Data

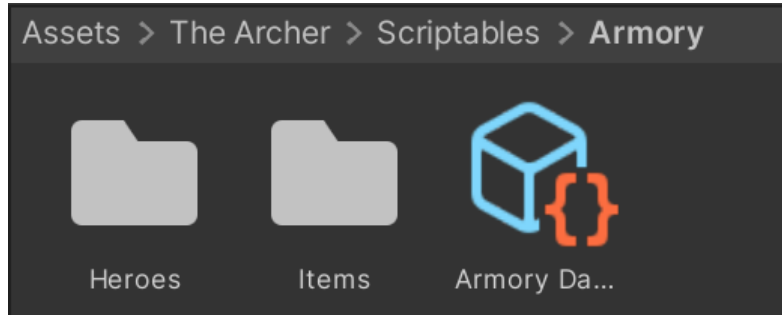
1. Open **Assets/The Archer/Scriptables/Armory/Item.**



2. Clone an existing weapon data asset, or create a new one via:
Right-click → Create → October → Armory → Weapon Data
3. Rename the new asset.

- Fill out the data fields according to the [Item Data](#) description from the [Armory Overview Page](#).

Item Type should be set to Weapon, Icon - to a sprite you've imported at the beginning.



- Open **Assets/The Archer/Scriptables/Armory**, select the **Armory Database** asset, and add your new weapon data to the **Items** list.
- At the bottom of the Inspector, you'll find the **Animation Sets** grid.

	Signature Bow	Ceremonial Wand
Archer	Bow Anima	Wand Anim
Thief	Bow Anima	Wand Anim

- A new column for your weapon will appear, copied from the previous weapon.
- You can either select existing animation sets that best fit your weapon or create new ones to make it unique.

That's it!

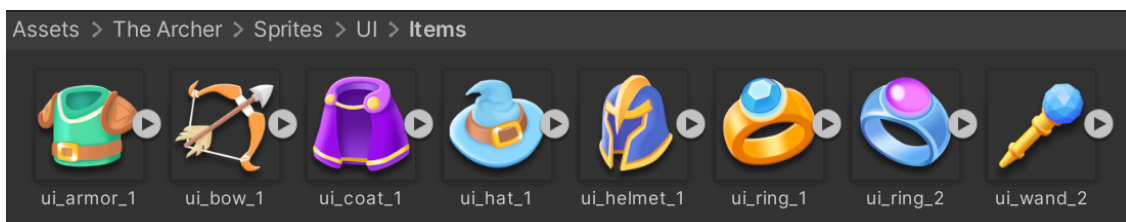
When you launch the game, your new weapon should now appear in the **Armory Page** of the **Main Menu**.

How to add a new Item

This page provides a step-by-step guide on how to add a new item (not a weapon) to the project.

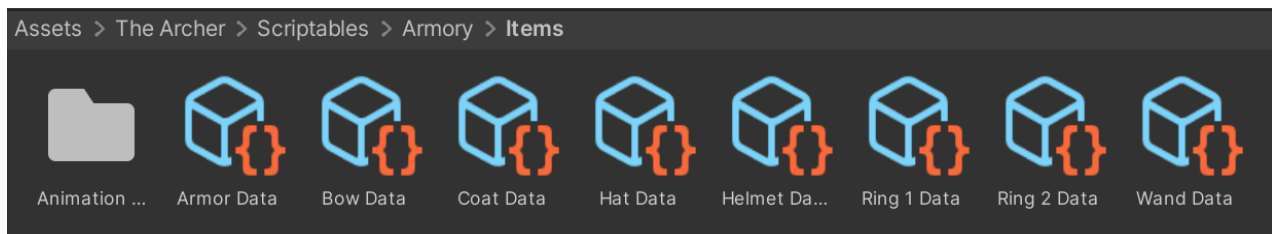
Importing assets

For a simple item you would only need a sprite. Import it inside **Assets/The Archer/Sprites/UI/Items** folder. Set it's **Texture Type** to **Sprite (2D and UI)** in the **Inspector**.



Creating Weapon Data

1. Open **Assets/The Archer/Scriptables/Armory/Items**.



2. Clone an existing weapon data asset, or create a new one via:
Right-click → Create → October → Armory → Item Data
3. Rename the new asset.
4. Fill out the data fields according to the [Item Data](#) description from the [Armory Overview Page](#). Set Icon to a sprite you've imported at the beginning.
5. Open **Assets/The Archer/Scriptables/Armory**, select the **Armory Database** asset, and add your new weapon data to the **Items** list.

That's it!

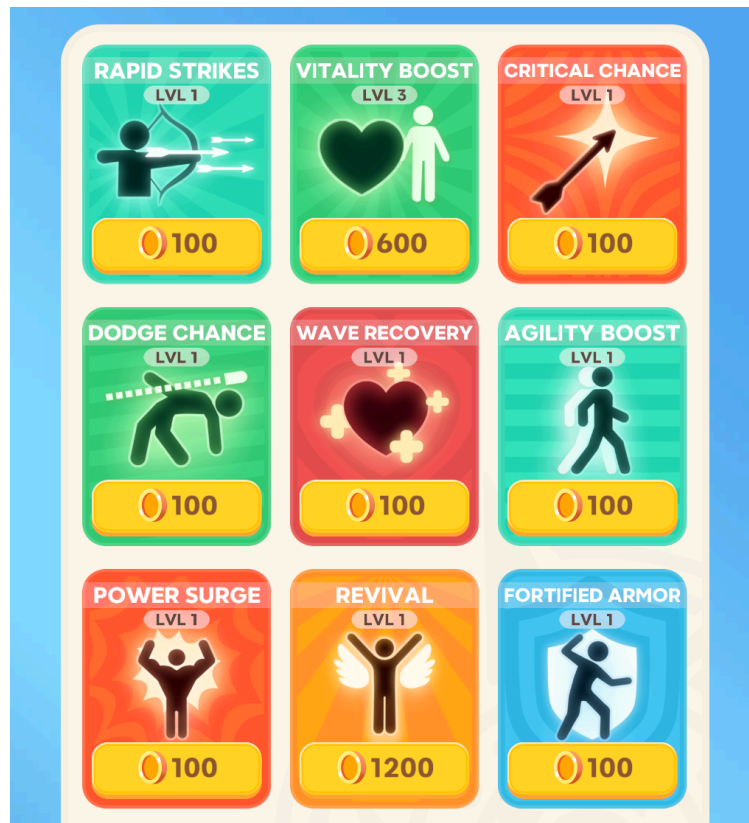
When you launch the game, your new item should now appear in the **Armory Page** of the **Main Menu**.

Upgrades

Upgrades Overview

Upgrades are one of the core pillars of rogue-like genre. They allow for permanent progression between standard runs, and allow players to venture further each time.

Naturally, this template features **Upgrade System** as well. In the game they are represented as a part of the Main Menu:

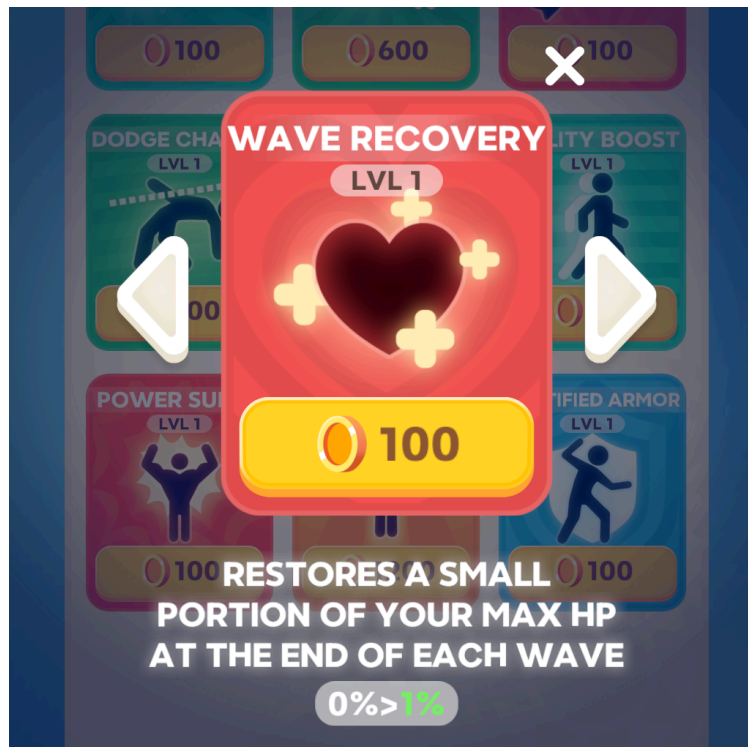


While being expandable, it includes several useful upgrades out of the box:

- **Rapid Strikes** - enhances the player's attack speed.
- **Vitality Boost** - increases the player's max HP.
- **Critical Chance** - multiplies the critical hit probability.
- **Dodge Chance** - makes the player dodge enemy attacks more frequently.
- **Wave Recovery** - restores some amount of health after each wave.
- **Agility Boost** - increases the player's movement speed.
- **Power Surge** - boosts the player's attack damage.
- **Revival** - lets you revive after a defeat.

- **Fortified Armor** - increases the player's damage reduction.

Opening an Upgrade will show a detailed description and adjacent stats:



Upgrades System Structure

Upgrades Manager

The **UpgradesManager** is located in the **Main Menu** scene hierarchy under the **Game Controller** GameObject.

It is responsible for:

- Loading upgrade behaviors during stage initialization
- Unloading them when returning to the main menu
- Providing access to all available **Upgrade Data**

You can access it in code via the property:

```
GameController.UpgradeManager
```

All methods of **UpgradesManager** are declared as `virtual`.

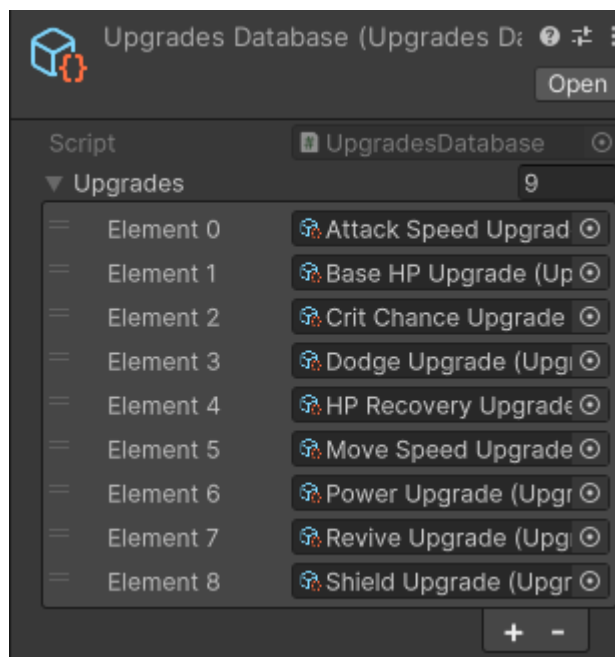
If you wish to modify or extend its functionality, it is recommended to create a **new class that inherits** from it, rather than editing the original directly.

Upgrades Database

The **Upgrades Database** is a simple asset that contains a list of **Upgrade Data** assets.

Only the upgrades listed in this database will be available in the game.

You can find the asset at: `Assets/The Archer/Scriptables/Upgrades`



Upgrade Data

Each upgrade is defined by an **Upgrade Data** asset that stores all information and values related to that specific upgrade.

Unlike abilities, **all upgrades share the same Upgrade Data class.**

`UpgradeData.cs` contains the following key fields:

Property	Description
UpgradeType type	An enum value that uniquely identifies the upgrade. There must not be two upgrades with the same type.
Sprite icon	The upgrade's icon, shown on the UI.
string title	The display name of the upgrade. Shown on the UI.
string description	The display description of the uprade. Shown on the UI.
string statsText	The text that will be used inside string.Format function to show how increasing upgrade level will change it's value. For example, here is the statsText value of the Attack Speed Upgrade: "{0}><color=#75F05F>{1}"
float initialValue	This value is shown in statsText when the upgrade isn't purchased. Usually 0 for values and 1 for multipliers.
GameObject prefab	The prefab that represents this upgrade. Its root object must include an Upgrade Behavior component that defines the upgrade's functionality.
int devStartLevel	If bigger that -1 that at the start of the game the upgrade will be purchased and it's level id will be set to this value. It is used in demo build of the game to unlock Revive Upgrade by default.

You can subscribe to the event `onUpgradeLevelChanged` to execute custom logic whenever the upgrade's level is increased.

Upgrade Level

All upgrades use a shared **Upgrade Level** class, which defines two key fields:

- **Cost** - The amount of gold required to unlock or upgrade this level.

- **Value** - The upgrade's numerical value. Each **Upgrade Behavior** interprets this value differently depending on the type of upgrade.

Upgrade Behavior

Each upgrade must have its own prefab with a component that **inherits from the `UpgradeBehavior` class**.

This base class provides several commonly overridden methods used to control initialization, level updates, and cleanup.

Commonly Overridden Methods

```
public virtual void Init(UpgradeData data)
```

Called once at the start of the stage when the upgrade is activated.

```
protected virtual void SetUpgradeLevel(int levelId)
```

Called when the upgrade is first activated and whenever its level changes.

 Note: Under the default template behavior, upgrade levels **do not change during gameplay**.

```
public virtual void Clear()
```

Called when the upgrade is removed.

By default, this method destroys the upgrade's **GameObject**.

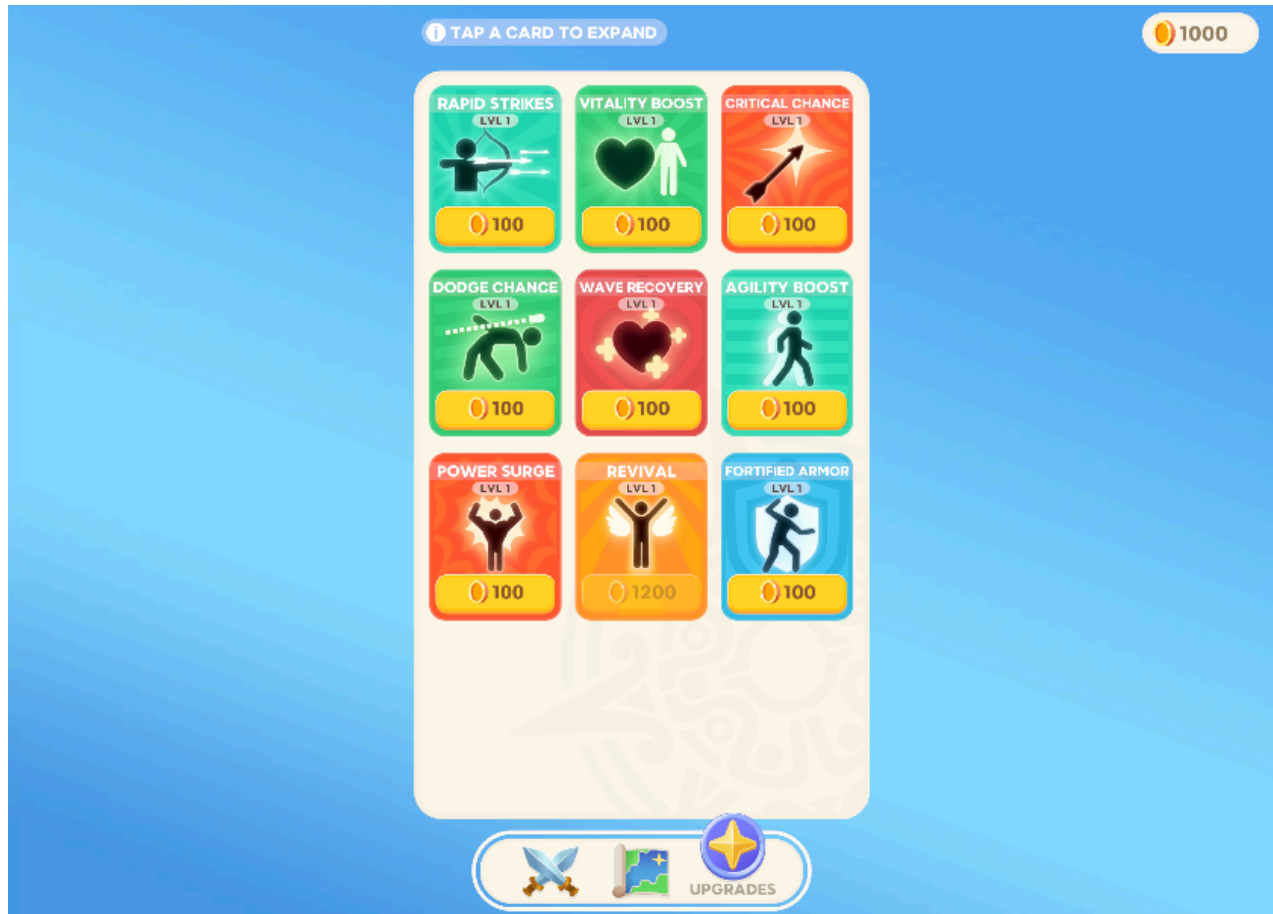
If you override it, make sure to call `base.Clear()` at the end to ensure proper cleanup.

Upgrades Page

The **Upgrades Page** is a UI page located inside the **Main Menu** scene.

You can access it by pressing the **right icon** on the **Dock**.

It allows the player to **purchase** and **upgrade** various upgrades.



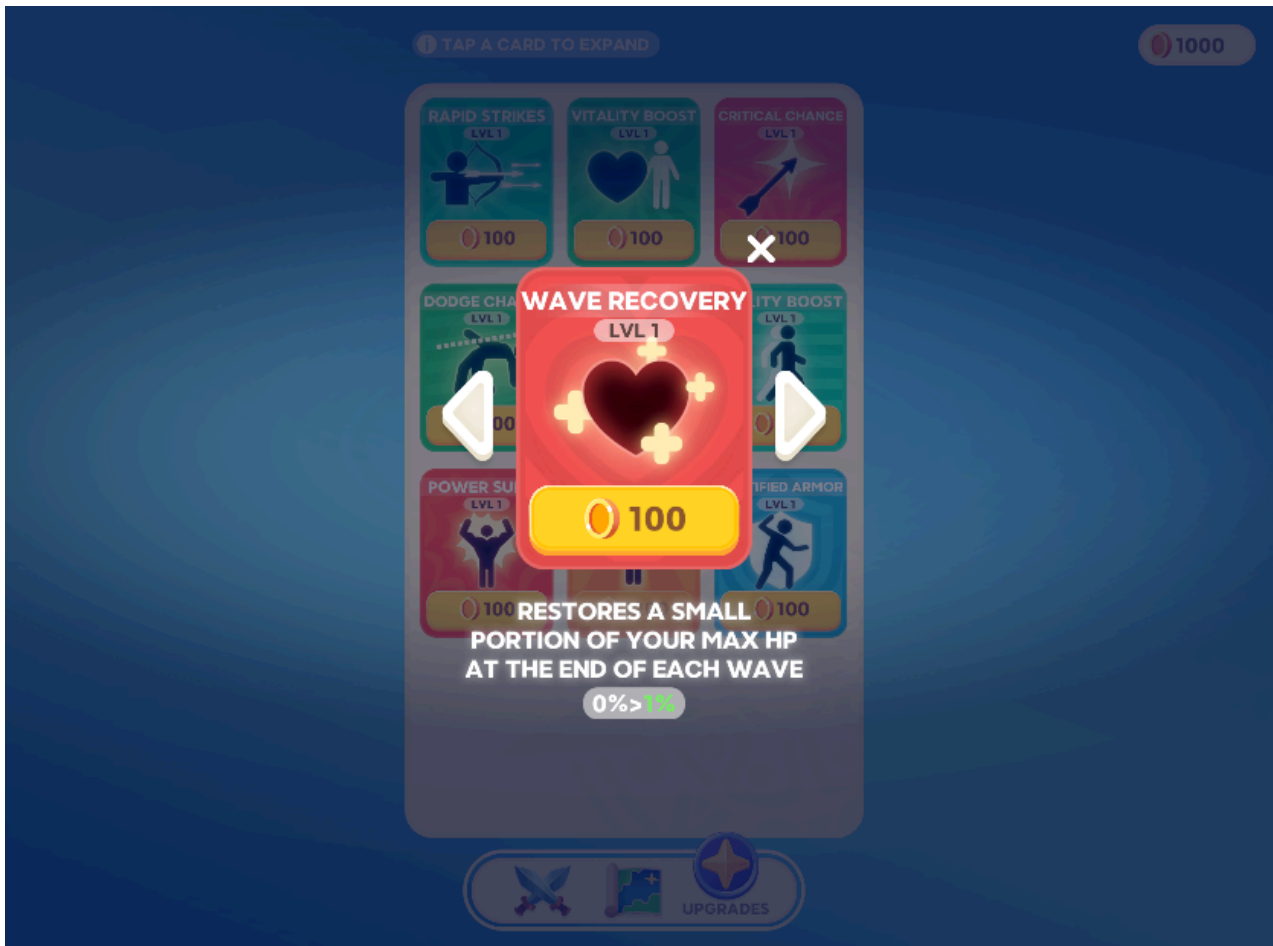
Upgrades Page

Overview

The main area of the Upgrades Page is a **scrollable grid** that displays **Upgrade Cards** - UI elements showing the upgrade's **name**, **level**, **icon**, and a **button** to increase its level for a price.

When the player clicks on a card, an **Expandable Popup** opens.

This popup displays the **detailed description** of the upgrade and shows how the upgrade's values will **change at the next level**.

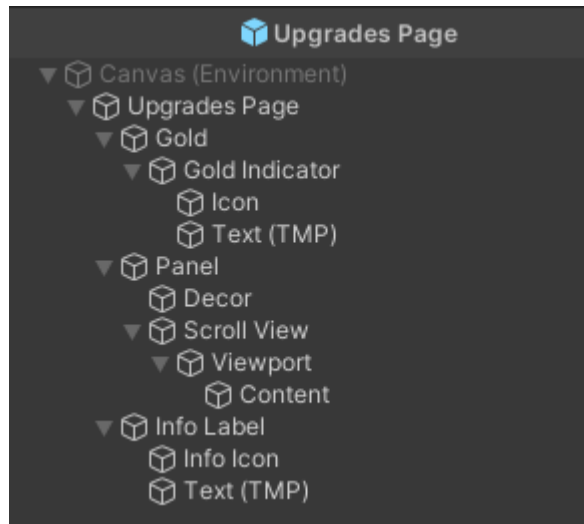


Expandable upgrade popup

Prefab Structure

The **Upgrades Page** prefab has a simple and well-organized structure:

- **Gold Indicator** - Displays the player's current amount of gold.
- **Main Panel** - Contains decorative elements and a **Scroll View** with a **Grid View** component attached to its content.
- **Info Label** - A small text label that instructs the player how to open the expanded popup.

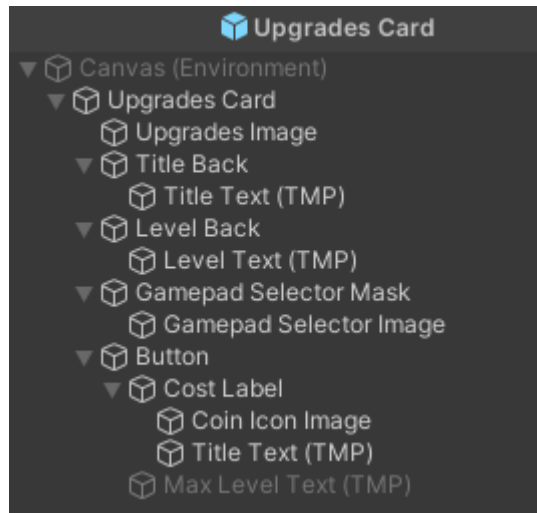


Upgrades Page prefab

Upgrade Card Prefab Structure

Each **Upgrade Card** prefab represents a single upgrade and contains the following elements:

- **Upgrade Image** - A unique sprite that visually represents the upgrade.
- **Upgrade Title Text** - Displays the name of the upgrade.
- **Upgrade Level Text** - Shows the current level of the upgrade.
- **Gamepad Selector** - A highlight indicator that appears when the player navigates to the card using a gamepad.
- **Purchase Button** - A button that allows the player to increase the upgrade's level.
 - Includes a **Price Label** showing the cost of the next level.



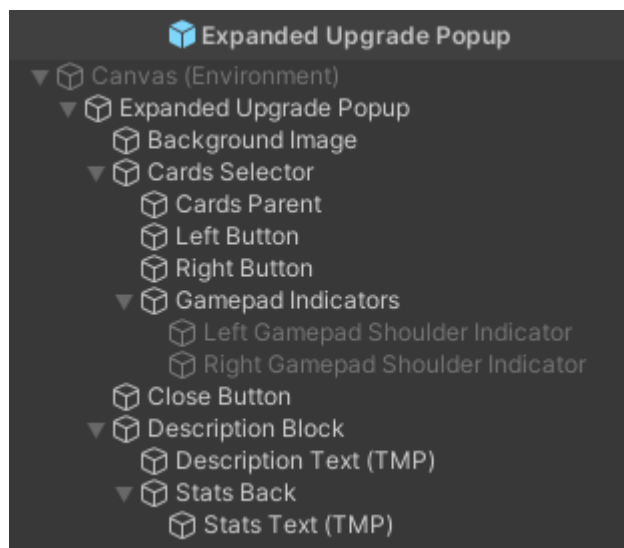
Upgrades Card Prefab

Expandable Upgrade Popup Prefab Structure

When the player selects an upgrade card, an **Expandable Popup** appears, providing more detailed information.

The popup consists of the following elements:

- **Background Image** - A dark, semi-transparent background that blocks raycasts to the UI behind it and increases popup visibility.
- **Cards Selector** - A custom component that allows the player to navigate between upgrades within the popup without closing it.
- **Description Block** - A container that displays the **upgrade description** and **stat change information** for the next level.



Expandable Upgrade Popup prefab

How to add a new Upgrade

This guide explains how to add a new upgrade to the game.

As an example, we'll create an upgrade that **increases healing** of the Player, called **Healing Increase**.

1. Importing assets

1. Import the upgrade icon into

`Assets/The Archer/Sprites/UI/Upgrades`

 Note that for upgrades we use 564×724 resolution

2. Set its **Texture Type** to **Sprite (2D and UI)**.
3. If your upgrade behavior requires additional assets such as **meshes**, **textures**, or **particles**, import them into the appropriate subfolders under `Assets/Common`.

2. Adding new Upgrade Type

1. Open the script:

`Assets/The Archer/Scripts/Upgrades/UpgradeType.cs`

2. Add a new entry for your upgrade.

In an unmodified template, the last index is `8`, so use `9`:

```
Shield = 4,  
Revive = 5,  
AttackSpeed = 6,  
MorePower = 7,  
MoveSpeed = 8,  
HealingIncrease = 9
```

3. Creating Upgrade Behavior script

1. Open `Assets/The Archer/Scripts/Upgrades/Behaviors` folder, create a new script named **HealingIncreaseUpgradeBehavior.cs**
2. Add the following code:

```

public class HealingIncreaseUpgradeBehavior : UpgradeBehavior
{
    // Multiplier that modify the player's healing stat
    protected StatMultiplier healingMultiplier = 1;

    public override void Init(UpgradeData data)
    {
        base.Init(data);
        // Register multiplier to the stats

        StageController.Player.Stats.HealingStat.AddMultiplier(healingMultiplier);
    }

    public override void SetUpgradeLevel(int level)
    {
        base.SetUpgradeLevel(level);
        // Set multiplier value
        healingMultiplier.Value = Level.Value;
    }

    public override void Clear()
    {
        // Remove multiplier when upgrade gets destroyed

        StageController.Player.Stats.HealingStat.AddMultiplier(healingMultiplier);
        base.Clear();
    }
}

```

4. Creating Upgrade Prefab

1. Go to:
Assets/The Archer/Prefabs/Upgrades
2. Right-click → **Create** → **Prefab**
Name it **Healing Increase Upgrade** and open it.
3. Select the root object and add the HealingIncreaseUpgradeBehavior component.

5. Creating Upgrade Data

1. Go to:

Assets/The Archer/Scriptables/Upgrades

2. Right-click → **Create** → **October** → **Upgrades** → **Upgrade**

3. Select the newly created asset and fill in its fields according to the [Upgrade Data Structure](#) documentation.

- Assign the prefab you created in the previous step to the **Prefab** field.
- Add at least one **Upgrade Level**. Set a value slightly higher than **1**.

4. Finally, open the **Upgrades Database** asset located in the same folder and add the **Healing Increase** data asset to the **Upgrades List**.

6. Testing

Start the game and open [Upgrades Page](#).

You should now see the Increase Healing Upgrade at the bottom of the grid.

Drop

Drop Overview

This template features Drop System - a way for the player to get armory items and XP gems from fallen enemies, get heal and ability related drops.

There are several types of drops included:

Drops from enemies

- **XP Gem** - a visual indicator of experience recieved.
- **Item** - a box containing certain armory item.
- **Heal** - a basic healing item.

Drops independently

- **Rage Star** - related to the ability with the same name.
- **Invulnerability Star** - same as the above.

How to add new Drop

This guide explains how to add a new drop to the game.

1. Importing Assets

Your drop should have a prefab with visuals - such as a 3D model, sprite, or particle effect.

Import these assets into the appropriate subfolders inside **Assets/The Archer/** (for example: **Models**, **Sprites**, or **Particles**).

2. Adding a new Drop Type

1. Open **DropType.cs** located at **Assets/The Archer/Scripts/Drop/Data/**
2. Add a new entry to the **DropType** enum with a descriptive name for your new drop.

```
public enum DropType
{
    Heal,
    RageStar,
    InvincibilityStar,
    XPGem,
    Item,
    YourNewDrop // ← Add your new drop here
}
```

3. Creating Drop Behavior Script

1. Open the folder **Assets/The Archer/Scripts/Drop/Behaviors/**
2. Create a new script and name it **[Name]DropBehavior.cs**.
3. Open the script and inherit from **DropBehavior**.
Override the **OnPickedUp()** method to define what happens when the drop is collected.

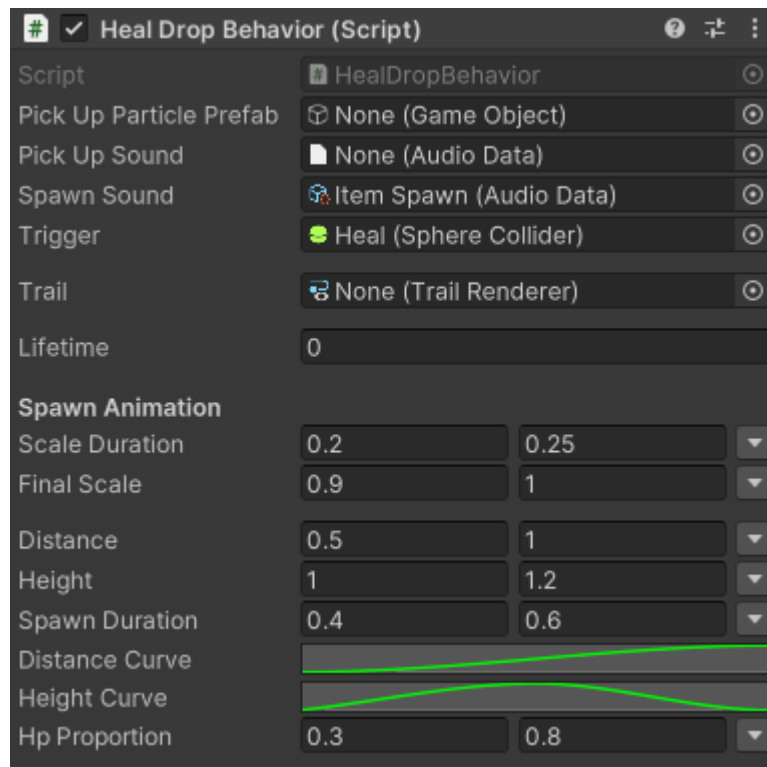

```
public class YourNewDropBehavior : DropBehavior
{
    public override void OnPickedUp()
    {
        base.OnPickedUp();

        // Custom code for when the drop is picked up
    }
}
```

 **Tip:** Check other drop behavior scripts in this folder for reference and implementation examples.

4. Creating Drop Prefab.

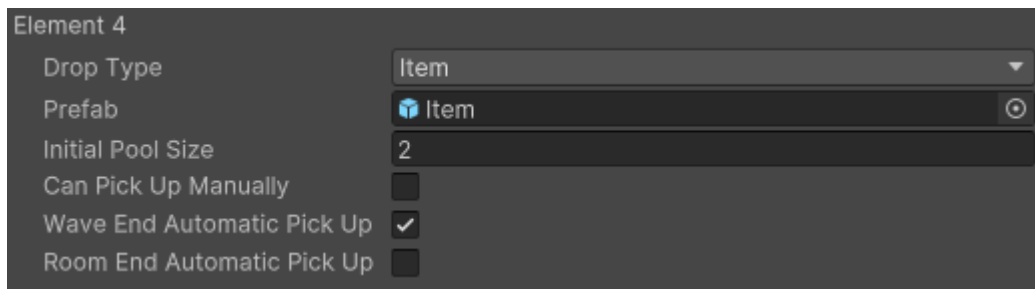
1. Open **Assets/The Archer/Prefabs/Drop/**
2. **Right-click → Create → Prefab**, then rename and open it.
3. Add your visual elements (models, particles, etc.) as children of the prefab root.
4. On the root GameObject:
 - Add your **Drop Behavior** component created in the previous step.
 - Add a **Sphere Collider**, enable **Is Trigger**, and assign it to the **Trigger** field.
5. Fill out the **Drop Behavior** component fields:



- Pick Up Particle - Optional particle prefab to play when picked up. (*Reference to another prefab, not a particle inside this one.*)
- Pick Up Sound - Optional `AudioData` played when picked up.
- Spawn Sound - Optional `AudioData` played when spawned.
- Trigger - The trigger collider you created.
- Trail - Optional `TrailRenderer` reference. Used to reset trail visuals when re-spawned.
- Lifetime - Time (in seconds) before the drop despawns. Set to `0` to never despawn.
- Spawn Animation parameters - Settings that control the drop's spawn animation. Copy values from an existing drop prefab and tweak as desired.

5. Registering Drop Data

1. Open `Assets/The Archer/Scriptables/Drop Database.asset`
2. Add a new item to the **Drop List** and fill in its fields:



- Drop Type - The enum value you created in Step 2.
- Prefab - The prefab created in Step 4.
- Initial Pool Size - Number of drops pre-instantiated at stage start.
- Can Pick Up Manually - If true, the player can collect this drop by walking near it.
- Wave End Automatic Pick Up - If true, the drop flies to the player at the end of the wave.
- Room End Automatic Pick Up - If true, the drop flies to the player at the end of the room.

6. Testing

1. Open the **Stage Creator** by pressing **Shift + S** or via **Tools → October → Stage Creator**.
2. Open the first stage and select the first wave of the first room.
3. Select any enemy in the scene. In its Inspector, locate the **Enemy Overrides Behavior** component.
4. Add your new drop to the **Additional Drop List**.
5. Press **Test Wave**.
6. When you defeat that enemy, your new drop should appear and function as expected.

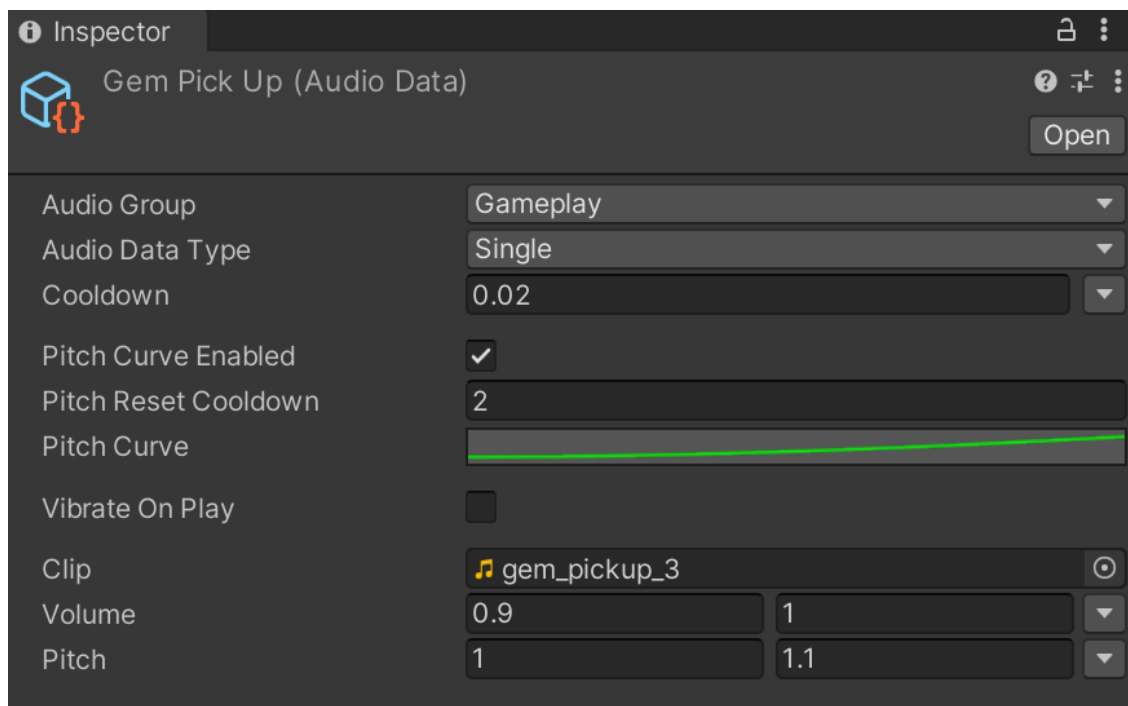
Audio And Vibration

Audio System Overview

This template features a customizable audio solution that's handy for many scenarios.

Settings

Every audio is presented as a Scriptable file with settings inside:



- To make mixing easier, all audio files are categorized into 3 groups: Gameplay, Music, and UI
- Audio Data can be of two types: **Single** and **Multi**.

Single audio is self-descriptive, while **Multi** is used when there's a need for several different sounds for one action.

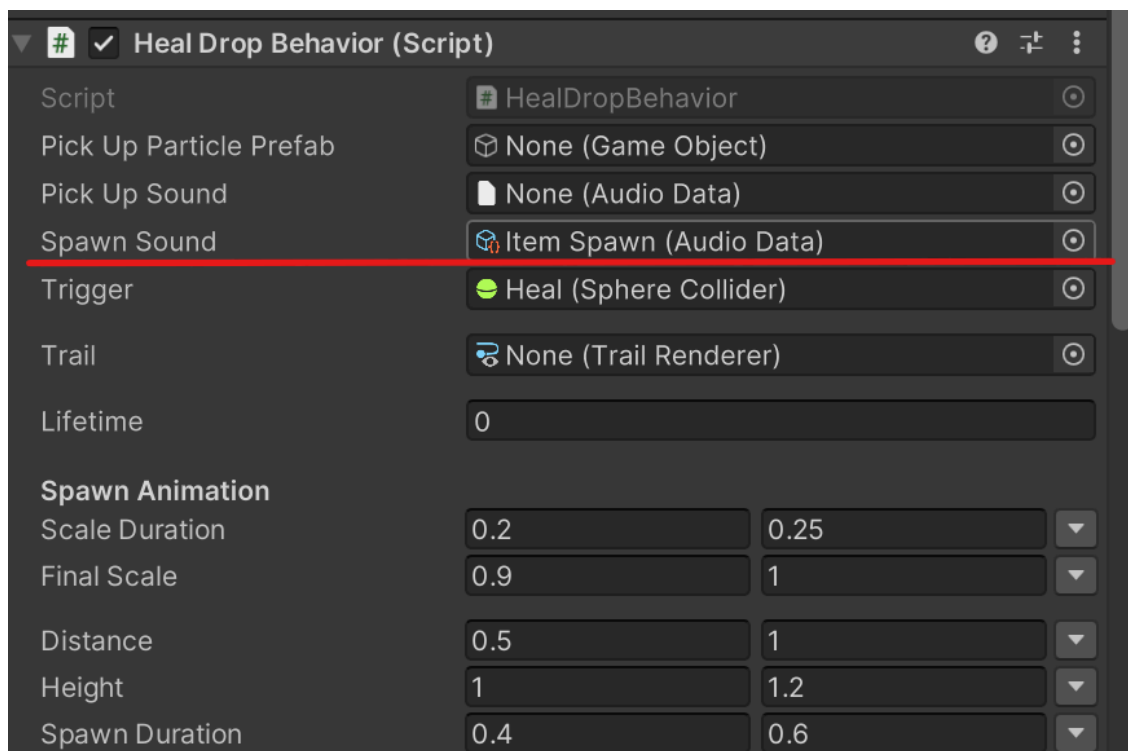
- ✓ For example, there are some repetitive sounds like shooting an arrow or walking steps. These can be made a lot easier to listen to with several sound clips attached.

- **Cooldown** is used for highly repeated sounds, for example, the gem pick-up sound, when it's played multiple times in a row in a fast sequence. Setting a short cooldown will smooth this out.
- **Pitch Curve** is an option to progressively change the pitch of a repeated sound. You can set up your custom curve and cooldown options there as well.
- **Vibrate on Play** will give a light, short vibration when this sound is being played.

Below the settings, you can assign your audio file, and for each one, set volume and Pitch settings. These settings can be set as a Constant value or as a Random Between Two.

Usage

These audio data files are used everywhere throughout the template, for example, in the Heal drop prefab:



Audio Data can be assigned in any slot that requires this type of file.

In code, to play Audio Data, use:

```
GameController.AudioManager.PlayAudio(audioData);
```

PlayAudio() method returns an AudioSource Component or null if AudioManager rejects playing the sound because of cooldown.

Template uses Audio Mixer located in Assets/Common/Scriptables/Audio. It has two snapshots:

- Normal - All sounds are at the full strength
- Paused - UI sounds are at the full strength, Gameplay sounds are muted and the music has lower volume.

There are two audio datas that are directly assigned inside AudioManager:

- Button Click.
- Main Menu music.

You can play them from Audio Manager directly:

```
GameController.AudioManager.PlayMainMusic();  
GameController.AudioManager.PlayButtonClick();
```

You can modify them inside Audio Manager Component, that located in Main Menu scene under Game Controller GameObject.

Vibration System Overview

The template includes a lightweight **Vibration System** that handles haptic feedback across supported platforms.

Supported Platforms

- **PC (Gamepad)**
- **Android**
- **iOS**
- **WebGL** (*gamepad vibration not supported on WebGL*)

Triggering Vibration

To activate a vibration effect, use:

```
GameController.VibrationManager.Vibrate(duration, intensity);
```

Both parameters are `float` values:

- **duration** - how long the vibration lasts (in seconds)
- **intensity** - strength of the vibration, from 0 to 1

Quick Access Methods

For convenience, the `VibrationManager` provides predefined vibration patterns:

```
public virtual void LightVibration() => Vibrate(0.08f, 0.4f);  
public virtual void MediumVibration() => Vibrate(0.1f, 0.6f);  
public virtual void StrongVibration() => Vibrate(0.15f, 1f);
```

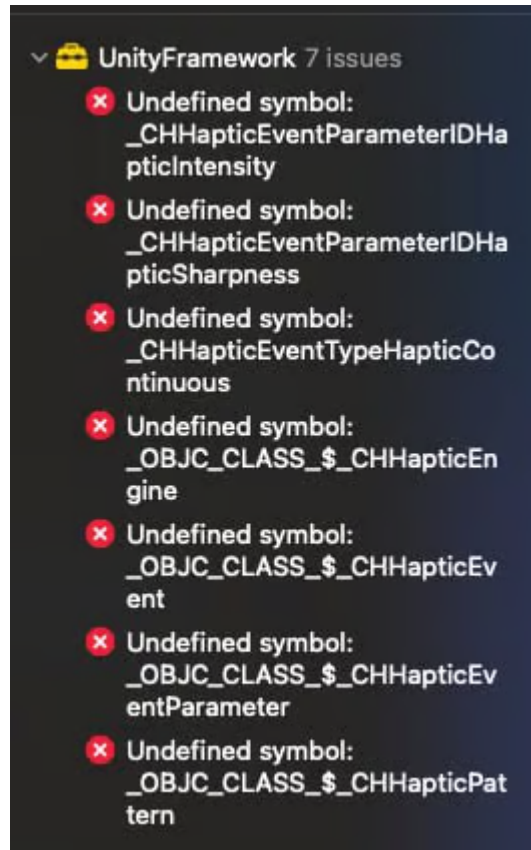
MacOS Note

On some **MacOS** devices, a haptic-related error may occur when building the game.

Refer to the [Haptic iOS Build Error](#) page for instructions on how to fix it.

Haptic IOS Build Error

If you happen to have Haptic errors when building the project in the XCode, follow this steps to fix them.

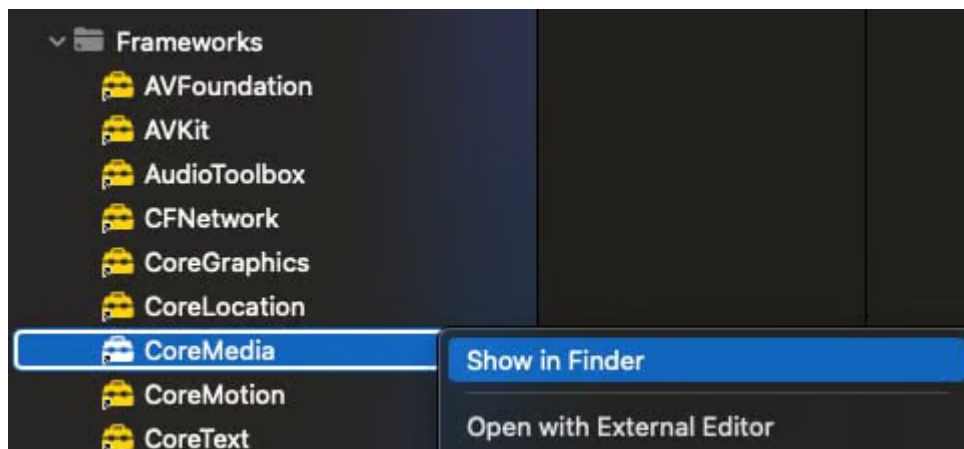


Haptic errors in XCode

1

Navigating to the frameworks folder

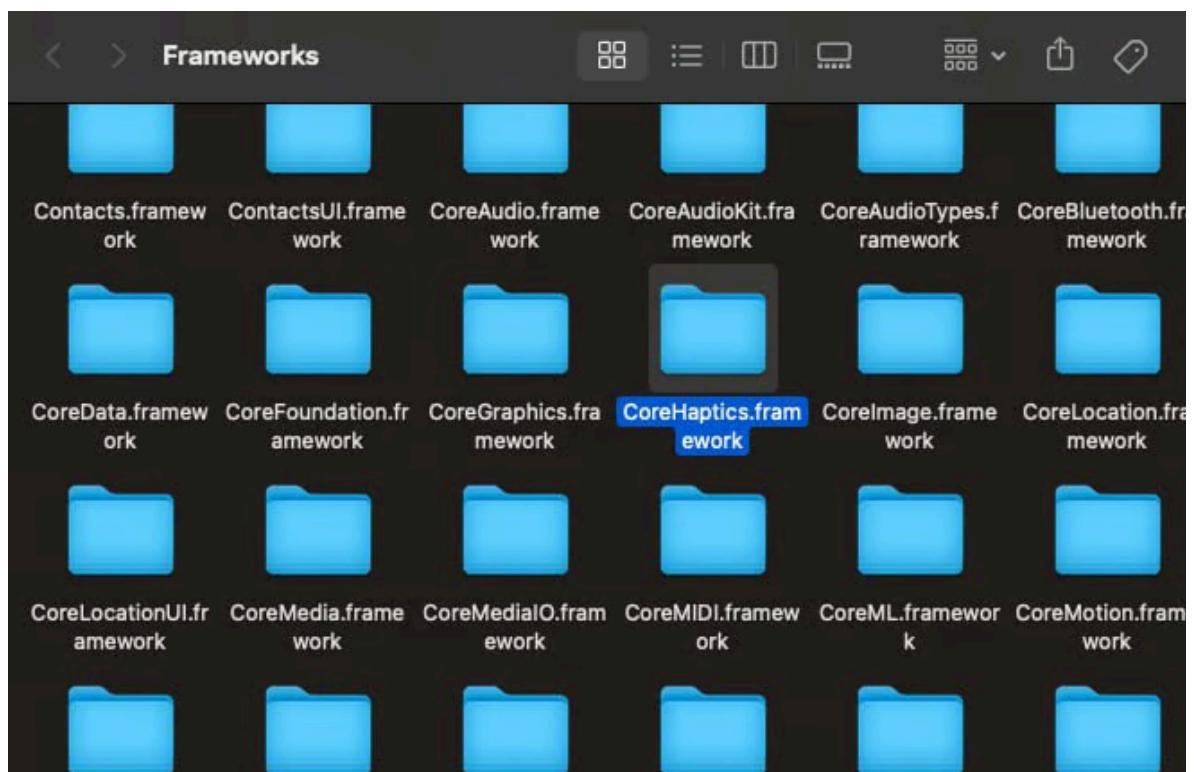
Select any framework in the frameworks topdown (in our case it was Core Media framework), right click on it and select Show In Finder



2

Locating CoreHaptic framework

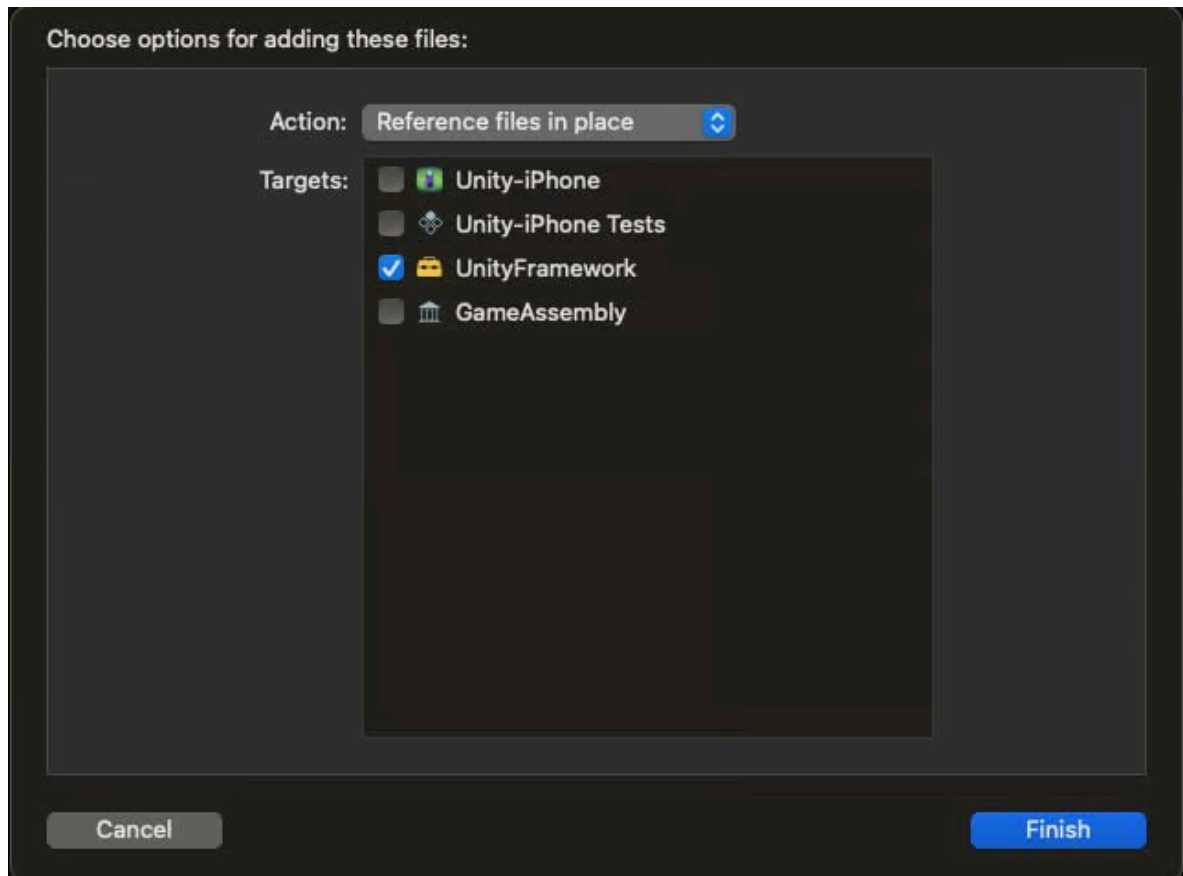
Inside the finder window that just opened there should be CoreHaptics.framework folder. Select it and drag and drop it to the frameworks topdown from the previous step



3

Finishing touches

The popup will show inside the XCode. Make sure that only the UnityFramework Target is selected, and press Finish.



Projectiles

Projectiles System Structure

Projectiles Manager

The **Projectiles Manager** is a simple manager class responsible for storing and managing pooled projectiles.

It keeps an internal dictionary where each key is a projectile prefab and each value is a pool of instances for that prefab.

The Projectiles Manager is located in the **Game scene** under the **Stage Controller** GameObject.

You can access it in code via:

```
StageController.ProjectilesManager
```

Whenever a hero or an enemy uses an attack that fires a projectile, that projectile **must be registered** during the object's `Start` method:

```
[SerializeField] protected GameObject projectilePrefab;

protected override void Start()
{
    base.Start();

    StageController.ProjectilesManager.RegisterProjectile(projectilePrefab)
;
}
```

To launch a projectile, simply request an instance from the manager using the *same prefab*:

```
public void Attack()
{
    var projectile = StageController.ProjectilesManager
        .GetProjectile<SimpleProjectile>(projectilePrefab);

    projectile.transform.position = transform.position + Vector3.up +
        Vector3.forward;
    projectile.transform.forward = transform.forward;

    projectile.Launch(damage);
}
```

IProjectileTarger

Any object that can be hit by a projectile must implement the **IProjectileTarget** interface.

It provides useful methods such as:

- `TakeDamage()`
- `ApplyStatusEffect()`
- `ShowHitEffect()`
- and others.

`IProjectileTarget` itself inherits from **IDefeatable**, which provides:

- `SubscribeOnDefeat()`
- `UnsubscribeOnDefeat()`

Projectiles do not distinguish between heroes and enemies in code—this is instead handled by **layers**:

- **Player Projectile**
 - Collides with **Enemy**
 - Ignores **Player**
- **Enemy Projectile**
 - Collides with **Player**

- Ignores **Enemy**

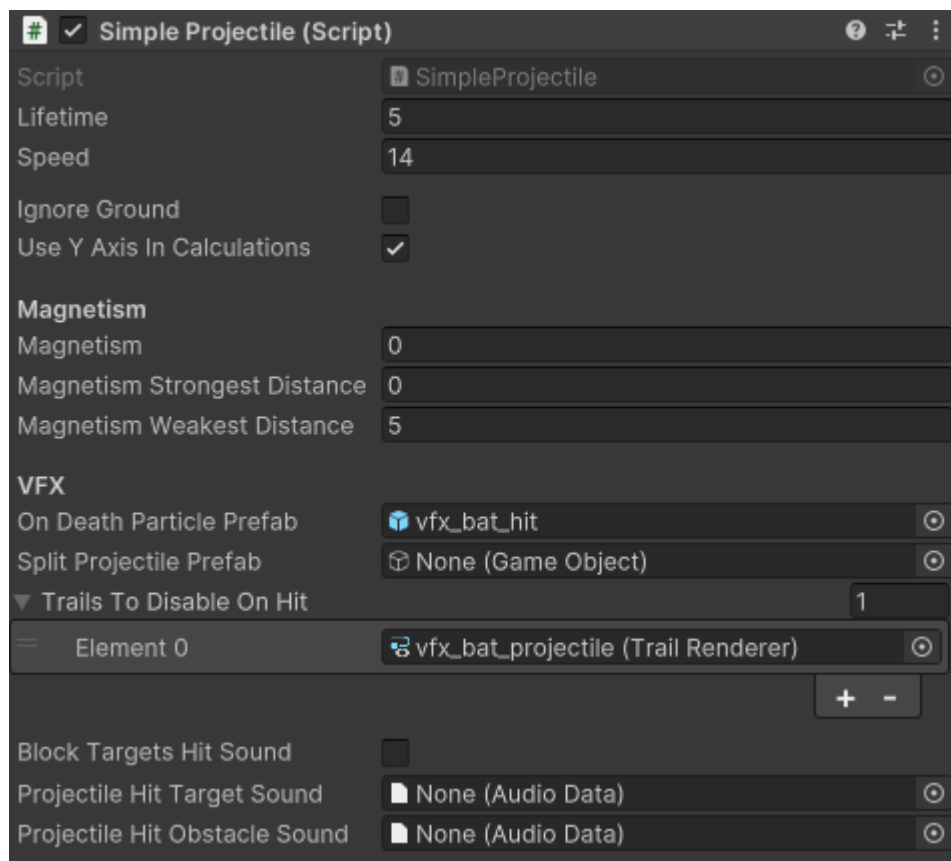
All projectiles also collide with **Obstacle** and **Ground** layers.

Simple Projectile Behavior

`SimpleProjectileBehavior` is a versatile, feature-rich projectile class that covers most common projectile mechanics.

It supports bouncing, ricochets, splitting, status effects, magnetic homing, and distance-based damage scaling.

Component Fields:



- **Lifetime** - How long the projectile exists before despawning if it hits nothing.
- **Speed** - Projectile travel speed.
- **Ignore Ground** - If enabled, the projectile will not despawn when hitting the ground.
- **Use Y Axis in Calculations** - If disabled, projectile movement is restricted to the XZ plane.

- **Magnetism** - Strength of the homing force toward the target.
- **Magnetism Strongest Distance** - Distance at which full magnetism is applied.
- **Magnetism Weakest Distance** - Distance beyond which magnetism is zero; values between the two distances are lerped.
- **On Death Particle Prefab** - Particle prefab to spawn on hit (must be a reference to a particle prefab, not a ParticleSystem component).
- **Split Projectile Prefab** - Projectile prefab to spawn if this projectile splits on hit.
- **Trails To Disable On Hit** - Trails to reset when the projectile despawns or hits a target.
- **Block Target Hit Sound** - If true, the target will not play its default hit sound.
- **Projectile Hit Target Sound** - Sound played when hitting a target (optional).
- **Projectile Hit Obstacle Sound** - Sound played when hitting an obstacle or ground (optional).

Tips for creating a new projectile

- For most cases, you can **reuse or extend** `SimpleProjectileBehavior` through inheritance.
- If your projectile requires behavior that cannot be achieved by adjusting its parameters, create a new class that inherits from **AbstractProjectile**.
 - This base class contains minimal required functionality: status effects storage, bounce/ricochet logic, and integration with the Projectiles Manager.

Status Effects

Status Effects Overview

A **status effect** is a temporary modifier that can be applied to a player or enemy, altering their behavior or stats for a short duration. The template includes **four types** of status effects:

- **Ignite** - Deals damage over time in short bursts to its target.
- **Freeze** - Slows down or completely immobilizes the target for a duration.
- **Poison** - Inflicts a small amount of damage over time and reduces the target's strength.
- **Shock** - Deals minor damage over time and can spread to nearby targets.

Status effects are typically applied by **abilities** through their **projectiles**, although some enemies can also inflict them directly through their attacks.

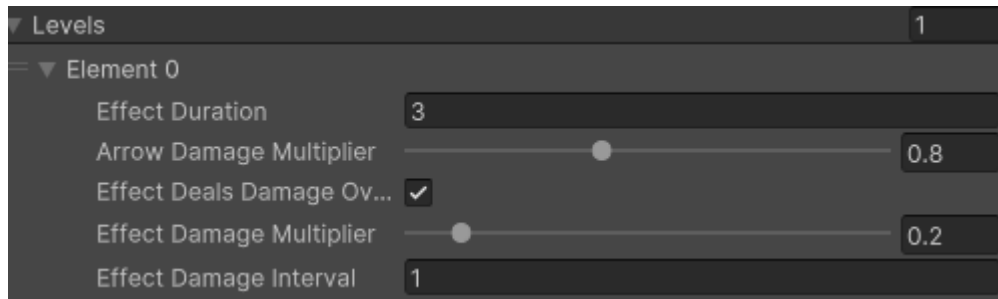
How Status Effects Work

Let's take a closer look at how **status effects** work using the **Ignite Ability** as an example.

1

Data For Status Effect

There isn't a dedicated asset that stores universal status effect data. Instead, each **ability** or other entity has the freedom to define and initialize its own effect values as needed.



Ignite Ability Level Data

In the case of the **Ignite Ability**, the status effect parameters are stored within its **Level** data.

These include the following fields:

- **Effect Duration** - The amount of time the effect remains active on its target.
- **Arrow Damage Multiplier** - Modifies the base damage of the projectile itself, giving creators finer control over game balance.
- **Effect Deals Damage Over Time** - If enabled, the effect will periodically deal damage while it's attached to the target.
- **Effect Damage Multiplier** - The amount of damage dealt each tick, calculated as the projectile's base damage multiplied by this value.
- **Effect Damage Interval** - The time interval between each burst of damage.

2

Status Effect Component

When the **Ignite Ability** is acquired by the player, its `SetData` method creates an instance of the **IgniteStatusEffect** and attaches it as a component to its own `GameObject`.

It then instructs the `PlayerBehavior` to apply this status effect to every projectile the player fires.

```
protected override void SetData(IgniteAbilityData data)
{
    base.SetData(data);

    effect = gameObject.AddComponent<IgniteStatusEffect>();
    StageController.Player.SetProjectileEffect(effect);
}
```

Next, in the `SetAbilityLevel` method, the ability assigns values from its **Ability Level** to the attached **Status Effect** instance. This process also occurs each time the ability is upgraded.

```
protected override void SetAbilityLevel(int levelId)
{
    base.SetAbilityLevel(levelId);

    effect.Duration = AbilityLevel.EffectDuration;
    effect.EffectDamageMultiplier =
AbilityLevel.EffectDamageMultiplier;
    effect.ArrowDamageMultiplier =
AbilityLevel.ArrowDamageMultiplier;
    effect.DamageInterval = AbilityLevel.EffectDealsDamageOverTime
?
    AbilityLevel.EffectDamageInterval :
AbilityLevel.EffectDuration + 1;
}
```

 Each entity that applies a status effect must create its **own instance** of that effect. This can be done either **through code**, as shown in the example above, or by **adding the status effect component** (e.g., `IgniteStatusEffect`) directly to the `GameObject`.

In both cases, the effect must be **initialized and populated with its specific data** in code to ensure proper behavior.

3

Applying Status Effects to Targets

The **PlayerBehavior** component attaches status effects from abilities to the projectiles it fires.

When a projectile hits its target, it instructs the status effect instance created earlier to attach itself to that target:

```
appliedStatusEffects[i].ApplyToTarget(target, GetDamage());
```

Each **Status Effect instance** keeps track of all targets it is currently attached to, when to apply damage and when to remove itself after expiration.

Any class that can be affected by projectiles must implement the

`IProjectileTarget` interface.

This interface defines the methods used to manage status effects:

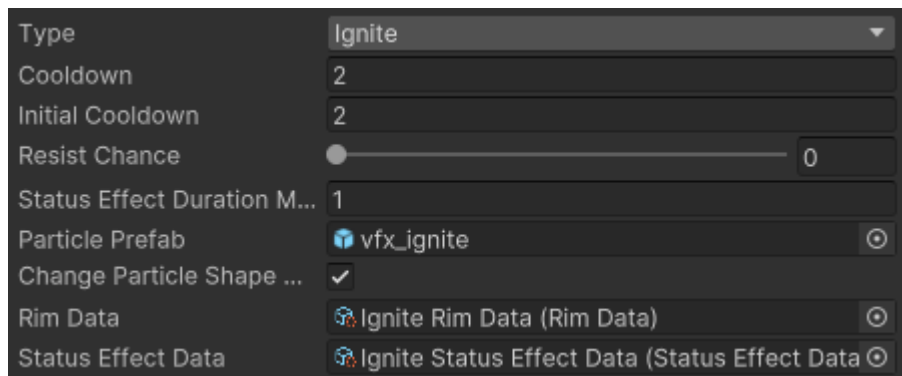
```
bool ApplyStatusEffect(StatusEffectType type, float multiplier);  
void RemoveStatusEffect(StatusEffectType type);
```

The target is responsible for handling its response to each effect.

For example, in the template, every target includes the **`StatusEffectHelper`** and **`RenderersHelper`** components, which control the **visual and audio feedback** for active effects, such as rim highlights, particles, and sounds, etc.

Status Effects Helper

The **StatusEffectsHelper** component defines how **heroes** and **enemies** respond to status effects applied to them. For each status effect, it contains its own configurable set of data fields:



Effect Data

- **Type** - The type of status effect.
- **Cooldown** - The duration during which the target is immune to this status effect after the previous one expires.
- **Initial Cooldown** - The duration during which the target is immune to this status effect after spawning.
- **Resist Chance** - The probability that the target will resist the status effect entirely.
- **Status Effect Duration Multiplier** - Adjusts how long the status effect lasts, allowing certain enemies to be more resistant.
- **Particle Prefab** - The particle effect that will be instantiated and played while the status effect is active.
- **Change Particle Shape** - If enabled, the particle's shape will be set to match the target's mesh renderer.
- **Rim Data** - A small asset containing color, strength, gradient, and direction values for a rim effect, specific to the **October Shader**.
- **Status Effect Audio Data** - A small asset containing the sounds that play when the effect is applied, ticks, or expires.

Other Systems

Save System

Save System Overview

The save system allows storing **objects** instead of simple values.

Each entry in the save database consists of a **unique string key** and a **value object** that implements the `ISave` interface.

Creating a New Save Class

For example, let's track how many enemies the player has defeated.

Create a new class that implements `ISave`:

```
public class DefeatedEnemiesCounterSave : ISave
{
    public int defeatedEnemiesCount;

    public void Flush()
    {
        // Called before saving, for cleanup or data processing
    }
}
```

Accessing Save Data

When the player starts a stage, get the save instance via `SaveManager`:

```
var save = GameController.SaveManager
    .GetSave<DefeatedEnemiesCounterSave>("Defeated Enemies Counter");
```

The **Save Manager** will:

- Search its database for an entry with the key `"Defeated Enemies Counter"`.
- If found - return the existing instance.
- If not found - create a new one, store it in the database, and return it.

Updating the Data

Each time an enemy is defeated, increase the counter:

```
save.defeatedEnemiesCount++;
```

You can then update the UI manually, for example:

```
StageController.GameScreen.UpdateDefeatedEnemies(save.defeatedEnemiesCount);
```

Alternatively, the UI can fetch the same save instance and subscribe to its updates.

Save Complexity

`ISave` instances can be as simple or complex as needed.

You can use properties and events to notify the UI or other systems when a value changes.

The `Flush()` method can also be used for cleanup if needed.

Example:

```
public class DefeatedEnemiesCounterSave : ISave
{
    [SerializeField] private int defeatedEnemiesCount;

    public event UnityAction<int> onDefeatedEnemiesCountChanged;

    public int DefeatedEnemiesCount
    {
        get => defeatedEnemiesCount;
        set
        {
            defeatedEnemiesCount = value;
            onDefeatedEnemiesCountChanged?.Invoke(value);
        }
    }

    public void Flush()
    {
    }
}
```

Once created, the `SaveManager` automatically stores and saves the instance - you **do not need** to manually notify it of value changes.

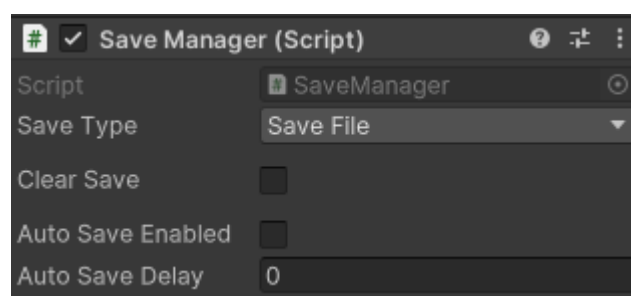
Save Manager

The **Save Manager** is located in the **Main Menu Scene** under the **Game Controller** GameObject.

It behaves as a singleton - one instance persists for the entire game session.

Save Manager Inspector

Save Manager component includes the following fields:



- Save Type - Enum that determines whether to use a dedicated save file or Unity `PlayerPrefs`.
- Clear Save - If `true`, the save data is reset on game start. Useful for testing.
- Auto Save Enabled - If `true`, enables automatic saving via coroutine.
- Auto Save Delay - Interval (in seconds) between automatic saves.

Easing System

The template includes an **Easing System** that allows you to animate **Transforms**, **RectTransforms**, **Images**, and other components using a variety of easing functions.

Animations are driven by coroutines and can be chained or yielded for sequential effects.

Easing Coroutine Methods

Each easing animation returns an `IEasingCoroutine`, which supports the following configuration methods:

```
// Sets one of the built-in easing functions.
// Default easing is Linear.
IEasingCoroutine SetEasing(EasingType easingType);

// Uses a custom AnimationCurve instead of an easing function.
// X-axis: time (0 -> 1)
// Y-axis: normalized progress (0 = start, 1 = end)
// Values outside 0-1 can create overshoot/bounce effects.
IEasingCoroutine SetEasingCurve(AnimationCurve easingCurve);

// Called when the easing animation completes.
IEasingCoroutine SetOnFinish(UnityAction action);

// If true, uses Time.unscaledDeltaTime instead of deltaTime.
// Default is false.
IEasingCoroutine SetUnscaledTime(bool unscaledTime);

// Adds a delay (in seconds) before the easing starts.
// Must be set before the coroutine begins.
IEasingCoroutine SetDelay(float delay);
```

Extension Methods

Transform Extensions

- `DoPosition`

- `DoRotation`
- `DoLocalEulerAngles`

RectTransform Extensions

- `DoAnchorPosition`
- `DoStretchedOffset`
- `DoSizeDelta`

Other Components

Additional utility extensions exist for animating UI and other component values.

Easing Manager Utility Methods

The **Easing Manager** also provides standalone coroutine helpers:

- **DoFloat** - lerps a float value from start to end
- **DoAfter** - executes an action after a delay or after a condition is met
- **DoNextFrame** - runs an action on the next frame
- **DoNextFixedFrame** - runs an action on the next physics frame
- **DoRepeatedly** - calls an action repeatedly at a given interval for a duration
- **DoTimeScale** - animates `Time.timeScale`
- **StartCustomCoroutine** - allows non-MonoBehaviour classes to start coroutines

All of these methods integrate seamlessly with Unity coroutines.

Example

This coroutine moves a transform forward for 1 second, then right for another second.

Both movements use the `SineInOut` easing function:

```
protected virtual IEnumerator MoveForwardRightCoroutine()
{
    yield return transform
        .DoPosition(transform.position + Vector3.forward, 1f)
        .SetEasing(EasingType.SineInOut);

    yield return transform
        .DoPosition(transform.position + Vector3.right, 1f)
        .SetEasing(EasingType.SineInOut);
}
```

Scene Requirements

To use the Easing System, an **Easing Manager** component must exist in the active scene.



Important:

The Easing Manager **must not** be placed under `DontDestroyOnLoad`.
The system relies on easing coroutines stopping automatically when scenes unload.
Making the manager persistent will cause various issues.

Stopping Easing Coroutines

To stop an easing animation early, use:

```
easingCoroutine.Stop();
```

Or safely stop it only if it exists:

```
easingCoroutine.StopIfExists(); // does nothing if null
```

`StopIfExists` is an extension method and avoids exceptions when the coroutine has not been initialized.

Navigation

Overview

The template's navigation system is built on Unity's **NavMesh**. Both **Enemy** and **Player** prefabs use the **NavMesh Agent** component, while obstacles rely on **NavMesh Obstacle**. Each **Room Holder** includes two **NavMesh Surfaces** — one dedicated to enemies and another to the player.

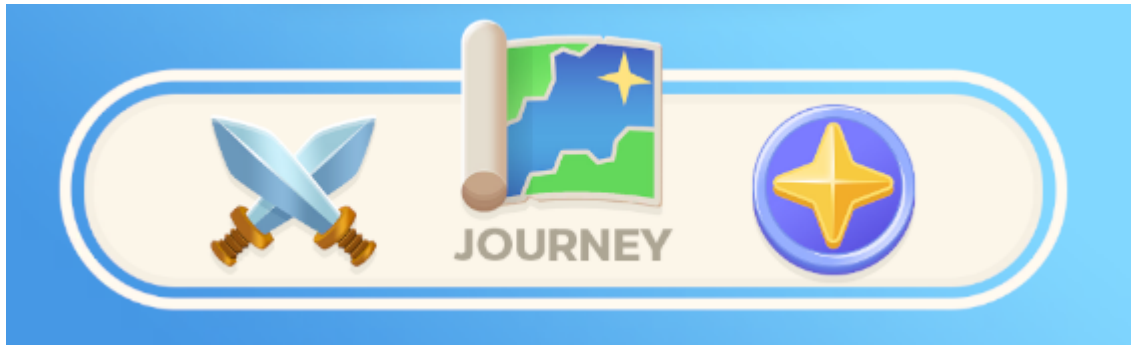
All code-level interactions with the NavMesh are performed through **abstract classes**, giving users the flexibility to replace the NavMesh system with a custom navigation solution if desired.

Navigation classes

There are two main navigation-related classes in the template:

- **NavigationHandler** - an abstract class that controls movement behavior for enemies and the player.
 - Its NavMesh-based implementation is **NavmeshNavigationHandler**.
- **NavigationManager** - an abstract class responsible for higher-level navigation operations and calculations.
 - Abilities often reference it to find available positions within the room.
 - Its NavMesh implementation is **NavmeshNavigationManager**, which is instantiated as a child of the **StageController** GameObject in the Game scene.

Dock



The **Dock** is the primary navigation element inside the **Main Menu**.

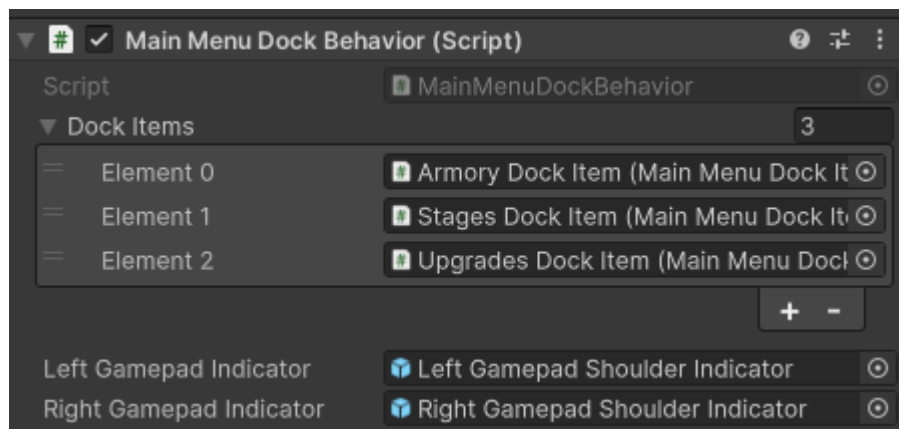
It contains three dock items, each corresponding to one of the main menu pages:

- **Armory Dock Item**
- **Stages Dock Item (Journey)**
- **Upgrades Dock Item**

i If you need to modify any existing class in the project, it's **strongly recommended** to **inherit** from the existing class instead of editing the original file directly. This approach makes updating the template to a new version much easier. Original classes are designed for extensibility and make extensive use of `protected` `virtual` methods while rarely using `private` ones.

Dock Overview

Dock Inspector



The **Dock** Inspector contains the following fields:

- **Dock Items** - A list of all dock items.
- **Left Gamepad Indicator** - Displayed only when the active input type is set to *Gamepad* and any item other than the leftmost one is selected.
- **Right Gamepad Indicator** - Displayed only when the active input type is set to *Gamepad* and any item other than the rightmost one is selected.

Dock Behavior Script

The `MainMenuDockBehavior` script manages the dock's functionality and interaction logic.

It defines two properties, one event, and eight methods.

Properties

- ```
// Indicates which dock item (and by extension, which page) is
currently selected.
public MainMenuPageType SelectedPage { get; protected set; }
```
- ```
// When set to false, the dock will not respond to gamepad input.
// This is used to prevent page switching when popups (like
Settings) are open.
public bool IsDockEnabled { get; set; } = true;
```

Event:

- ```
// Invoked when a dock item (that was not currently selected) is
clicked.
public event UnityAction<MainMenuPageType> onPageSelected;
```

Methods:

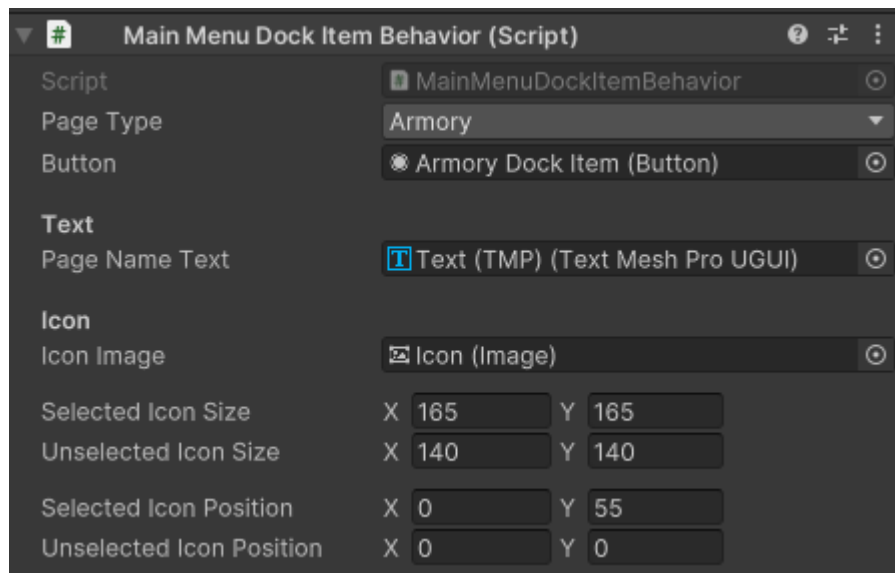
- ```
// Subscribes to input events.
// Not done in Awake to ensure Input Manager is initialized.
protected virtual void Start();
```

- ```
// Sorts dock items by their X anchor position,
// selects the dock item with the specified Page Type,
// and initializes gamepad indicators.
// Called by MainMenuScreenBehavior in its Start method.
public virtual void Init(MainMenuPageType firstPageType);
```
- ```
// Returns the index of a dock item by its page type.  
public virtual int GetDockItemIndex(MainMenuPageType pageType);
```
- ```
// Called when a dock item is selected.
// Deselects all other dock items, invokes onPageSelected,
// and manages gamepad indicators.
protected virtual void OnPageSelected(MainMenuPageType pageType);
```
- ```
// Updates visibility of left and right gamepad indicators  
// based on active input type and current dock item position.  
protected virtual void CheckGamepadIndicators();
```
- ```
// Subscribed to GameController.InputManager.onInputChanged event.
// Updates gamepad indicators when input type changes.
protected virtual void OnInputChanged(Input.InputType prevInput,
Input.InputType newInput);
```
- ```
// Subscribed to  
GameController.InputManager.InputAsset.UI.Triggers.performed event.  
// Allows controlling the dock using gamepad trigger buttons.  
protected virtual void  
OnTriggersPerformed(UnityEngine.InputSystem.InputAction.CallbackCont  
ext ctx)
```
- ```
// Unsubscribes from input events when destroyed.
protected virtual void OnDestroy()
```

## Dock Item Overview

Each **Dock Item** represents one button in the Dock and is linked to a specific **Main Menu Page**.

**Dock Item Behavior inspector.**



The **Dock Item** Inspector has the following fields:

- **Page Type** - Enum linking this dock item to a specific main menu page.
- **Button** Reference to the `Button` component. When pressed, it selects this dock item and opens the corresponding page. The `onClick` event is assigned via code.
- **Page Name Text** - Reference to the `TMP_Text` component displaying the page name. It is only visible when the item is selected.
- **Icon Image** - Reference to the `Image` component that is animated on selection and deselection.
- **Selected Icon Size** - Icon size when selected. Should be slightly larger than when unselected.
- **Unselected Icon Size** - Icon size when not selected.
- **Selected Icon Position** - Anchored position of the icon when selected (usually slightly higher).
- **Unselected Icon Position** - Anchored position of the icon when not selected.

You can freely adjust these sizes and positions to better fit your desired UI style.

## Dock Item Behavior script

The `MainMenuDockItemBehavior` script controls the visuals and interactions of a single dock item.

It defines two properties, one event, and four methods.

#### Properties:

- ```
// Links the Dock Item to a Main Menu Page.  
public MainMenuPageType PageType => pageType;
```
- ```
// Indicates whether this dock item is currently selected.
public bool IsSelected { get; protected set; } = false;
```

#### Event:

- ```
// Invoked when the dock item is clicked and becomes selected.  
public event UnityAction<MainMenuPageType> onSelected;
```

Methods:

- ```
// Subscribes OnButtonClicked to the button's onClick event.
protected virtual void Awake()
```
- ```
// Plays the deselection animation (makes icon smaller and lower,  
hides text).  
public virtual void Select()
```
- ```
// Plays deselect animation. Makes icon smaller, lower, and hides
text
public virtual void Deselect()
```
- ```
// Called when the dock item is clicked.  
// Calls Select(), invokes onSelected event, and plays button click  
sound.  
protected virtual void OnButtonClicked()
```